云计算的处理对象：大数据

- 什么是大数据

  海量数据或巨量数据，其规模巨大到无法通过目前主流的计算机系统在合理时间内获取、存储、管理、处理并提炼以帮助使用者决策。

- 大数据4V+1C特征
  - Volume (数据量大)：PB/EB 级别成为常态 。
  - Velocity (快速)：数据产生与处理要求极速响应 。
  - Variety (多样)：结构化、半结构化、非结构化混合 。
  - Value (价值密度低)：需从海量沙砾中淘金 。
  - Complexity (复杂度高)：分析难度呈指数上升

云计算的定义

- 根据NIST (美国国家标准与技术研究院) 定义：
- 云计算是一种计算模型，允许无处不在的、便捷的、按需的网络访问
- 具备共享的可配置计算资源池（包括网络、服务器、存储、应用和服务）
- 资源能够用最小的管理代价或服务提供商的交互实现快速配置和发布

NIST
National Institute of Standards and Technology

"Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. "

云计算的本质特征

- 按需自服务
- 弹性扩展
- 资源池化
- 网络访问
- 按量计费

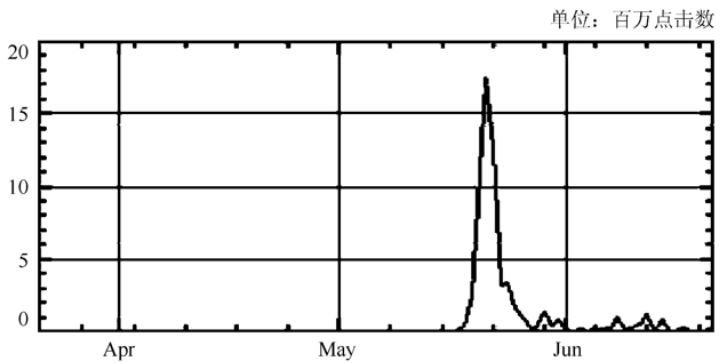
云计算的本质

- 按需自服务
  - 用户可以根据单方面需要，自动配置计算能力（如服务器时间和网络存储）
  - 整个过程无需与每个服务提供商进行人工交互
  - 高度自动化的 API 调用是实现按需自服务的基础

云计算的本质

弹性扩展

- 能力可以快速、弹性地提供，在某些情况下可以自动横向扩展 (Scale-out) 或缩减 (Scale-in)
- 对于消费者而言，可供配置的能力看起来是无限的
- 完美契合流量突发型业务（如电商大促、突发热点事件）



某典型网站的流量数据

云计算的本质

- 资源池化
  - 多租户（multi-tenancy）
  - 上万台服务器被融合成一个巨大的计算资源池
  - 供应商的计算资源被池化以服务于多个消费者
  - 互相隔离、互不干扰

云计算的本质

- 网络访问
  - 能力可通过标准机制在网络上获取，支持异构客户端
- 按量计费
  - 云系统通过在某种抽象级别（如存储、处理、带宽或活动用户帐户）提供计量能力，自动控制和优化资源使用
  - 经济优势： 硬件成本、电价和管理费用全面下降
  - 资源利用率从传统模式的 10%~15% 提升至 80% (5~7倍提升)

云计算中心如何选址?

- 超大规模带来管理成本显著下降（单管理员可管理超1000台服务器）
- 最大物理开销：电力和制冷成本
- 选址关键：寻找便宜的电力和自然冷源
  - BAT到贵州山区建立云计算中心

价格	地点	可能的定价原因
3.6美分	爱达荷州	水力发电，没有长途输送
10.0美分	加州	电力在电网上长途输送，湾区的输送线有限，加州不允许煤电
18.0美分	夏威夷	发电的能源需要海运到岛上

云计算的服务模式

- Local Deployment：自己盖房子
  在本地运行应用，拥有从硬件到应用以及扩缩的所有资源
- IaaS：聘请工人盖房子
  租用运行应用的硬件，但需负责管理操作系统、运行时、扩缩和所有数据
- PaaS：租房自带水电气与家具
  自带代码并部署，但将服务器管理和纵向扩容交给云服务提供商
- SaaS：拎包入住-日常维护
  支付费用以使用特定用途的完整应用，仅需负责自己的数据

让我们以房屋的例子类比！

基础设施即服务（IaaS）

- IaaS： 将多台服务器组成的“云端”服务（内存、I/O、存储、计算能力）作为计量服务提供。
- 优势
  - 消除前期购买硬件设备的资金投入
  - 更好的灾难恢复
  - 专注于开发核心业务
- 典型产品：
  - Amazon EC2 (提供可调整大小的计算容量)
  - Amazon S3 (对象存储)

平台即服务（PaaS）

- PaaS：将服务器平台或者开发环境作为服务进行提供
- 优势
  - 减少OS、数据库、运行环境配置的开销
  - 只需要管理最上层的应用和数据
- 典型产品：
  - Google App Engine
  - Microsoft Azure

软件即服务（SaaS）

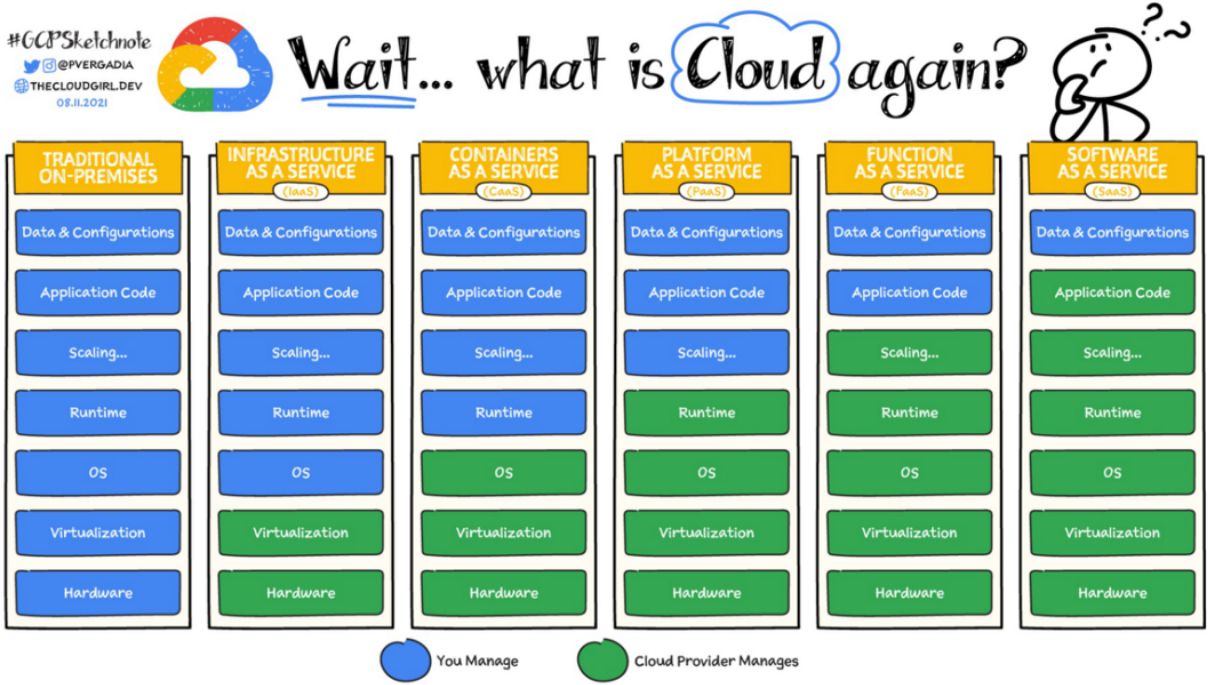
- SaaS：通过Internet提供软件的模式，用户无需下载软件客户端，而是向提供商租用基于Web的软件
- 优势：
  - 可以使用先进的应用程序
  - 只为自己使用的东西付费
  - 免客户端软件使用
  - 轻松增强员工移动性
  - 从任何位置访问应用数据
- 典型产品：
  - 文档协同、在线会议、企业OA…

三种云交付模型的比较

云交付模型	服务对象	使用方式	关键技术	用户的控制等级	系统实例
IaaS	需要硬件资源的用户	使用者上传数据、程序代码、环境配置	虚拟化技术、分布式海量数据存储等	使用和配置	Amazon EC2、Eucalyptus等
PaaS	程序开发者	使用者上传数据、程序代码	云平台技术、数据管理技术等	有限的管理	Google App Engine、Microsoft Azure、Hadoop等
SaaS	企业和需要软件应用的用户	使用者上传数据	Web服务技术、互联网应用开发技术等	完全的管理	Google Apps、Salesforce CRM等

注！随着云计算的深化发展，不同云计算解决方案之间相互渗透融合，同一种产品往往横跨两种以上的类型。

新兴服务模式：FaaS和CaaS



云计算的部署模式

部署模式	备注
私有云-Private Cloud	企业利用自有或租用的基础设施资源自建的云
公有云-Public Cloud	出租给公众的大型的基础设施的云
社区云-Community Cloud	为特定社区或行业所构建的共享基础设施的云
混合云-Hybrid cloud	由两种或两种以上部署模式组成的云

云计算的部署模式

部署模式	优点 (Pros)	缺点 (Cons)	典型适用场景
公有云	• 零初始硬件成本 • 极致弹性扩展 • 无需底层运维	• 数据控制权较弱 • 存在隐私与安全顾虑	互联网应用、初创企业、突发流量业务
社区云	• 多方均摊成本 • 满足特定行业的定制化与安全合规标准	• 参与方管理协调成本较高 • 扩展性不及公有云	医疗数据联盟、高校科研算力共享、政务云
私有云	• 最高的安全与隐私 • 满足严苛合规要求 • 底层资源完全可控	• 初始采购成本极高 • 需专门IT团队运维 • 扩展受限于物理硬件	政府机构、大型金融机构、核心机密系统
混合云	• 兼顾安全与弹性 • 优化整体IT成本	• 系统架构最复杂 • 跨云网络互通与数据同步难	核心稳态数据留存本地 + 突发流量溢出至公有云

云计算的三大基础设施

云核心技术栈：计算

云怎么把一台物理机切分给多个用户?

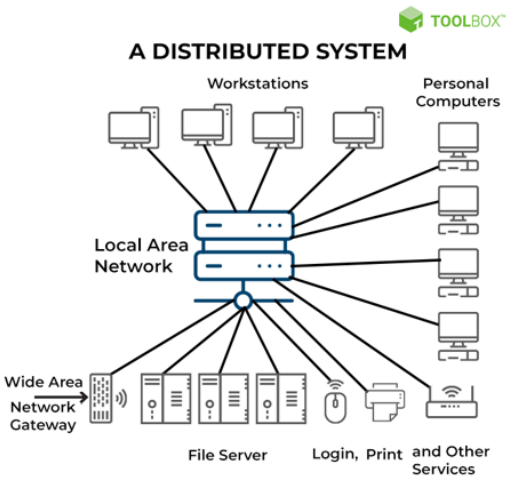
- 虚拟化技术 (Virtualization)
  - 通过 Hypervisor 层，将一台物理机分成多台拥有独立操作系统的虚拟机
  - 隔离性极强，安全性高，但相对笨重（每个VM都要跑完整的OS）
- 容器技术 (Containers - 如 Docker)
  - 多个容器共享同一个宿主机的操作系统内核，只隔离应用及其依赖库
  - 极致轻量、启动达到秒级/毫秒级，现代云原生架构的基石

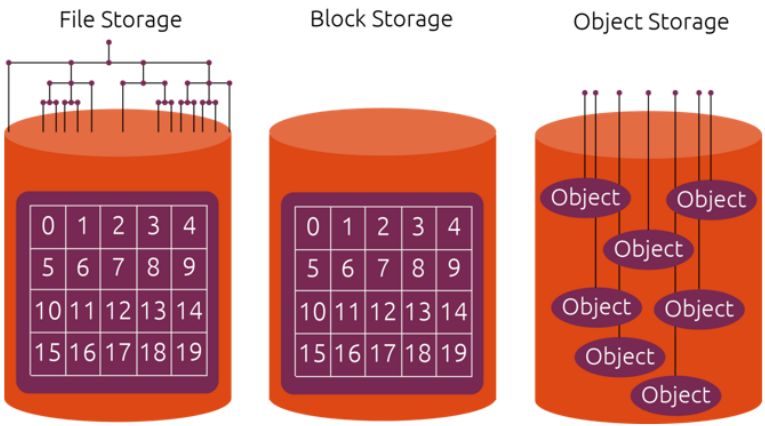
云核心技术栈：存储

云计算怎么存储庞杂数据？

- 分布式存储 (Distributed Storage)
  - 将海量数据分散存储在多台服务器节点上，通过网络将其组合成一个巨大的逻辑磁盘
  - 极高的吞吐量和容错率（多副本机制）
- 对象存储 (Object Storage)
  - Amazon S3 / 阿里云 OSS
  - 抛弃了传统文件系统的目录树结构，采用扁平化命名空间。数据和元数据打包成一个对象，通过唯一的 URL (API) 进行访问。
  - 容量几乎无限扩展，适合存储非结构化数据（图片、视频、备份文件）

云核心技术栈：存储

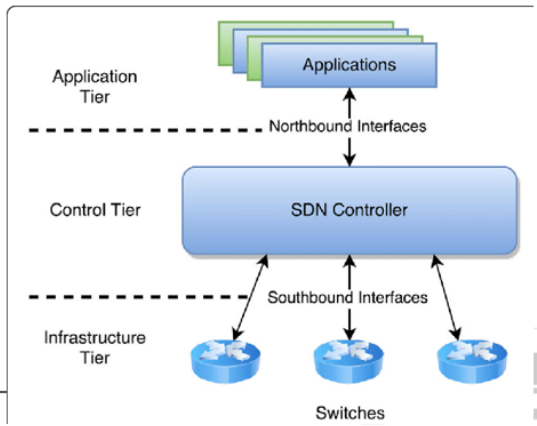

TOOLBOX
A DISTRIBUTED SYSTEM
Workstations
Personal Computers
Local Area Network
Wide Area Network Gateway
File Server
Login, Print and Other Services


File Storage
Block Storage
Object Storage
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19
Object
Object
Object
Object
Object
Object
Object

云计算的基础设施：网络

云端海量服务器怎么用网络管理起来?

- 软件定义网络 (SDN - Software-Defined Networking):
  - 将网络的控制面与数据面分离，通过纯软件编程的方式来动态管理和配置网络资源
- 弹性负载均衡 (Load Balancing):
  - 面对突发海量并发请求，将外部请求均匀分发到多个计算节点上



西南交通大学
Southwest Jiaotong University

虚拟化技术与Docker

CLOUD COMPUTING

丛荣

西南交通大学
计算机与人工智能学院

虚拟化的定义

- 虚拟化就是Virtualization
- 虚拟化是一种资源管理技术
- 实体资源：CPU、内存、磁盘、网络...
- 虚拟化：让每个进程看起来拥有单独的硬件资源
- 虚拟机：由虚拟化层提供的高效独立的虚拟计算机系统
- 从软件角度来看，虚拟机=真实机器

虚拟化（Virtualization）是一种资源管理技术，是将计算机的各种实体资源（CPU、内存、磁盘空间、网络适配器等），予以抽象、转换后呈现出来并可供分割、组合为一个或多个电脑配置环境。由此，打破实体结构间的不可切割的障碍，使用户可以比原本的配置更好的方式来应用这些电脑硬件资源。这些资源的新虚拟部分是不受现有资源的架设方式，地域或物理配置所限制。

虚拟化带来的优势

- 资源利用率：资源动态分配和调度
- 运维成本：自动化开通能力
- 升级迁移：方便系统的平滑升级与跨硬件迁移
- 测试生产：安全的隔离环境

硬件虚拟化V.S.软件虚拟化

如何实现虚拟化（怎么骗过OS）？

- 软件虚拟化
  - 通过虚拟机监控器 (VMM) 捕获并翻译特权指令
  - 兼容性好
  - 系统性能开销较大，实现逻辑复杂
  - KEMU、VMware

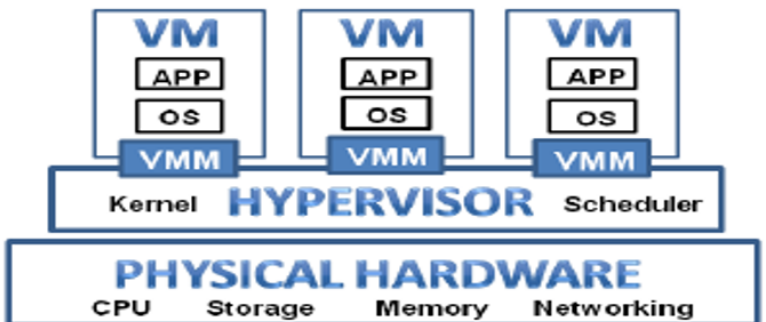
硬件虚拟化V.S.软件虚拟化

如何实现虚拟化（怎么骗过OS）？

- 硬件虚拟化
  - 依靠 CPU 等底层硬件架构级的扩展支持，增加一套专门给虚拟机的特权指令通道（Ring-1）
  - 无需软件干预，性能极高
  - 和硬件设计绑定
  - Intel VT-x, AMD-v

虚拟化核心组件：Hypervisor/VMM

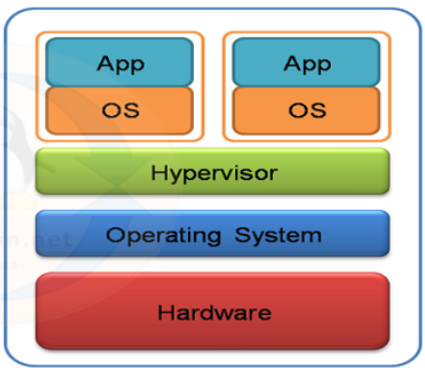
- 职责1：物理硬件资源→虚拟资源的逻辑映射
- 职责2：虚拟机的上下文隔离，保障多租户环境安全
- 运行虚拟机时，Hypervisor 中的 VM kernel 会装载 VMM



Hypervisor典型架构

- 寄居虚拟化架构 (Hosted Virtualization)
  - Hypervisor 作为普通进程运行在Host OS之上
  - 例如：Vmware（个人PC）
  - 硬件兼容性好、部署方便
  - 系统开销大

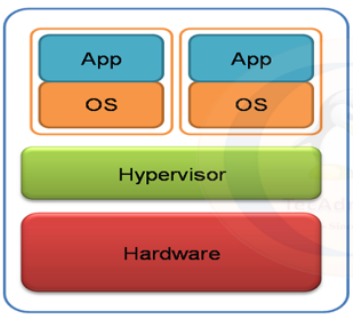
    I/O 请求需经过 Host OS 转发，上下文切换频繁



Type 2 (Hosted) Virtualization

Hypervisor典型架构

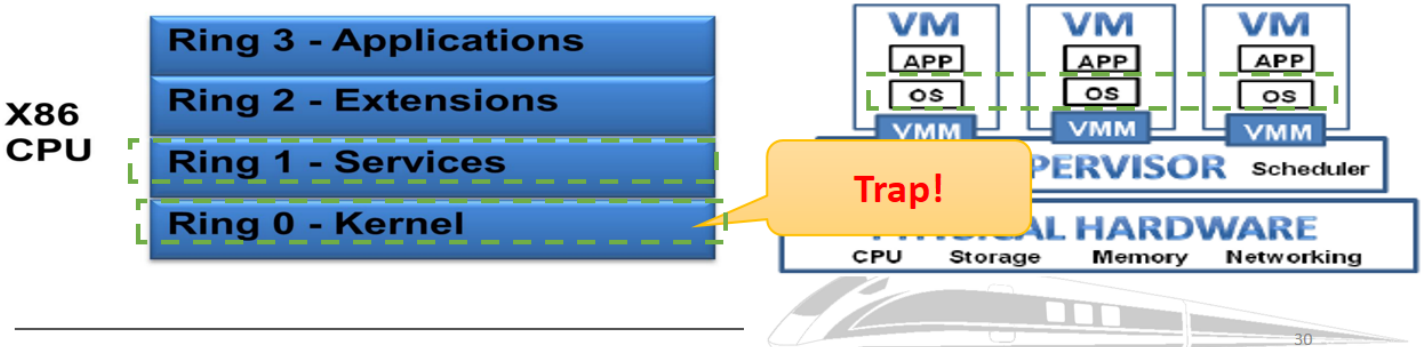
- 裸机虚拟化架构 (Bare-metal Virtualization)
  - Hypervisor 直接运行于裸机硬件之上
  - 实现从虚拟资源到物理资源的映射
  - 例如：KVM（数据中心）
  - 省去底层 Host OS 的中间损耗
  - 系统性能最接近物理机
  - 硬件兼容性要求严格、部署难度大



Type 1 (Bare Metal) Virtualization

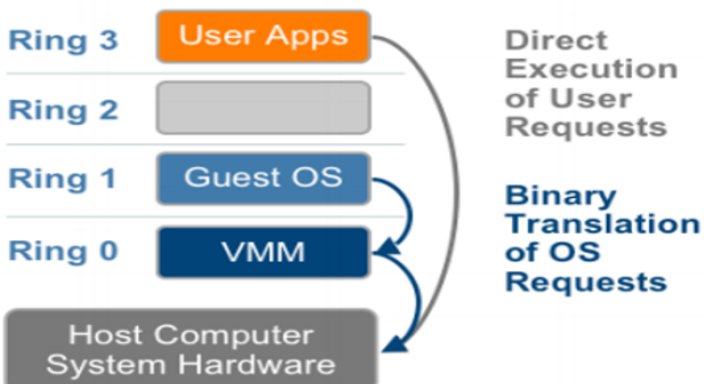
CPU虚拟化的原理

- CPU 虚拟化
  - VT-x: Intel X86
  - VT-i: Intel Itanium
  - AMD-v: AMD
- CPU指令分级



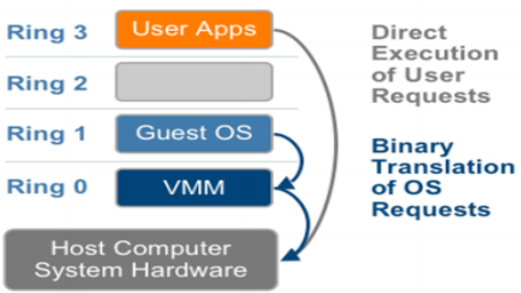
CPU虚拟化的原理

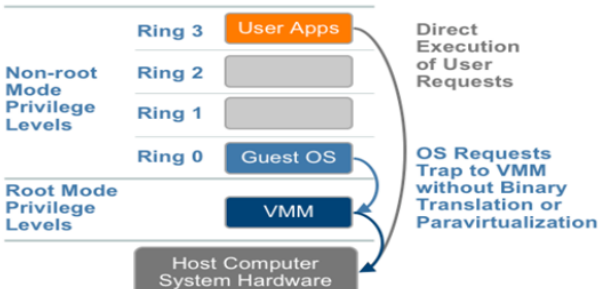
- 怎么解决CPU指令的权限冲突?
  - 软件虚拟化：VMM截获Ring0指令并转化
  - 硬件虚拟化：在CPU芯片直接支撑虚拟化指令



CPU虚拟化的原理

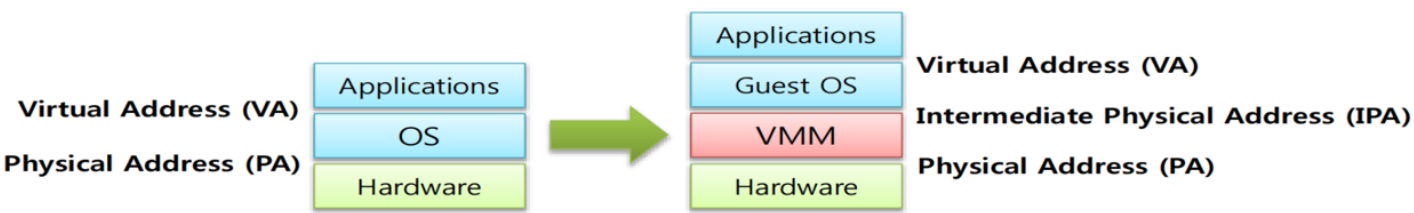
- VT-x引入了两种操作模式
  - **根操作模式（Root）**：VMM 运行所处的模式
  - **非根操作模式 （Non-Root）**：VM运行所处的模式
  - Non-Root模式也有一套Ring0-Ring3，Guest OS可以直接使用这套指令（**硬件直接运行**）
  - 只有当Guest OS试图执行影响物理机硬件的指令时，才会将控制权交给VMM转换





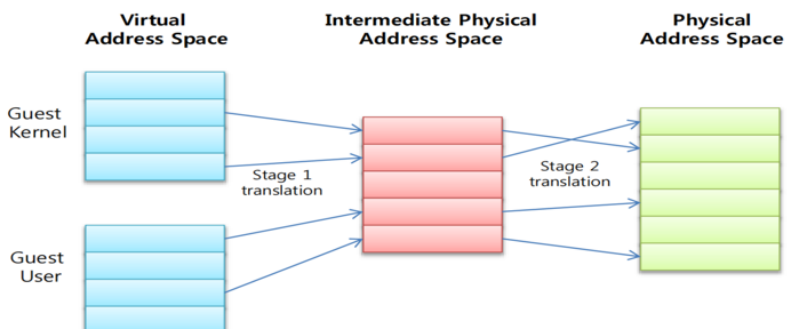
内存虚拟化的原理

- 内存地址转换：
  - 物理机：虚拟地址 → 物理地址
  - 虚拟机：虚拟地址 →Guest伪物理地址 →物理地址



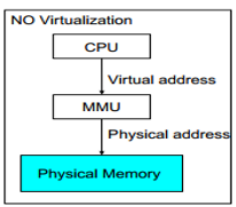
内存虚拟化的原理

- 怎么完成两次内存地址转换?
  - 影子页表（Shadow Page Table，SPT）
    - Hypervisor自己维护一个影子页表
    - 记录Guest伪物理地址到物理地址的转换
    - 拦截、更新、同步所有的内存转换
    - 传递转换后的地址给物理机CPU
  - 局限：两次转换，内存读写缓慢

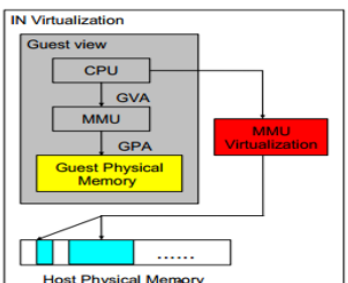


内存虚拟化的原理

- 怎么完成两次内存地址转换？
  - 软件：影子页表（Shadow Page Table，SPT）
  - 硬件：扩展页表（Extended Page Table，EPT）
    - 内存管理单元（MMU）进行两次地址转换
    - 硬件查询维护，效率高速度快

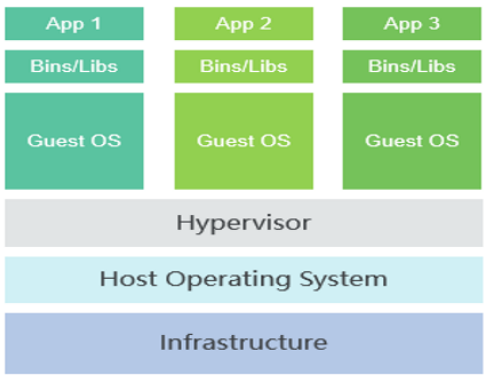


GVA: guest virtual address
GPA: guest physical address



虚拟化技术的局限性

- 资源冗余：每一个虚拟机都必须运行一个完整Guest OS
- 弹性迟缓：分钟级的系统启动时间
- 交付困难：开发、测试与生产环境的底层依赖不一致问题



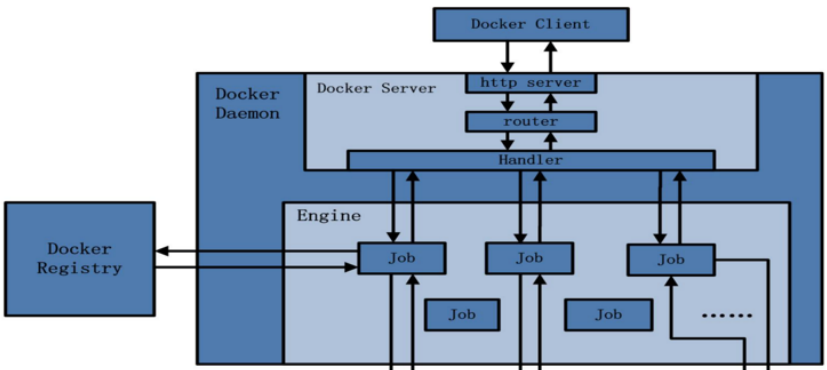
传统虚拟化

容器V.S.虚拟机

	虚拟机	容器
OS依赖	完整的独立操作系统 (Guest OS)	共享宿主机操作系统内核
启动速度	**分钟级** (需经历完整的系统引导)	**毫秒/秒级** (仅启动一个进程)
空间占用	**GB 级别** (包含海量系统文件)	**MB 级别** (仅包含业务代码与依赖)
性能损耗	**较大** (需通过 Hypervisor 转换)	**极小** (原生执行指令)
隔离级别	**极强** (硬件级隔离，高度安全)	**较弱** (进程级/Namespace逻辑隔离)

Docker架构

- Client/Server架构
- Docker Client
  - build, run
- Docker Daemon
  - 守护进程 dockered
- Docker Registry
  - 容器镜像仓库
  - pull, push, search



一条Docker指令会经历哪些流程?

Linux内核基石

Docker容器是怎么隔离进程的?

- 封装了Linux内核的API
  - Namespace 隔离
  - Cgroup 资源限制

Linux内核基石：Namespace

- Linux提供的一种内核级别环境隔离的方法
- include/linux/sched.h; include/linux/nsproxy.h；kernel/nsproxy.c
  - nsproxy 是进程（task_struct）到所有 Namespace 的中转站

```
struct task_struct {
    ……..
    /* namespaces */
    struct nsproxy *nsproxy;
    …….
}
struct nsproxy {
    atomic_t count;      // nsproxy可以共享使用，count字段是该结构的引用计数
    struct uts_namespace *uts_ns;
    struct ipc_namespace *ipc_ns;
    struct mnt_namespace *mnt_ns;
    struct pid_namespace *pid_ns_for_children; // 解决进程号隔离性
    struct net             *net_ns;            // 解决网络相关设备、路由表、socket等资源的隔离性
};
```

Linux内核基石：Namespace

- 容器内的进程运行的时候看不到宿主机的进程
  - PID Namespace : 进程隔离
  - NET Namespace: 网络隔离
  - MNT Namespace: 挂载/文件系统隔离)
  - UTS Namespace: 主机名隔离(Ubuntu-Server)
  - IPC Namespace: 进程间通信隔离（消息队列IPC）
  - USER Namespace： 用户权限隔离

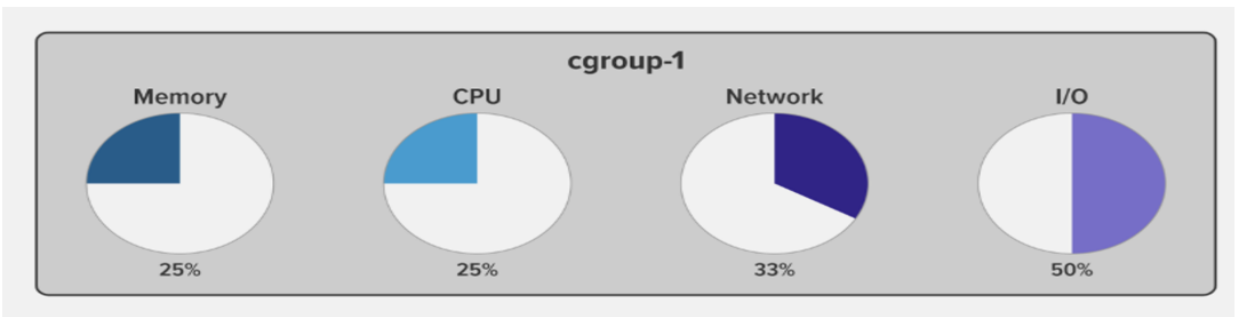
Linux内核基石：Namespace

```
struct nsproxy {
    atomic_t count;
    struct uts_namespace *uts_ns;
    struct ipc_namespace *ipc_ns;
    struct mnt_namespace *mnt_ns;
    struct pid_namespace *pid_ns_for_children;
    struct net           *net_ns;
};
```

Linux: clone(CLONE_NEWPID|CLONE_NEWNET)

Linux内核基石：Cgroup

- Linux提供的一种内核级别资源限制的方法
- Cgroup：control group
- 控制什么：一组进程的资源使用、进程起始、优先级
- /sys/fs/cgroup



Linux内核基石：Cgroup

- 可控制的资源有哪些?

```
# CentOS 8.2
$ mount -t cgroup
cgroup on /sys/fs/cgroup/systemd type cgroup (rw,nosuid,nodev,noexec,relatime,xattr,release_age
cgroup on /sys/fs/cgroup/rdma type cgroup (rw,nosuid,nodev,noexec,relatime,rdma)
cgroup on /sys/fs/cgroup/freezer type cgroup (rw,nosuid,nodev,noexec,relatime,freezer)
cgroup on /sys/fs/cgroup/cpu,cpuacct type cgroup (rw,nosuid,nodev,noexec,relatime,cpu,cpuacct)
cgroup on /sys/fs/cgroup/net_cls,net_prio type cgroup (rw,nosuid,nodev,noexec,relatime,net_cls,
cgroup on /sys/fs/cgroup/blkio type cgroup (rw,nosuid,nodev,noexec,relatime,blkio)
cgroup on /sys/fs/cgroup/memory type cgroup (rw,nosuid,nodev,noexec,relatime,memory)
cgroup on /sys/fs/cgroup/devices type cgroup (rw,nosuid,nodev,noexec,relatime,devices)
cgroup on /sys/fs/cgroup/pids type cgroup (rw,nosuid,nodev,noexec,relatime,pids)
cgroup on /sys/fs/cgroup/cpuset type cgroup (rw,nosuid,nodev,noexec,relatime,cpuset)
cgroup on /sys/fs/cgroup/hugetlb type cgroup (rw,nosuid,nodev,noexec,relatime,hugetlb)
cgroup on /sys/fs/cgroup/perf_event type cgroup (rw,nosuid,nodev,noexec,relatime,perf_event)
```

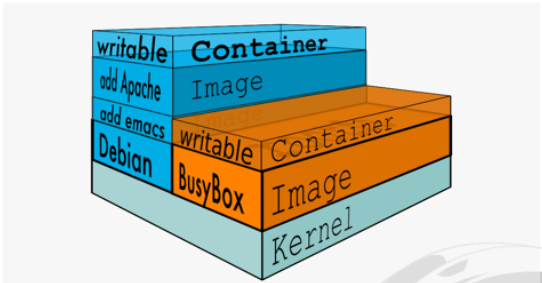
镜像与容器的分层结构

- 镜像层、容器层
  - 容器运行时会在镜像层只读层基础上添加可写层
  - 相同基础层（系统环境、语言库...）服务器仅存一次
  - 大规模镜像分发：网络和存储成本大幅降低

联合文件系统（UnionFS）

镜像层与容器层怎么动静结合联合使用?

- 将分属不同物理位置的只读目录合并，挂载至一个虚拟文件系统位置（挂载点），实际物理位置是分开的
- 容器启动后, Docker Engine在最顶端动态附加一层可写层
- 容器生命周期内，所有文件系统操作均发生于顶部可写层



西南交通大學
Southwest Jiaotong University

云计算技术

第3讲 云原生基础与微服务

授课教师：戴朋林

课程代码：SCAI004712

2026年3月

云原生的定义（CNCF）

CNCF（Cloud Native Computing Foundation）官方定义

云原生技术使组织能够在公有云、私有云和混合云等现代动态环境中，构建和运行可弹性扩展的应用。云原生的代表技术包括容器、服务网格、微服务、不可变基础设施和声明式API。这些技术能够构建容错性好、易于管理和便于观察的松耦合系统。结合可靠的自动化手段，云原生技术使工程师能够轻松地对系统进行频繁且可预测的重大变更。

- **Cloud：**应用长在云上（云计算的五大本质）
  - 按需自服务、弹性扩展、资源池化、网络访问、按量计费
- **Native：**应用从写代码的第一天起，就是为了适应云环境而设计

01 云原生的核心要素

动态环境：应用运行在多样的云基础设施上

弹性扩展：根据负载自动伸缩

容错性：系统能优雅地处理故障

松耦合：组件之间依赖最小化

自动化：减少人工干预，提高变更效率

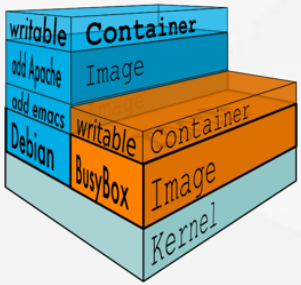
要素	定义	核心价值
容器化（Containerization）	应用及其依赖打包为容器镜像，确保环境一致性	可移植性、快速部署
微服务（Microservices）	将应用拆分为小型、独立的服务，每个服务专注于单一业务功能	独立开发、独立部署、独立扩展
声明式 API（Declarative APIs）	用户声明期望状态，系统自动实现和维护	简化运维、自动化管理
不可变基础设施（Immutable Infrastructure）	基础设施组件一旦部署不再修改，更新通过替换而非修补	环境一致性、可审计性

01 不可变基础设施

传统模式（可变）：在服务器上直接修改配置、安装补丁，易导致配置漂移，形成“雪花服务器”，使环境一致性难以保障。

云原生模式（不可变）：当需要变更时，通过构建新镜像并替换旧实例的方式完成更新，从根源上保证了环境的一致性与可重复性。

对比	可变基础设施	不可变基础设施
更新方式	SSH 登录服务器手动修改	构建新镜像，滚动替换
环境一致性	低（易出现配置漂移）	高（镜像统一）
回滚	困难	简单（直接使用旧版本镜像）
可审计性	弱	强（镜像版本化管理）



01 云原生 v.s. 传统开发模式

传统开发模式以单体架构、低频部署和人工运维为特征，而云原生模式通过微服务、容器化与自动化交付，实现了更敏捷、弹性和可靠的软件生命周期管理。

维度	传统开发模式	云原生模式
架构	单体应用（Monolith）	微服务架构
部署单元	WAR/EAR 包部署到应用服务器	容器镜像部署到 K8s
部署频率	每月 / 每季度	每天多次
扩展方式	垂直扩展（加 CPU / 内存）	水平扩展（加实例）
环境管理	手动配置、脚本	声明式配置（IaC）
故障处理	人工排查、手动恢复	自动检测、自动恢复
开发流程	瀑布式 / 长周期迭代	CI/CD、DevOps
基础设施	物理机 / 虚拟机	容器 / Serverless
状态管理	有状态（Session）	无状态（Stateless）

“这些技术能够构建容错性好、易于管理和便于观察的**松耦合系统**”
——CNCF定义

微服务的核心特征

微服务架构（Microservices Architecture） 是一种将应用程序构建为一组小型、自治的服务的架构风格。每个服务围绕特定业务能力构建，独立部署，通过轻量级机制（通常是HTTP API）进行通信。 ——Martin Fowler & James Lewis, 2014

特征	说明
单一职责	**每个服务只负责一个明确的业务功能**
独立部署	**服务可以独立构建、测试和部署，不影响其他服务**
独立扩展	根据各服务的负载独立进行水平扩展
技术异构	不同服务可使用不同的编程语言和数据存储
轻量级通信	**服务间通过 REST API、gRPC 或消息队列通信**
去中心化治理	各团队自主选择技术栈和开发流程
去中心化数据	每个服务拥有自己的数据库（Database per Service）
容错设计	服务需要考虑依赖故障的处理（熔断、降级、重试）

如何确定服务边界？

1. 按业务领域拆分（DomainDriven Design, DDD）

- 基于领域驱动设计的限界上下文（Bounded Context）
- 每个限界上下文对应一个微服务
- 例：电商系统：用户服务、商品服务、订单服务、支付服务、库存服务

2. 按数据边界拆分

- 每个服务拥有自己的数据库
- 服务间不直接访问对方数据库
- 通过API或事件进行数据交换

3. 按变更频率拆分

- 变更频率高的功能独立为服务
- 稳定的功能保持在一起

微服务通信方式——同步通信

- REST：
  - 前端调用后端（对外API）
  - 基于 HTTP/JSON
  - 人类易读、简单通用、生态成熟、浏览器支持好
- gRPC：
  - 微服务A调用微服务B（对内API）
  - 基于 HTTP/2 和 Protocol buffer（二进制）
  - 体积小、传输快、更安全、适合微服务内部高性能通信

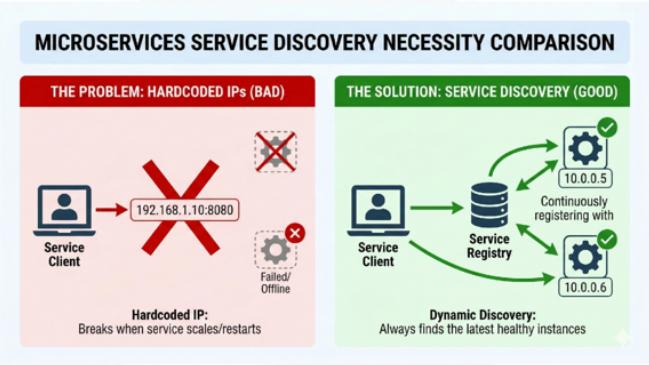
特性	REST (Representational State Transfer)	gRPC (Google Remote Procedure Call)
协议	HTTP/HTTPS	HTTP/2
数据格式	JSON（主流）、XML	Protocol Buffers（二进制，高效）
设计风格	资源导向（GET/POST/PUT/DELETE）	方法调用导向（RPC）
优点	简单、通用、生态成熟	高性能、强类型、双向流、多语言代码生成
缺点	文本协议，序列化开销大；缺乏类型安全	学习曲线较高，调试不如 REST 直观

消息队列（Message Queue）

异步通信以消息队列为核心，通过 Kafka、RabbitMQ 等消息中间件，将生产者发送的消息暂存并异步交付给消费者处理，实现服务间的解耦；它具备削峰填谷、保障最终一致性的优势，能有效提升系统的弹性与可用性，适用于事件通知、异步处理及跨服务数据同步等场景。

为什么需要服务发现？

- 在单体时代，数据库IP是写死在配置文件里的
- 微服务架构中，服务实例的IP地址和端口动态变化
- 服务可能随时扩缩容、故障重启、迁移到新节点
- 硬编码地址行不通，需要动态发现服务位置



客户端发现（ClientSide Discovery）

服务 A 发起调用请求，经负载均衡器或 API 网关路由，查询服务注册中心获取可用实例，再将请求转发至服务 B 完成通信。

- 代表：Kubernetes Service + kubeproxy
- 优点：架构简单，无额外网络跳转，延迟低
- 缺点：客户端需集成服务发现与负载均衡逻辑，与语言耦合

服务端发现（Server-Side Discovery）

服务 A 直接向负载均衡器 / API 网关发起请求，由网关或负载均衡器查询服务注册中心，获取服务 B 的可用实例列表，再通过负载均衡策略将请求转发到目标实例完成通信。

- 代表：Nginx、Spring Cloud Gateway、Kubernetes Service + kube-proxy
- 优点：客户端逻辑简单，与开发语言无关，负载策略统一在网关侧维护
- 缺点：多了一层网络跳转，会引入额外延迟，且网关 / 负载均衡器本身需要高可用保障

对比维度	客户端发现（Client-Side Discovery）	服务端发现（Server-Side Discovery）
核心流程	服务 A 直接查询注册中心，获取服务 B 实例列表，本地负载均衡后直接发起调用	服务 A 将请求发送给负载均衡器 / API 网关，由网关查询注册中心并完成负载均衡，再转发到服务 B 实例
代表实现	Eureka + Ribbon、Consul + 客户端负载均衡	Nginx、Spring Cloud Gateway、Kubernetes Service + kube-proxy
优点	架构简单，无额外网络跳转，延迟低	客户端逻辑极简，语言无关，负载策略统一维护，便于管控
缺点	客户端需集成服务发现与负载均衡逻辑，与语言耦合	多一跳网络延迟，网关 / 负载均衡器成为潜在单点，需高可用保障

主流服务注册中心在一致性模型与能力侧重上各有特点：

- etcd 与 Consul 均基于 Raft 实现强一致性，前者是 Kubernetes 的默认组件，专注键值存储；后者则内置了服务发现、健康检查等全栈能力。
- ZooKeeper 作为老牌组件，通过 ZAB 协议保证强一致性，在配置管理场景中应用广泛。
- Nacos 则支持 AP/CP 模式切换，由阿里开源，兼顾了服务发现与配置管理的双重需求，更适配云原生微服务架构。

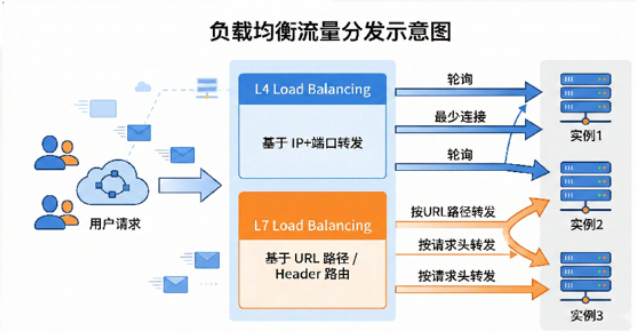
注册中心	一致性模型	特点
etcd	强一致（Raft）	K8s 默认使用，键值存储
Consul	强一致（Raft）	内置服务发现、健康检查、KV 存储
ZooKeeper	强一致（ZAB）	老牌，配置管理
Nacos	AP/CP 可切换	阿里开源，服务发现 + 配置管理

负载均衡策略

负载均衡（Load Balancing）是一种将网络流量或工作负载分发到多个服务实例上的技术，旨在提高可用性、提升性能和避免单点过载。

算法	原理	适用场景
轮询（Round Robin）	按顺序依次分发到每个实例	实例性能相同
加权轮询（Weighted RR）	根据实例权重（性能）按比例分发	实例性能不均
最少连接（Least Connections）	分发到当前连接数最少的实例	长连接场景
随机（Random）	随机选择实例	简单场景
一致性哈希（Consistent Hash）	相同请求哈希到相同实例	缓存命中率优化
IP 哈希	按客户端 IP 哈希选择实例	Session 亲和性

05 负载均衡层次



层次	技术	说明
L4负载均衡	TCP/UDP层	基于IP+端口转发，性能高
L7负载均衡	HTTP层	基于URL路径、Header等路由，功能丰富

- L4 负载均衡
  - TCP/UDP 层
  - 基于 IP 和端口进行转发
  - 速度快，只基于IP和端口
  - 轮询、最少连接等算法
- L7 负载均衡
  - HTTP 层
  - 智能路由：根据 URL 路径、请求头等
  - 功能更丰富灵活，满足复杂的业务需求

问题：级联故障

在微服务架构中，服务A调用服务B，服务B调用服务C。

如果服务C崩溃：

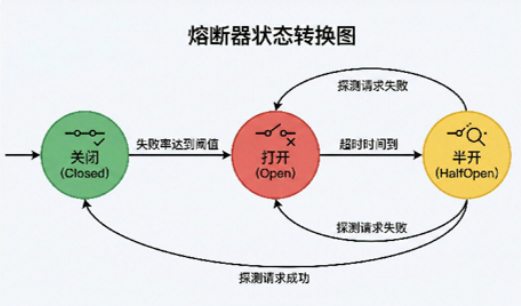
- 服务B等待C响应 → 线程阻塞 → 线程池耗尽
- 服务A等待B响应 → 同样阻塞
- **雪崩效应（Cascading Failure）**：一个服务故障导致整个链路崩溃

06 熔断器模式原理

熔断器模式（Circuit Breaker Pattern）受电路断路器启发。当检测到下游服务故障达到阈值时，自动"断开"对该服务的调用，直接返回降级响应，防止故障扩散。

- 熔断器模式的三种状态
  - 关闭（closed）：正常通信
  - 打开（opened）：触发熔断机制，返回fast-fail，停止调用下游微服务
  - 半开（halfopen）：尝试通过少量请求，探测下游服务是否恢复

状态	说明	行为
关闭（Closed）	正常状态	所有请求正常转发；统计失败率
打开（Open）	熔断触发	所有请求直接返回降级响应；不调用下游
半开（HalfOpen）	尝试恢复	允许少量请求通过；成功则关闭熔断，失败则继续打开



熔断器通过三种核心状态的切换，实现对服务调用的保护：

Closed（关闭状态）

这是熔断器的常态，服务调用正常进行。当调用失败率达到预设阈值时，熔断器会立即切换到 Open（打开） 状态，快速熔断，防止故障扩散。

Open（打开状态）

此时，所有对目标服务的调用都会被直接拒绝，并快速返回错误。经过一段超时时间后，熔断器会自动进入 HalfOpen（半开） 状态，试探性地检查服务是否恢复。

HalfOpen（半开状态）

熔断器会允许少量探测请求通过。

如果探测请求成功，说明服务已恢复，熔断器将恢复到 Closed 状态。

如果探测请求失败，说明服务仍不可用，熔断器将回到 Open 状态，继续保护调用方。

熔断器实现与实践——关键参数

熔断器的核心参数共同定义了其容错行为：

- 失败率阈值：触发熔断的失败比例，典型值为 50%，即失败请求占比过半时熔断。
- 统计窗口：用于统计失败率的时间范围，通常为 10 秒，确保判断基于近期状态。
- 最少请求数：触发熔断所需的最小请求量，如 20 次，避免少量波动误判。
- 熔断持续时间：Open 状态的持续时长，如 30 秒，给服务恢复留出时间。
- 半开允许请求数：HalfOpen 状态下允许通过的探测请求数，如 5 次，用于验证服务是否恢复。
- 这些参数协同工作，让熔断器既能快速响应故障，又能稳妥地尝试恢复服务。

参数	说明	典型值
失败率阈值	触发熔断的失败比例	50%
统计窗口	统计失败率的时间窗口	10秒
最少请求数	触发熔断需要的最少请求量	20次
熔断持续时间	Open状态持续时间	30秒
半开允许请求数	HalfOpen状态允许通过的请求数	5次

与其他容错模式配合

- 重试（**Retry**）：短暂故障时自动重试（配合指数退避）
- 超时（**Timeout**）：设置调用超时上限，防止长时间阻塞
- 舱壁（**Bulkhead**）：资源隔离，不同服务使用独立的线程池/连接池
- 降级（**Fallback**）：熔断触发后返回默认值或缓存数据

限流（Rate Limiting）：通过限制单位时间内的请求数量来保护系统免受过载

常见限流算法		
算法	原理	特点
固定窗口计数器	时间窗口内计数，超过阈值拒绝	简单，但有窗口边界突刺问题
滑动窗口计数器	滑动时间窗口，平滑限流	更平滑，实现稍复杂
令牌桶（Token Bucket）	固定速率生成令牌，请求消耗令牌	允许突发流量（桶满时）
漏桶（Leaky Bucket）	请求进入桶，以固定速率流出	严格平滑流量

07 降级策略（Degradation）

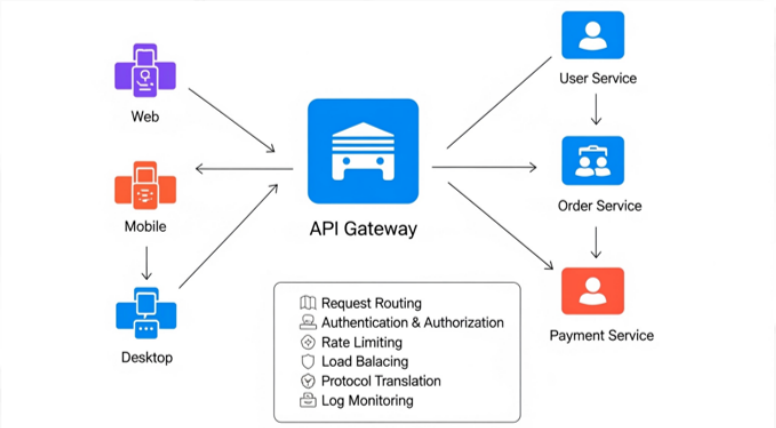
降级：当系统压力过大或依赖服务故障时，主动降低服务质量以保证核心功能可用

策略	说明	示例
返回默认值	返回预设的默认响应	推荐服务降级 → 返回热门商品
返回缓存	返回上次成功的缓存数据	用户画像服务降级 → 返回缓存画像
功能裁剪	关闭非核心功能	高峰期关闭评论、推荐功能
流量排队	请求排队等待处理	秒杀场景排队机制

API网关

- API网关（API Gateway） 是微服务架构中的统一入口，负责接收所有客户端请求，进行路由、认证、限流、协议转换等处理后，将请求转发到对应的后端微服务。

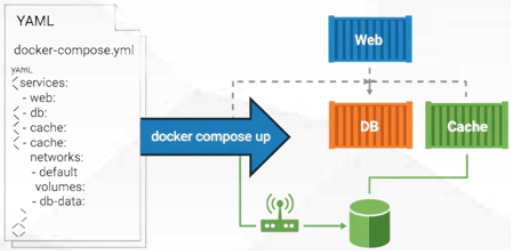
功能	说明
请求路由	根据URL路径将请求路由到对应微服务
认证鉴权	统一的身份验证和权限校验
限流熔断	保护后端服务免受过载
负载均衡	将请求分发到多个服务实例
协议转换	外部REST → 内部gRPC
请求聚合	将多个微服务的响应聚合为一个
日志监控	统一的请求日志和指标采集



Docker Compose 核心概念

- Docker Compose 是一个用于定义和运行多容器Docker应用的工具
  - 使用YAML文件描述应用的服务、网络和卷
  - 一次声明定义，一键拉起整个集群
  - ~~Docker run~~

    Docker –compose up



Define once, run anywhere

03 Compose文件核心结构

顶级key	说明
services	定义应用包含的服务（容器）
networks	定义自定义网络
volumes	定义持久化卷
configs	定义配置文件
secrets	定义敏感信息

Compose 文件的核心结构由五大顶级 key 构成，它们各司其职，共同定义了整个多容器应用的编排规范：

- **services**：定义应用所包含的所有服务（容器），是整个文件的核心，用于配置每个服务的镜像、端口、环境变量等。
- **networks**：定义应用所需的自定义网络，实现服务间的隔离与通信，确保容器在安全可控的网络环境中交互。
- **volumes**：定义数据持久化卷，用于保存容器运行时产生的数据，避免容器删除后数据丢失。
- **configs**：定义配置文件，将配置与镜像解耦，方便在不重建镜像的情况下更新应用配置。
- **secrets**：定义敏感信息（如密码、密钥），通过安全的方式管理和传递敏感数据，提升应用的安全性。

这些顶级 key 协同工作，让复杂的多容器应用部署变得清晰、可维护且可复现。

03 服务（Service）配置选项

选项	说明	示例
image	使用已有镜像	image: nginx:alpine
build	从Dockerfile构建	build: ./api
ports	端口映射	ports: ["8080:80"]
environment	环境变量	environment: [DB_HOST=db]
volumes	数据卷挂载	volumes: [./data:/app/data]
depends_on	服务依赖关系	depends_on: [db, redis]
networks	加入的网络	networks: [frontend, backend]
deploy	部署配置	副本数、资源限制
healthcheck	健康检查	命令、间隔、超时

Docker Compose 中 services 作为核心配置模块，通过九大高频选项实现对容器服务的全维度定义，覆盖镜像获取、构建、网络、数据、依赖及运维管控等核心场景：

镜像与构建：image直接指定已有镜像快速启动容器，build则通过本地Dockerfile自行构建镜像，灵活适配不同部署需求；

网络与端口：ports实现容器端口与宿主机的映射，networks 为服务分配自定义网络，保障服务间通信的隔离性与可控性；

数据与环境：volumes挂载数据卷实现持久化存储，environment 配置服务运行所需的环境变量，实现配置与镜像的解耦；

依赖与运维：depends_on定义服务启动的依赖顺序，deploy配置副本数、资源限制等部署策略，healthcheck则通过自定义规则监控服务健康状态，确保容器稳定运行。

默认网络行为

- Compose会自动创建一个**默认网络**（项目名_default）
- 同一网络中的容器可以通过**服务名**直接通信（Docker内置DNS）
- 例：userservice容器可以通过 userdb:5432 访问数据库

在 Docker Compose 环境中，网络与服务发现机制为微服务间的通信提供了极大的便利：

- 自动网络管理：Compose 会为每个项目自动创建一个默认网络（命名规则为 项目名_default），所有服务默认加入此网络。
- **内置 DNS 服务发现：**在同一网络内，容器之间无需知晓对方的 IP 地址，可直接通过服务名进行通信。Docker 内置的 DNS 服务器会自动将服务名解析为对应容器的内部 IP。
- 便捷通信示例：例如，userservice 容器可以直接通过 userdb:5432 这样的地址访问数据库服务，无需手动配置 IP，大大简化了服务间的调用配置。

西南交通大學
Southwest Jiaotong University

云计算技术

第4讲 Kubernetes基础

授课教师：戴朋林

课程代码：SCAI004712

2026年3月

01 Kubernetes整体架构：Master-Worker模型

集群组成

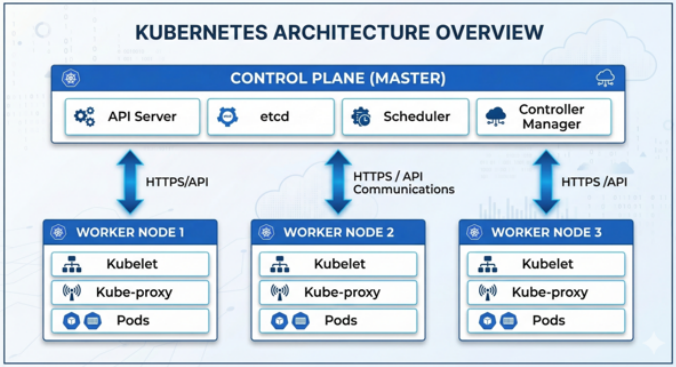
Kubernetes 采用经典的 主从架构 (Master-Slave)

1. Control Plane (Master)：集群的"大脑"

   负责决策（调度、管理、API响应）

2. Worker Nodes：集群的"手脚"

   负责干活（运行容器负载）



Master节点核心组件 (1/2)

1. kube-apiserver (入口/大脑)

唯一入口：所有组件（kubectl, dashboard, node）都只和它通信。

无状态：可水平扩展。

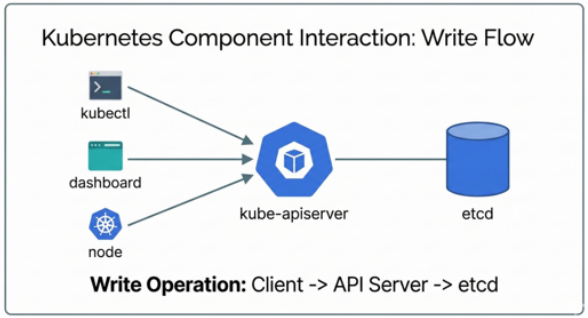
职责：认证、授权、准入控制、API 注册。

2. etcd (记忆/存储)

分布式键值数据库：保存集群所有状态数据。

高可用：基于 Raft 协议，通常奇数部署 (3或5个)。

敏感性：只有 API Server 能直接访问 etcd。



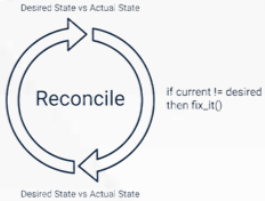
Master节点核心组件 (2/2)

3. kube-controller-manager (决策者)

控制循环 (Control Loop)：不断比较 "期望状态" vs "实际状态"。

内置控制器：节点控制器、副本控制器、Service控制器等。

逻辑：if current != desired then fix_it()

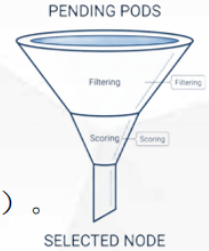


4. kube-scheduler (调度器)

职责：给新创建的 Pod 找一个最合适的节点 (Node)。

两步走：

1. 预选 (Filtering)：排除资源不足的节点。
2. 优选 (Scoring)：给剩余节点打分（亲和性、负载均衡）。



01 Worker节点核心组件

1. kubelet (节点管家)

每个节点的代理人 (Agent)。

接收 API Server 的指令，管理 Pod 生命周期。

调用 CRI (容器运行时接口) 启动容器。

2. kube-proxy (网络代理)

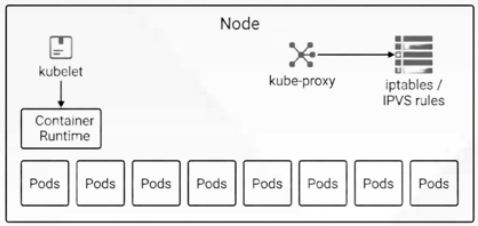
维护节点上的网络规则 (iptables / IPVS)。

实现 Service 的负载均衡和流量转发。

3. Container Runtime (运行时)

真正运行容器的软件：containerd, CRI-O (Docker 已被垫片化)。

Kubernetes Worker Node

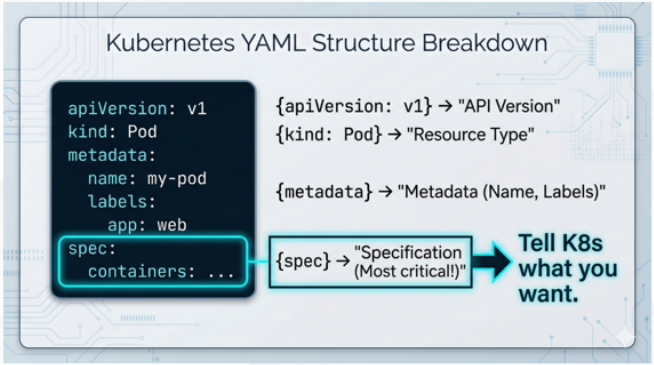


声明式API与YAML结构

一切皆对象 (Everything is an Object)。K8s 通过 YAML 文件描述期望状态 (Desired State)。

YAML 四大金刚（必填字段）

```yaml
apiVersion: v1          # API 版本
kind: Pod               # 资源类型
metadata:               # 元数据（名称、标签）
  name: my-pod
  labels:
    app: web
spec:                   # 规格（最关键！）
  containers: ...       # 用户期望的具体配置
```



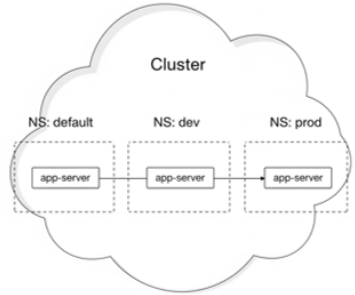
01 逻辑隔离：Namespace

什么是 Namespace?

虚拟集群：在同一个物理集群中，逻辑上划分出的独立空间。

作用：资源隔离、权限控制、避免命名冲突。

名称	用途
default	默认工作区
kube-system	K8s 系统组件（切勿随意修改）
dev / prod	用户自定义：开发 / 生产环境



Pod：核心特性解析

为什么不直接管理容器？

容器之间往往需要紧密协作（如日志收集、代理）。

Pod 提供共享环境，简化协作。

Pod 的共享机制

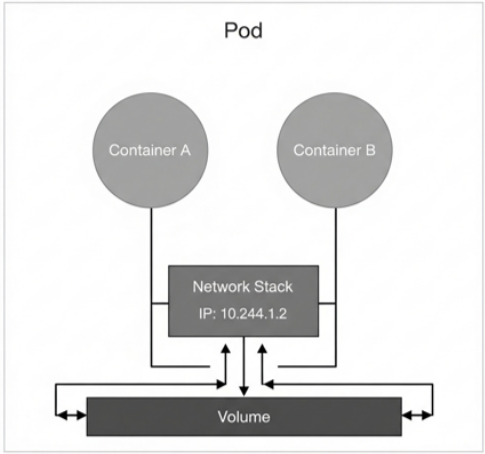
1. 共享网络（Network Namespace）

   Pod 内所有容器共用一个 IP。

   容器间通过 localhost 通信。

2. 共享存储（Shared Volume）

   容器间可以通过挂载同一个 Volume 交换文件。



Pod YAML 关键配置


```yaml
spec:
  containers:
  - name: web
    image: nginx:1.25
    ports:
    - containerPort: 80
    # 资源配额（生产环境必须配置!）
    resources:
      requests:
        cpu: "250m"       # 调度依据
        memory: "64Mi"
      limits:
        cpu: "500m"       # 运行上限
        memory: "128Mi"
```

重点解读：

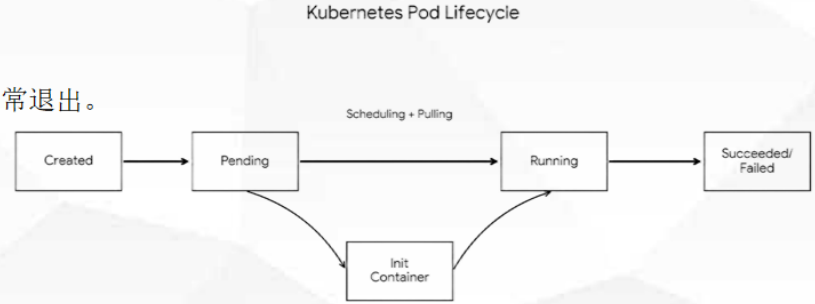
Requests：节点最少要有这么多资源，Pod 才能调度上去。

Limits：容器用超了会被限流（CPU）或 杀掉（Memory OOM）。

Pod 的生命周期

状态流转（Phase）

1. Pending：等待调度或拉取镜像。
2. Running：容器已启动。
3. Succeeded/Failed：任务结束或异常退出。
4. Unknown：与节点失联。



Init Containers

- 在主容器启动之前执行。
- 用途：初始化配置、等待数据库就绪。
- 特点：必须按序、成功执行完，主容器才会启动。

探针 (Probes)：健康检查机制

K8s 如何知道应用"活着"？

不能只看进程 PID，要看服务是否正常响应。

```yaml
livenessProbe:
  httpGet: { path: /healthz, port: 80 }
  initialDelaySeconds: 15  # 启动后等15秒再查
```

探针类型	作用	失败后果
Liveness	存活检测（比如死锁）	重启容器
Readiness	就绪检测（比如启动中）	切断流量（从 Service 移除）
Startup	启动检测（针对慢启动应用）	抑制其他探针

02 Pod QoS (服务质量) 等级

根据资源配置 (requests/limits)，K8s 将 Pod 分为三等.

1. Guaranteed (头等舱)

   request == limit

   待遇：最后被驱逐，最稳定。

2. Burstable (商务舱)

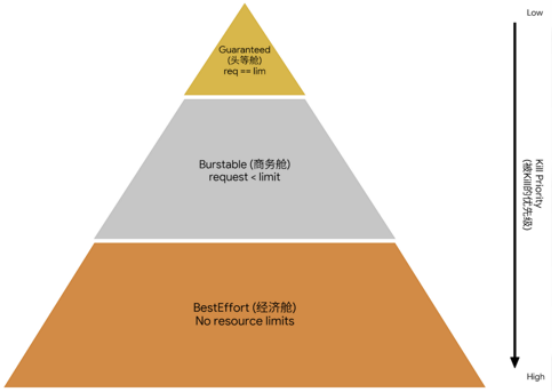
   request < limit

   待遇：资源紧张时，可能会被驱逐。

3. BestEffort (经济舱)

   未设置资源限制

   待遇：节点缺资源时，第一个被杀掉。



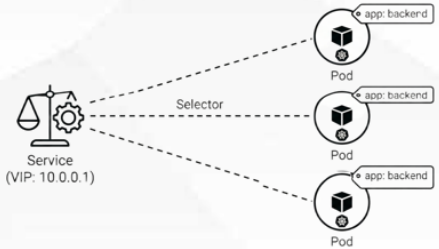
Service：定义逻辑集合

为什么需要 Service?

Pod 是短暂的，重启后 IP 会变。Service 提供一个 VIP（虚拟 IP），屏蔽后端的动态变化。

核心机制：Labels & Selectors

Service 通过标签选择器找到后端 Pod。




```yaml
# Service 定义
selector:
  app: backend    # 寻找标签为 app=backend 的 Pod
ports:
  - port: 80      # Service 暴露端口
    targetPort: 8080 # 容器实际端口
```

03 Service 的三种常用类型

1. ClusterIP（默认）

   仅集群内访问。

   适用于数据库、内部微服务。

2. NodePort

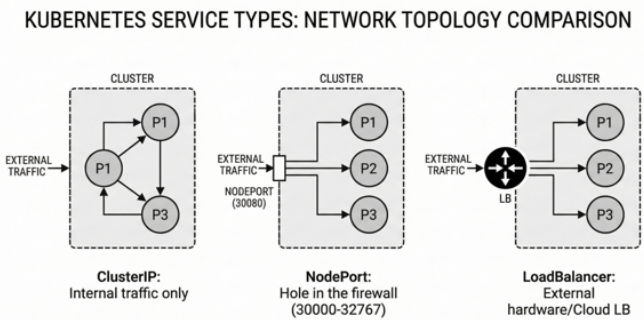
   在每个 Node 上开一个端口（30000-32767）。

   外部访问：NodeIP ： NodePort。

3. LoadBalancer

   申请云厂商（AWS/Aliyun）的硬件负载均衡器。

   外部访问：LB 公网 IP。



Service 发现机制：DNS

不用记 IP，只记名字

K8s 集群内置 CoreDNS。

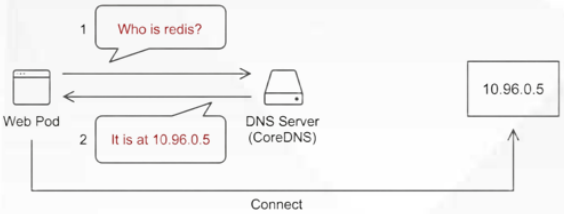
Pod 可以通过域名直接访问 Service。

域名格式 <service>.<namespace>.svc.cluster.local

服务名：redis

命名空间：default

访问地址：redis.default 或 简写为 redis



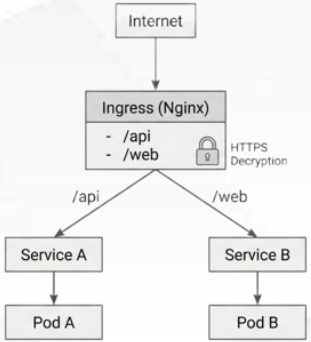
03 Ingress：七层路由入口

Service (L4) vs Ingress (L7)

- Service：只能基于 IP/Port 转发。
- Ingress：可以基于 域名（Host）和 路径（Path）转发。

优势

- 节省公网 IP（一个 IP 代理多个服务）。
- 支持 SSL/TLS 终止。

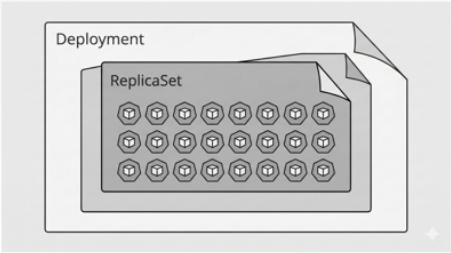


Deployment：无状态应用管家

核心功能

1. 维持副本数：声明 3 个，就永远保持 3 个。
2. 滚动更新：平滑升级，不中断服务。
3. 版本回滚：一键回到过去。

用户通常只操作 Deployment，不直接操作 ReplicaSet。



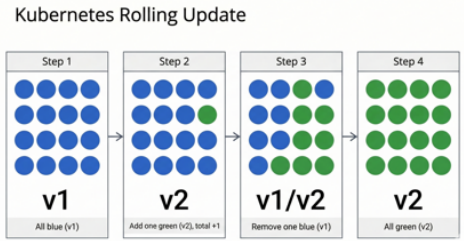
滚动更新 (Rolling Update)

什么是滚动更新？

"先起新，后停旧"，逐步替换，确保服务始终可用。

关键参数

- maxSurge：允许比期望多几个？（如：多跑1个新版本）
- maxUnavailable：允许比期望少几个？（如：不允许任何Pod挂掉）

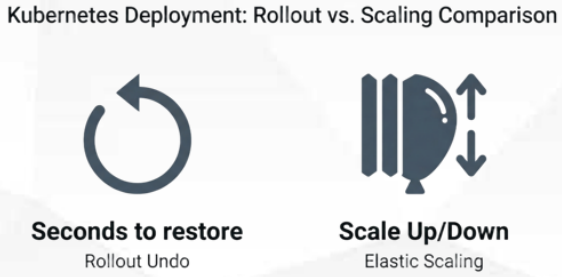


03 版本回滚与扩缩容

后悔药：Rollout Undo

发布出错？立即回滚。

```
# 回滚到上一个版本
kubectl rollout undo deployment/web
弹性伸缩：Scale
流量突增？立即扩容。
bash
# 手动扩容到 10 个副本
kubectl scale deployment/web --replicas=10
```

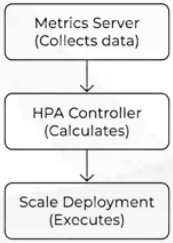


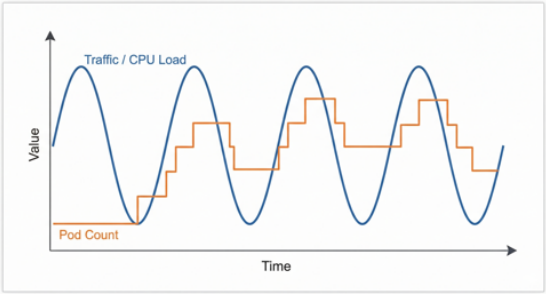
HPA：水平自动扩缩容

根据 CPU / 内存利用率，自动调整 Pod 数量。

指标：CPU > 70% -> 扩容

范围：Min 2 ~ Max 10





03 kubectl 常用命令速查

增删改查（CRUD）

kubectl apply -f xx.yaml ：声明式创建/更新（推荐）

kubectl get pods/svc/deploy ：查看资源

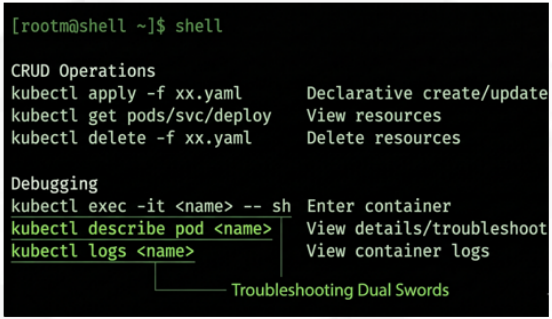
kubectl delete -f xx.yaml ：删除资源

调试（Debug）

kubectl describe pod <name>：查看详情/报错原因（最常用）

kubectl logs <name>：查看容器日志

kubectl exec -it <name> -- sh ：进入容器内部



西南交通大学
Southwest Jiaotong University

云计算技术

第5讲 Kubernetes深入与服务网格

授课教师：戴朋林

课程代码：SCAI004712

2026年3月

01 资源请求与限制 (Requests & Limits)

- 为什么需要设置资源？
  - Requests (请求)：调度器以此为依据选择节点（保底资源）。
  - Limits (限制)：容器运行时的最大上限（熔断资源）。
- 核心配置

```yaml
resources:
  requests:
    memory: "64Mi"   # 调度依据
    cpu: "250m"      # 0.25核
  limits:
    memory: "128Mi" # 超过OOM Kill
    cpu: "500m"      # 超过限流
```

01 调度器原理：预选与优选

- kube-scheduler 调度流程
  - 调度（Scheduling）是指将未绑定节点的Pod分配到合适Worker节点的过程。kube-scheduler通过两阶段筛选完成调度决策。
- 两阶段调度

阶段	名称	说明
阶段1	预选（Filtering / Predicates）	排除不满足条件的节点，得到候选节点列表
阶段2	优选（Scoring / Priorities）	对候选节点评分（0-100），选择得分最高的节点

- 预选常见规则

规则	说明
PodFitsResources	节点剩余资源≥Pod的Request
PodFitsHostPorts	节点端口无冲突
MatchNodeSelector	节点标签匹配Pod的nodeSelector
NoTaintUnlessTolerated	节点Taint需被Pod Toleration匹配
CheckNodeMemoryPressure	节点无内存压力

- 优选常见规则

规则	说明
LeastRequestedPriority	优先选择剩余资源最多的节点（分散调度）
BalancedResourceAllocation	CPU和内存使用率更均衡的节点得分更高
NodeAffinityPriority	匹配亲和性规则的节点得分更高
InterPodAffinityPriority	满足Pod亲和性的节点得分更高

01 节点亲和性（Node Affinity）

西南交通大学 Southwest Jiaotong University

- 节点亲和性（Node Affinity）是一种调度约束，允许Pod声明对特定节点的偏好，基于节点标签进行匹配。它是nodeSelector的增强版，支持更复杂的表达式。
- 两种模式

模式	说明	效果
requiredDuringSchedulingIgnoredDuringExecution	硬约束——必须满足	不满足则Pod无法调度
preferredDuringSchedulingIgnoredDuringExecution	软约束——优先满足	不满足仍可调度到其他节点

- 配置示例

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disk-type
          operator: In
          values: ["ssd", "nvme"] # 必须是 SSD
```

01 节点亲和性（Node Affinity）

- Pod亲和性（Pod Affinity）
  - Pod亲和性允许Pod声明希望与具有特定标签的其他Pod调度到同一节点/区域，提升通信效率。
  - 示例

```yaml
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchLabels:
          app: redis-cache
      topologyKey: kubernetes.io/hostname
```

  - 场景：Web 与 Redis 部署在一起

01 节点亲和性（Node Affinity）

西南交通大学 Southwest Jiaotong University

- Pod反亲和性（Pod Anti-Affinity）
  - Pod反亲和性允许Pod声明不希望与特定Pod在同一节点/区域，容灾（单点故障不影响整体）。
  - 示例

```yaml
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
    - weight: 100
      podAffinityTerm:
        labelSelector:
          matchLabels:
            app: my-web-app
        topologyKey: kubernetes.io/hostname
```

  - 场景：高可用分散部署

01 污点与容忍（Taints & Tolerations）

- 污点（Taint）是节点的属性，表示该节点拒绝接受特定Pod的调度。容忍（Toleration）是Pod的属性，表示该Pod可以容忍某个节点的污点。
  - 亲和性＝Pod主动选择节点（"我想去哪"）
  - 污点/容忍＝节点排斥Pod（"谁不能来"），Pod可声明豁免
- Taint效果类型

Effect	说明
NoSchedule	不调度新Pod到该节点（已运行的Pod不受影响）
PreferNoSchedule	尽量不调度，但不保证
NoExecute	不调度新Pod，且驱逐已运行的不匹配Pod

- 操作命令

```
# 添加Taint
kubectl taint nodes node1 gpu=true:NoSchedule
# 查看节点Taint
kubectl describe node node1 | grep Taint
# 移除Taint
kubectl taint nodes node1 gpu=true:NoSchedule-
```

- 典型场景
  - GPU节点：Taint为`gpu=true:NoSchedule`，只有需要GPU的Pod才容忍该Taint
  - Master节点：默认Taint `node-role.kubernetes.io/master:NoSchedule`，避免业务Pod调度到Master

01 ResourceQuota与LimitRange

- ResourceQuota——命名空间级资源配额
  - ResourceQuota限制一个命名空间中可使用的总资源量。

```
# ResourceQuota 示例
spec:
  hard:
    requests.cpu: "4"
    requests.memory: "8Gi"
    pods: "10"
```

- LimitRange——Pod/容器级默认资源约束
  - LimitRange为命名空间中的Pod/容器设置默认值和范围约束。

对比	ResourceQuota	LimitRange
作用范围	命名空间总量	单个Pod/容器
功能	限制总资源、对象数量	设置默认值、范围

02 StatefulSet有状态应用管理

- StatefulSet是Kubernetes中管理有状态应用的工作负载控制器。与Deployment不同，StatefulSet为每个Pod提供稳定的网络标识和持久化存储，确保Pod的身份在重建后保持不变。
- StatefulSet vs Deployment

特性	Deployment	StatefulSet
Pod名称	随机后缀（web-7d9f8c-xk2l3）	有序固定（web-0, web-1, web-2）
网络标识	无稳定标识	稳定的DNS名称（web-0.web-svc）
存储	Pod重建后存储丢失	每个Pod绑定独立的PVC，重建后保留
启动/停止顺序	并行	有序（0→1→2启动，2→1→0停止）
适用场景	无状态应用（Web API）	有状态应用（数据库、消息队列）

02 核心组件：Headless Service

- 为什么需要 Headless Service?
  - 普通 Service 有 ClusterIP，做负载均衡。
  - Headless Service(`clusterIP: None`)不分配 IP，直接返回后端 Pod 的 IP 列表。
  - 它配合 StatefulSet 生成稳定的 DNS 记录：`pod-name.service-name`。

```yaml
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None  # 关键点
  selector:
    app: mysql
```

02 核心特性：VolumeClaimTemplates

- 自动化存储绑定
  - Deployment 所有 Pod 共享一个 PVC（通常用于只读）。
  - StatefulSet 需要每个 Pod 有自己独立的 PVC。

```yaml
volumeClaimTemplates:  # 为每个 Pod 生成专属 PVC
- metadata:
    name: data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "fast-storage"
    resources:
      requests:
        storage: 1Gi
```

03 ConfigMap：非敏感配置

- ConfigMap是Kubernetes中用于存储非敏感配置数据的API对象，以键值对形式存储，实现配置与应用代码解耦。
- 使用场景
  - 环境变量：`DB_HOST`, `LOG_LEVEL`
  - 配置文件：`nginx.conf`, `redis.conf` (挂载为 Volume)

```
# 挂载配置文件
volumes:
- name: config-volume
  configMap:
    name: nginx-config
```

- 使用方式

方式	说明
环境变量注入	envFrom 或单个 valueFrom
卷挂载为文件	将ConfigMap挂载为目录中的文件
命令行参数	通过环境变量传递给命令

03 ConfigMap：非敏感配置

西南交通大学 Southwest Jiaotong University

- ConfigMap使用示例

方式1：作为环境变量

```
spec:
  containers:
  - name: app
    image: myapp:v1
    envFrom:                          # 注入所有键值对
    - configMapRef:
        name: app-config
    env:                              # 注入单个键值对
    - name: DATABASE_HOST
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: DB_HOST
```

方式2：挂载为文件

```
spec:
  containers:
  - name: nginx
    image: nginx:alpine
    volumeMounts:
    - name: config-vol
      mountPath: /etc/nginx/conf.d
  volumes:
  - name: config-vol
    configMap:
      name: nginx-conf
```

- 热更新
  - 环境变量方式：修改ConfigMap后，需要重启Pod才能生效
  - 卷挂载方式：kubelet周期性同步（约1分钟），自动更新文件内容，无需重启
  - 应用需要自行实现配置热加载逻辑（如watch文件变化）

03 Secret：敏感信息管理

- Secret是Kubernetes中用于存储敏感信息（密码、Token、证书等）的API对象。数据以Base64编码存储（注意：非加密，需配合etcd加密使用）。

类型	用途
Opaque	通用密钥（默认类型）
kubernetes.io/tls	TLS证书（tls.crt + tls.key）
kubernetes.io/dockerconfigjson	Docker Registry认证信息
kubernetes.io/service-account-token	ServiceAccount Token

- 创建方式

```
# 命令行创建
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=S3cureP@ss
```

```
# YAML创建（值需Base64编码）
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=          # echo -n "admin" | base64
  password: UzNjdXJlUEBzcw== # echo -n "S3cureP@ss" | base64
```

- Secret vs ConfigMap
  - 存储内容：密码、Token、TLS 证书 vs 普通配置。
  - 编码方式：Base64 编码（注意：Base64 不是加密，只是编码）。
  - 使用方式：挂载为 tmpfs (内存卷)，不会写入节点磁盘。

04 K8s 存储架构概览

- 问题：容器存储的临时性
  - 容器文件系统是临时的——容器重启或删除后数据丢失
  - 有状态应用（数据库、文件服务）需要持久化存储
- Kubernetes存储三层架构
  - Pod → PVC（存储请求） → PV（存储资源） → 底层存储（云盘/NFS/Ceph）

概念	全称	说明
PV	PersistentVolume	集群级别的存储资源，由管理员预先创建或动态供给
PVC	PersistentVolumeClaim	Pod对存储的请求声明（大小、访问模式）
StorageClass	-	定义存储的类型和供给策略，支持动态创建PV

  - 访问模式

模式	缩写	说明
ReadWriteOnce	RWO	单节点读写（最常用）
ReadOnlyMany	ROX	多节点只读
ReadWriteMany	RWX	多节点读写（需NFS/CephFS等共享存储）

  - 回收策略

策略	说明
Retain	PVC删除后PV保留（手动回收）
Delete	PVC删除后PV和底层存储一并删除
Recycle	已弃用（清空数据后重新使用）

04 静态供给 (Static Provisioning)

- 传统流程
  - 管理员手动创建 10 个 PV（预先分配）。
  - 开发人员创建 PVC。
  - K8s 寻找匹配的 PV 进行绑定。

```yaml
# 1. 管理员创建PV
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 50Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    server: 192.168.1.100
    path: /data/k8s-volumes
---
```

```yaml
# 2. 开发者创建PVC
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
```

```yaml
# 3. Pod使用PVC
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
  - name: app
    image: myapp:v1
    volumeMounts:
    - name: data
      mountPath: /app/data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: app-data-pvc
```

04 动态供给 (Dynamic Provisioning)

- 静态供给的问题
  - 管理员需要提前创建大量PV
  - PV大小和PVC请求不一定匹配（浪费资源）
  - 无法自动化
- 动态供给（Dynamic Provisioning），StorageClass定义存储的类型和供给者（Provisioner），当PVC请求匹配某个StorageClass时，自动创建对应的PV。

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs   # 供给者
parameters:
  type: gp3                          # AWS EBS gp3磁盘类型
  fsType: ext4
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer  # 延迟绑定
```

04 动态供给 (Dynamic Provisioning)

- StorageClass与动态供给
  - PVC使用StorageClass

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: fast-ssd    # 指定StorageClass
  resources:
    requests:
      storage: 100Gi
```

  - 动态供给流程

PVC请求（storageClassName: fast-ssd, 100Gi）
→ StorageClass（provisioner: aws-ebs）
→ 自动创建100Gi EBS卷
→ 创建PV并绑定PVC
→ Pod使用PVC

05 定义与背景

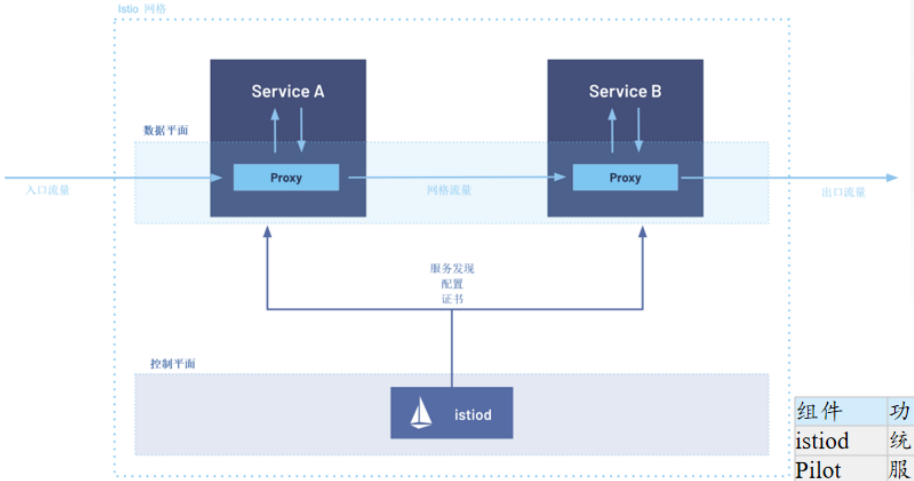
- 问题：微服务通信的复杂性，随着微服务数量增长，服务间通信面临诸多挑战：
  - 流量管理（负载均衡、重试、超时）
  - 安全（加密通信、身份认证）
  - 可观测性（链路追踪、指标、日志）

  传统方式：在每个服务中嵌入SDK（如Spring Cloud、Hystrix）——语言绑定，侵入性强
- Service Mesh定义
  - 服务网格（Service Mesh）是一个专门用于处理服务间通信的基础设施层。它通过在每个服务旁部署一个轻量级网络代理（Sidecar），以透明的方式处理服务发现、负载均衡、故障恢复、指标采集和安全通信。——William Morgan, Buoyant CEO, 2017
  - 核心价值

传统SDK方式	Service Mesh
业务代码中嵌入通信逻辑	通信逻辑下沉到基础设施
语言绑定（Java SDK无法用于Go服务）	语言无关（Sidecar代理）
升级SDK需要修改并重新部署所有服务	升级代理不影响业务代码
每个服务单独实现熔断/限流	统一的策略管理

Istio架构

- Istio是由Google、IBM和Lyft共同开发的开源服务网格平台，是目前最流行的Service Mesh



组件	功能
istiod	统一的控制面（合并了Pilot/Citadel/Galley）
Pilot	服务发现、流量管理规则下发
Citadel	证书管理、mTLS加密
Galley	配置验证和分发
Envoy	数据面代理（Sidecar），所有流量经过Envoy

05 Sidecar代理（Envoy）

- 什么是Sidecar模式？
  - 在Kubernetes中，Istio通过自动注入（Injection）的方式，为每个Pod添加一个Envoy代理容器（Sidecar）。所有进出Pod的网络流量都会被Envoy拦截和处理。

- Envoy简介
  - Envoy是由Lyft开发的高性能L4/L7代理，是CNCF毕业项目。它是Istio数据面的核心。

能力	说明
L7路由	基于HTTP头、路径、权重的智能路由
负载均衡	Round Robin、Least Connection、一致性哈希等
熔断	连接数限制、请求数限制
重试	自动重试失败请求
超时	请求级别超时控制
TLS	自动mTLS加密通信
可观测性	自动生成请求指标（Prometheus）、分布式追踪（Jaeger）

西南交通大学 Southwest Jiaotong University

- 路由规则定义
  - 决定请求到达 Service 后，去哪个版本的 Pod。

```yaml
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
    route:
    - destination:
        host: reviews
        subset: v2  # 测试用户去 v2
  - route:
    - destination:
        host: reviews
        subset: v1  # 其他人去 v1
```

- 定义子集与策略
  - Subsets：将一个 Service 的 Pod 划分为 v1, v2。
  - TrafficPolicy：负载均衡策略、连接池、熔断。

```yaml
spec:
  host: reviews
  subsets:
  - name: v1
    labels:
      version: v1
  trafficPolicy:
    loadBalancer:
      simple: RANDOM  # 随机负载均衡
```

05 Istio安全：mTLS

- mTLS（Mutual TLS）是双向TLS认证。传统TLS只有客户端验证服务端证书，mTLS要求双方互相验证对方身份。
- Istio mTLS工作流程
  - Istio Citadel为每个服务自动签发X.509证书
  - Envoy自动使用证书建立mTLS连接
  - 应用代码无需任何修改——加密对应用透明

Service A (Envoy) ←— mTLS加密通道 —→ Service B (Envoy)

↑ Citadel签发的客户端证书

↑ Citadel签发的服务端证书

05 金丝雀发布（Canary Release）与流量灰度

- 金丝雀发布是一种渐进式发布策略，先将少量流量导向新版本，观察无异常后逐步扩大新版本的流量比例，直到全量切换。
- Istio实现金丝雀发布
  - 阶段1: v1=100%, v2=0% → 部署v2，不导入流量
  - 阶段2: v1=90%，v2=10% → 10%流量到v2，观察指标
  - 阶段3: v1=50%，v2=50% → 扩大到50%
  - 阶段4: v1=0%，v2=100% → 全量切换到v2
- 对比其他发布策略

策略	原理	风险	回滚速度
滚动更新	逐步替换旧Pod	中等	慢（需回滚Deployment）
蓝绿部署	两套完整环境切换	低	快（切换路由）
金丝雀发布	按比例分流	最低	最快（调整权重）
A/B测试	按条件（用户/地区）分流	低	快

05 金丝雀发布（Canary Release）与流量灰度

西南交通大学 Southwest Jiaotong University

- Istio的优势
  - 流量权重精确控制：可以精确到1%的流量比例
  - 基于请求内容路由：根据Header、Cookie、URL等条件分流
  - 实时切换：修改VirtualService即刻生效
  - 回滚零成本：调整权重回到100% v1即可

西南交通大學
Southwest Jiaotong University

云计算技术

第6讲 消息队列与事件驱动架构

授课教师：戴朋林

课程代码：SCAI004712

2026年4月

01 消息队列的概念与作用

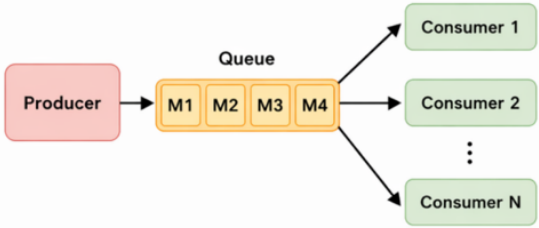
- 消息队列(Message Queue, MQ) 是一种应用间通过消息进行**异步通信**的中间件。
- 核心作用
  - ✓ 异步解耦：发送方无需等待接收方处理， 降低系统耦合。
  - ✓ 流量削峰：缓冲突发流置， 保护下游服务不披压垮。
  - ✓ 最终一致性： 通过可靠消息传递，达成系统最终一致的状态。

02 消息模型：点对点（Queue）与发布/订阅（Topic）

西南交通大学 Southwest Jiaotong University

点对点（Queue）

- 消息发送到队列，只有一个消费者能消费。
- 适用于任务分发、负载均衡。

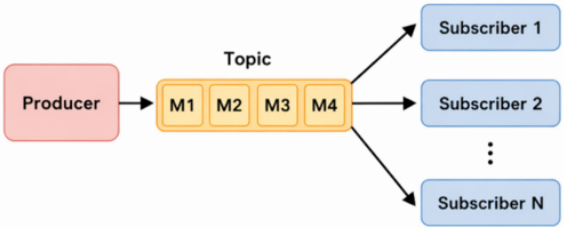


特点

- 每条消息只会被一个消费者消费
- 消费者竞争消费，实现负载均衡
- 不适合广播场景

发布/订阅（Topic）

- 消息发布到主题，所有订阅者都能收到。
- 适用于广播通知、事件分发。



特点

- 每条消息会被所有订阅者消费
- 支持一对多广播
- 适合事件通知、日志分发等场景

03 消息投递语义：At Most Once（最多一次）

消息投递语义：在消息传递过程中，确保消息按预期的方式到达消费者的可靠性保障。

At Most Once

- 每条消息最多投递一次，即消息可能会丢失，但绝不会重复发送。
- 适用场景：
  - 消息丢失不影响系统的正常运行，系统能容忍消息丢失，例如日志数据收集（丢失一些日志不影响整体分析结果）。
  - 适用于对消息可靠性要求不高的场景。
- 优点：效率高，处理过程简单，消息丢失时不需要额外的重试机制。
- 缺点：由于消息丢失的可能性，不能保证消息的完整性。

03 消息投递语义：At Least Once（至少一次）

消息投递语义：在消息传递过程中，确保消息按预期的方式到达消费者的可靠性保障。

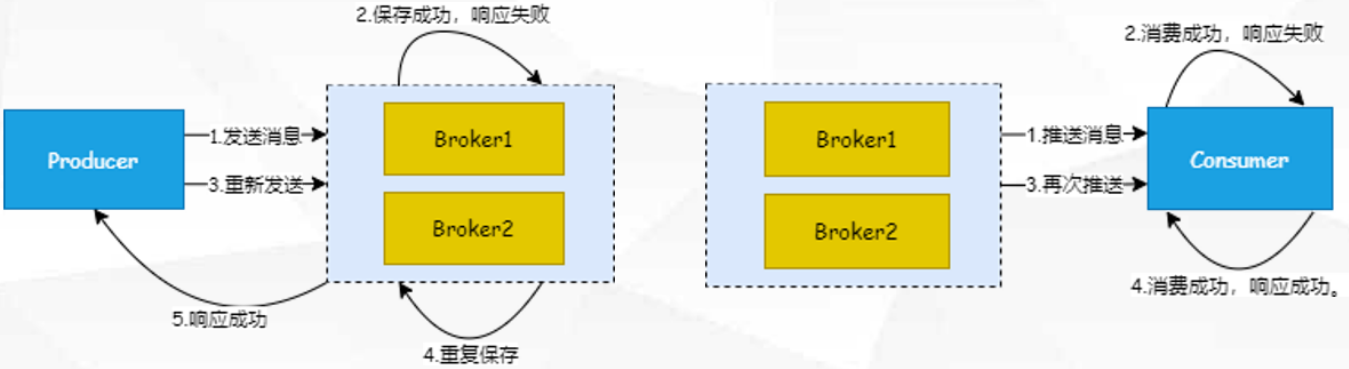
At Least Once

- 每条消息至少投递一次，保证每个消息至少有一个消费者接收。
- 适用场景：
  - 系统中非常注重**数据完整性**，**重复消费**并不会带来严重问题的场景。
  - 例如订单支付系统，如果多次支付请求被处理，最终会给用户产生冗余的扣款，但不会丢失任何支付请求。
- 优点：消息不会丢失，适用于对消息可靠性有较高要求的场景。
- 缺点：可能会有消息**重复投递**的情况，需要消费者能够处理重复数据，例如幂等性。

03 消息投递语义：At Least Once（至少一次）

什么情况下会导致消息重复?

- 生产者发送消息后，Broker 保存成功，但是没有成功给生产者返回 ACK
- 消费者消费消息后，给 Broker 返回 ACK 失败，导致 Broker 没有修改偏移量，同一条消息再次发送



03 消息投递语义：Exactly Once（恰好一次）

西南交通大学 Southwest Jiaotong University

消息投递语义：在消息传递过程中，确保消息按预期的方式到达消费者的可靠性保障。

Exactly Once

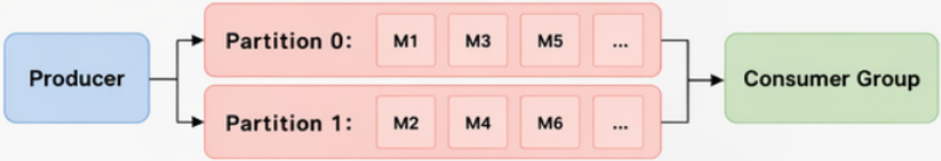
- 每条消息仅投递一次，并且没有重复消费的情况，确保消息只会被处理一次。
- 适用场景：
  - 对消息的准确性和重复消费非常敏感的场景，如银行转账、订单交易等。
  - 例如支付系统中，若订单支付成功，不能再重复扣款。
- 优点：完美的消息传递保证，确保消息只会被消费一次，适用于高价值和高准确性的场景。
- 缺点：实现复杂，性能开销较大，通常需要高级的幂等性保证和事务机制。

03 消息投递语义

投递语义	适用场景	优点	缺点
At Most Once	性能要求高、对消息丢失不敏感（如日志收集、事件记录）	高效，处理过程简单，性能开销低	可能会丢失消息，无法确保消息完整性
At Least Once	数据完整性要求较高，重复处理不影响（如订单支付、社交媒体消息）	不会丢失消息，保证消息传递可靠	可能会导致消息重复，需要额外处理重复数据（幂等性）
Exactly Once	高价值交易、金融支付、库存管理等高敏感场景	完美的消息传递保证，消息准确且无重复消费	实现复杂，性能开销大，通常需要高级的幂等性保证

04 消息顺序性保证

- **消息顺序性**是指在消息传递过程中，消息的接收顺序与发送顺序相同。
- 对于一些需要强一致性、高可靠性的系统（如交易系统），消息顺序性的保证是核心需求。
- 顺序性常见的保证方式：
  - 单分区保证： 在同一分区内，消息是有序的(Kafka、Redis Stream)
  - 单队列保证： 在同一队列中，消息具有FIFO （ 先入先出）
  - 多分区/多队列：无法保证全局顺序，只保证分区 / 队列内顺序

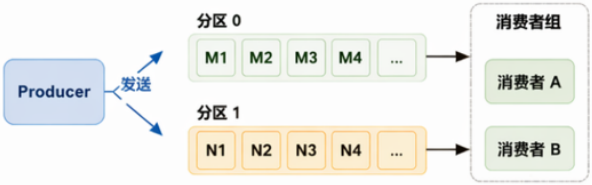


增加消息队列数量/分区数量，可增加并行性，也会牺牲全局顺序性

04 消息顺序性保证

特性	队列（Queue）	分区（Partition）
消息顺序	严格按照发送顺序消费	只保证单个分区内的顺序，跨分区不保证顺序
适用模型	点对点（Queue）	发布/订阅（Topic）
消费者	每条消息只被一个消费者消费	每个分区有独立消费者，多个消费者并行消费不同分区
并行性	不支持并行消费	支持并行消费，不同消费者处理不同分区的数据

发布/订阅（Topic + 分区）



顺序性：

- 在分区0内：M1 → M2 → M3 → M4（有序）
- 在分区1内：N1 → N2 → N3 → N4（有序）
- 跨分区：M 与 N 的顺序 **无法保证**

点对点（Queue）

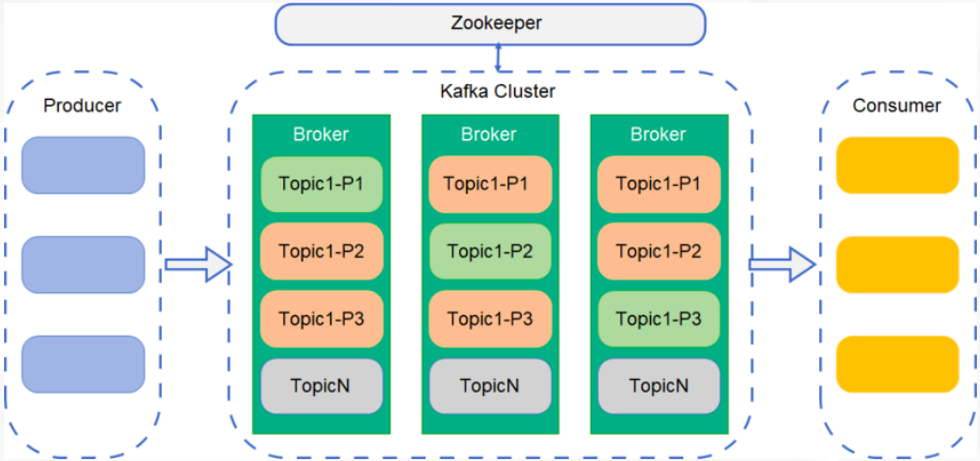
顺序性：

整个队列：M1 → M2 → M3 → M4 → M5（有序）

- 一个消息只能被一个消费者消费

01 Kafka基本介绍

- Kafka是最初由Linkedin公司开发，是一个分布式的发布/订阅消息系统
- Linkedin于2010年贡献给了Apache基金会并成为顶级开源项目
- Kafka核心由Broker、Topic、Partition、Consumer Group组成



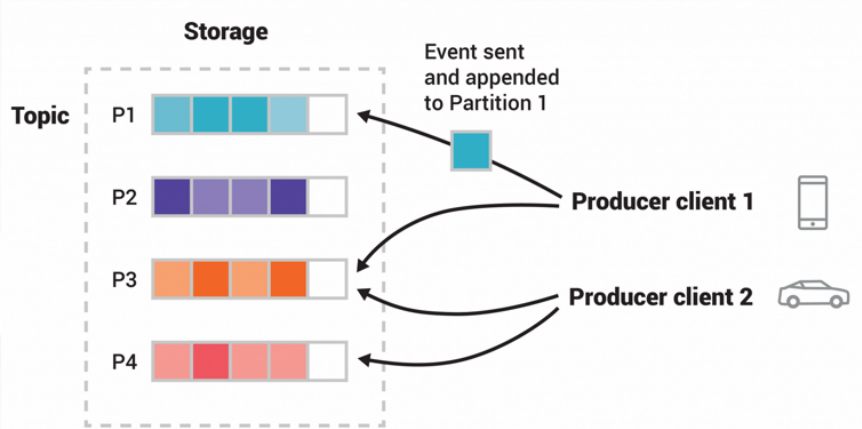
APACHE kafka
A distributed streaming platform

02 Kafka基本架构组件

组件	说明	关键特性
Broker	Kafka 集群中的服务器节点，负责数据存储，请求处理和副本管理。	- 水平扩展 - 高可用（副本机制）
Topic	消息的逻辑标识，用于分类和隔离数据。	- 支持多个分区 - 可配置副本数
Partition	Topic 的物理分片，是 Kafka 并行处理和扩展的基本单位。	- 分区内消息有序 - 分区间可并行
Consumer Group	一组消费者实例，共同消费一个或多个 Topic。	- 组内负载均衡 - 组间互不影响

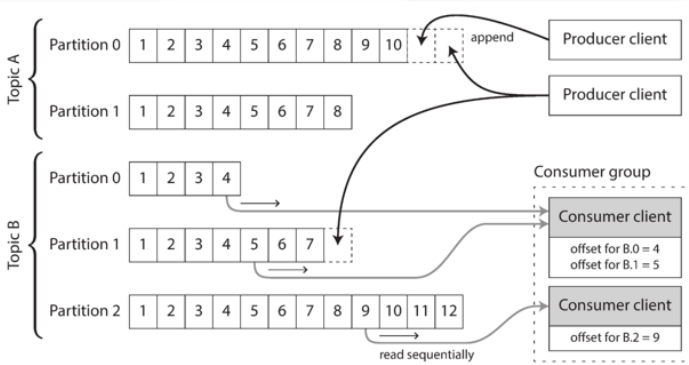
03 Partition分区

- Kafka 中 Topic 被分成多个 Partition 分区。
- Topic 是一个逻辑概念，Partition 是最小的存储单元，掌握着一个 Topic 的部分数据。
- 每个 Partition 都是一个单独的 log 文件，每条记录都以追加（append）的形式写入。
- Kafka 把 Topic 的多个 Partition 分布在多个 Broker 中



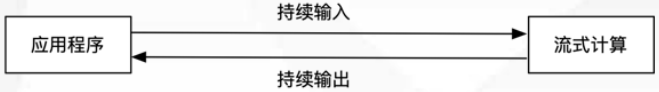
03 Offsets和消息的顺序

- Offset
  - Partition 中的每条记录都会被分配一个唯一的序号，称为 **Offset**（偏移量）。
  - Offset 是一个递增的、不可变的数字，由 Kafka 自动维护。
  - 当一条记录写入 Partition 的时候，它就被追加到 log 文件的末尾，并被分配一个序号，作为 Offset
- 消息的顺序
  - 一个 Partition 内部消息有序，一个 Topic 跨 Partition 是无序的。
  - 如果强制要求 Topic 整体有序，就只能让 Topic 只有一个 Partition。

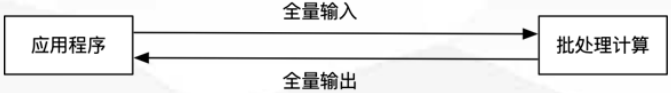


02 基于Kafka的流处理应用

- Kafka也常用于实时流处理，能够处理来自各个流的实时数据，支持数据的过滤、转换、聚合等操作。
- 结合流式计算引擎（如Apache Flink或Spark Streaming）进一步处理和分析
- 应用场景：
  - 实时分析：实时对数据流进行分析、计数、监控等。
  - 数据流转发：在数据流到达后，Kafka将其发送到目标系统进行进一步处理。



流处理应用



批处理应用

01 AMQP协议基础

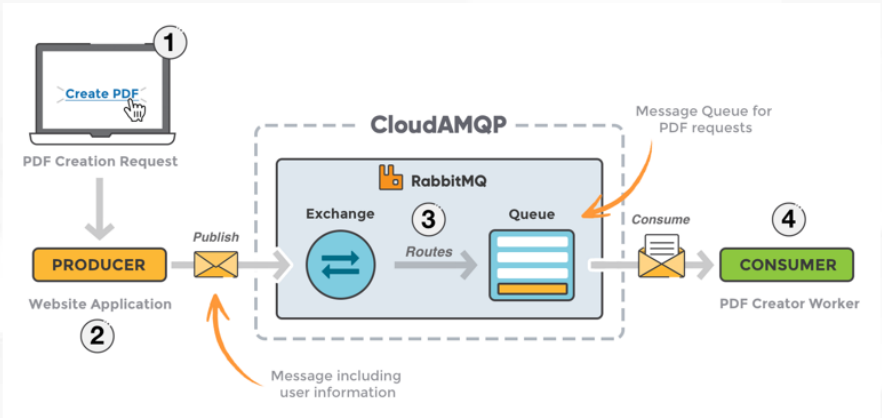
- **AMQP（Advanced Message Queuing Protocol）**是一个广泛应用于消息队列中的开放标准协议。
- 目标：为消息传递提供标准化的方式，确保消息系统之间的互操作性。
- 关键特性：
  - **可靠性**：确保消息在传递过程中不丢失。
  - **可扩展性**：支持从小型系统到大型分布式系统的扩展。
  - **灵活性**：支持多种消息传输模式，包括点对点、发布/订阅等。

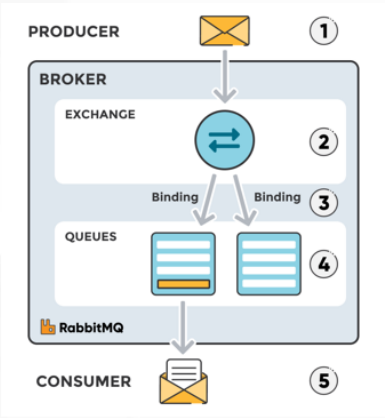
RabbitMQ核心概念

- **RabbitMQ是基于AMQP协议实现的消息队列系统**
- RabbitMQ如何实现AMQP协议：
  - 通过AMQP的定义来管理Producer、Consumer和消息的传递。
  - 提供AMQP协议中的核心功能：Exchange、Queue、Binding、Routing Key等。
  - 提供附加功能（如消息确认机制、死信队列、延迟消息等）来增强其功能。

02 RabbitMQ核心概念

概念	说明
Exchange	负责接收消息并将其路由到适当的队列。
Queue	存储消息，消费者从中获取消息进行处理。
Binding	将Exchange与Queue绑定的规则，定义消息路由路径。
Routing Key	消息的路由关键字，用于匹配Exchange与Queue的绑定。





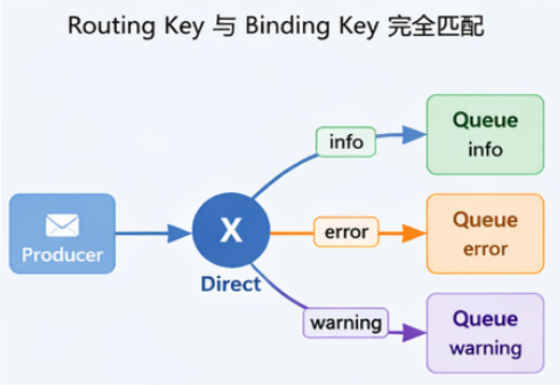
03 RabbitMQ中Exchange类型

生产者把消息发给 Exchange 后，RabbitMQ 到底是怎么决定“把消息送到哪个队列”？

- **Exchange 的作用**：接收生产者发送的消息，并按照规则路由到一个或多个队列
- **为什么不直接发给 Queue?**
  - 生产者不应该关心具体消费队列
  - 便于解耦、扩展和灵活路由
- **不同业务需要不同路由方式**
  - 精确投递 Direct
  - 广播投递 Fanout
  - 按模式匹配投递 Topic

03 Direct Exchange

- **路由规则**：Routing Key（消息携带）与 Binding Key（队列设置） **完全匹配**
- **特点**
  - 路由简单
  - 适合精确分类投递
- **适用场景**
  - 日志等级分发：info / warning / error
  - 订单按业务类型分发
  - 不同任务发送到不同处理队列

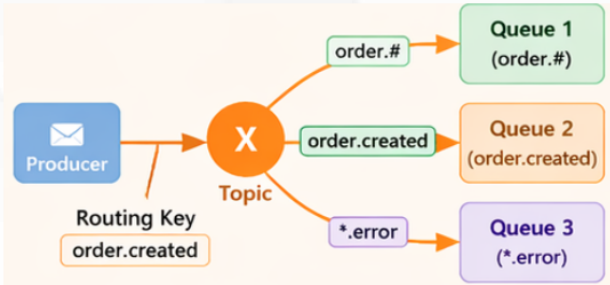


03 Fanout Exchange

- **路由规则：**忽略 Routing Key，把消息**广播发送**到所有绑定队列
- **特点**
  - 一条消息可被多个消费者同时接收
  - 适合广播通知
- **适用场景**
  - 系统公告
  - 配置更新通知
  - 聊天室群发消息
  - 缓存失效广播

03 Topic Exchange

- **路由规则**：Routing Key 与 Binding Key按**模式匹配**
- **常用通配符**：
  - *：匹配一个单词
  - #：匹配零个或多个单词
- **示例**
  - order.created
  - order.paid
  - user.login
- **适用场景**
  - 事件分类分发
  - 微服务事件总线
  - 日志按模块与级别分发

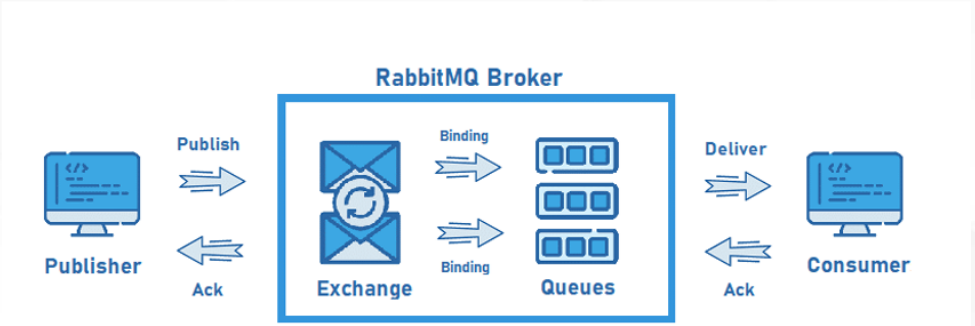


Topic Exchange 特别适合事件驱动架构。
事件天然就适合按照“领域.动作”这种方式命名

绑定键	含义
order.*	匹配 order.created、order.paid
*.error	匹配 db.error、app.error
order.#	匹配所有 order 开头的主题

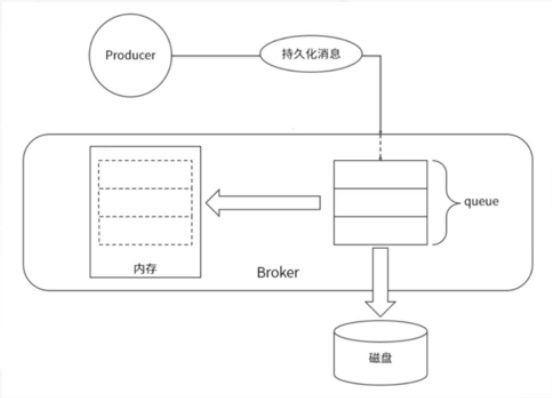
消息确认机制（ACK）

- 消息确认机制（ACK）：
  - 消费者处理完消息后发送ACK（确认）消息给RabbitMQ，告知消息已成功处理
  - 作用：确保消息不会丢失，只有当消费者确认后，消息才被标记为已处理。

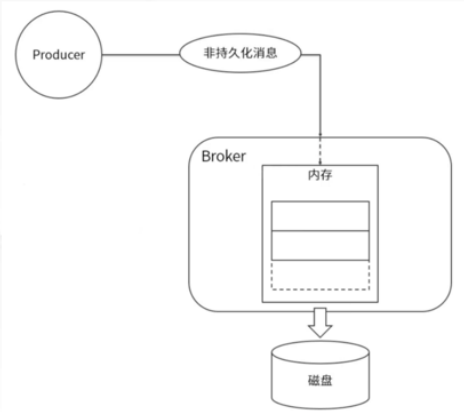


03 消息持久化

- 消息持久化：
  - 持久化消息会同时写入磁盘和内存（加快读取速度）
  - 非持久化消息会在内存不够用时，将消息写入磁盘（一般重启之后就没有了）
  - 作用：增强RabbitMQ的可靠性和持久性



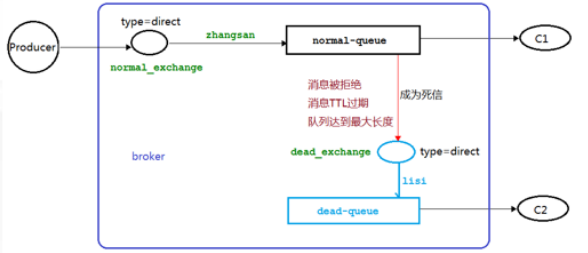
持久化



非持久化

04 死信队列（DLQ）与延迟消息

- **死信队列（Dead Letter Queue）**：当消息无法被正常消费时，消息将被转移到的专门队列
  - **死信（Dead Letter）**：无法被正常消费的消息
  - 例如：用户在商城下单成功并点击去支付后在指定时间未支付时自动失效
- 死信发生的常见原因：
  - 消息 TTL （Time To Live）过期
  - 队列长度达到最大值，消息被丢弃
  - 消息被拒绝
- **延迟消息**
  - 通过设置消息的延迟时间（如消息处理失败后，隔段时间再消费），消息会在指定的时间后再被消费。
  - 实现方式：延迟交换机（Delayed Message Exchange）来指定消息的延迟时间



西南交通大學
Southwest Jiaotong University

云计算技术

第7讲 分布式存储系统

课程代码：SCAI004712

授课教师：丛荣

2026年4月

03 CAP定理

CAP 定理（Brewer's Theorem）：由 UC Berkeley 的 Eric Brewer 于 2000 年提出，2002 年由 Gilbert & Lynch 正式证明。

核心结论：分布式系统中，一致性（C）、可用性（A）、分区容错（P）三者最多同时满足两个。

类比：餐厅同时做到"菜品新鲜（C）、上菜飞快（A）、食材断货也不停业（P）"——现实中三者很难兼顾，总要取舍。

特性	英文	含义
一致性 (C)	Consistency	所有节点同一时刻看到相同数据。 例：银行转账后，所有查询立即看到新余额
可用性 (A)	Availability	每个请求都收到响应（不保证是最新数据）。 例：系统总能回答，即使答案可能稍旧
分区容错 (P)	Partition Tolerance	节点间通信故障时系统仍可运行。 例：机房网线断了，各节点独立继续服务

04 CAP定理

西南交通大学 Southwest Jiaotong University

为什么只能三选二？ 网络故障（P）在真实环境中无法避免。分区发生时，节点 A 和节点 B 断联——此时只有两条路：

✔ 选 C（一致性）：拒绝响应，等节点联通后再服务 → 牺牲可用性（A✘）

✔ 选 A（可用性）：继续服务，可能返回旧数据 → 牺牲一致性（C✘）

选择	含义	典型系统
CP	网络分区时拒绝服务，确保数据绝对一致 ——宁可不响应，不返回脏数据	ZooKeeper（分布式协调） etcd（K8s 配置中心）、HBase
AP	网络分区时继续响应，允许短暂数据不一致 ——电商库存展示可短暂偏差	Cassandra、DynamoDB Couchbase（购物车、用户动态）
CA	理论上不存在：真实分布式系统无法完全避免网络分区，CA 仅适用于单机场景	MySQL / PostgreSQL（单节点部署） 注：集群模式下也要面对 P

05 CAP定理的现实理解

误解	纠正
CAP 三选二	不是静态"三选二"。P（网络分区）几乎不可避免， 实际是在分区发生时选 C 还是选 A
AP 系统完全不一致	AP 系统提供的是最终一致性，不是永远不一致。 例：朋友圈点赞数会在几秒内同步到所有人
CP 系统完全不可用	CP 系统只在分区期间拒绝服务，分区恢复后立刻恢复。 例：ZooKeeper 短暂不可用后自动重新选主恢复

📖**延伸：PACELC 模型**（Daniel Abadi 提出）：CAP 只关注"出故障时怎么选"。系统正常运行时同样面临**延迟（L）vs 一致性（C）**的权衡——是等所有副本同步后再响应（一致但慢），还是立即返回（快但可能读到旧数据）？

06 CAP定理的现实理解

西南交通大学 Southwest Jiaotong University

PACELC： 发生网络分区（P）时 → 选可用（A）还是一致（C）？ | 正常运行（E=Else）时 → 选低延迟（L）还是强一致（C）？

系统	分区时 P → A or C?	正常时 E → L or C?
DynamoDB	PA （优先可用）	EL （低延迟优先）
Cassandra	PA （优先可用）	EL （低延迟优先）
HBase	PC （优先一致）	EC （一致性优先）
MongoDB	PA （优先可用）	EC （一致性优先）

07 BASE理论

BASE 是对 CAP 中 AP 路线的进一步阐述，是大型互联网系统（淘宝、微信、亚马逊）常用的设计理念。

核心思想：放弃强一致性，换取高可用和高性能。

缩写	全称	含义
BA	Basically Available	允许部分功能降级或响应延迟，但系统整体依然可用。 例：淘宝大促时搜索变慢、但下单功能正常
S	Soft State	数据不需要时刻一致，允许副本间短暂不同步。 例：微博点赞数在各数据中心可短暂不一致
E	Eventually Consistent	停止写入后，经过一段时间所有副本数据趋于一致。 例：DNS 更新后，全球各地 IP 解析逐步同步到新值

08 BASE理论

ACID vs BASE ——两种截然不同的数据库设计哲学

维度	ACID (atomicity, consistency, isolation, durability)	BASE (Basically Available, Soft State, Eventual Consistency)
全称	原子性 / 一致性 / 隔离性 / 持久性	基本可用 / 软状态 / 最终一致
一致性	强一致性	最终一致性
可用性	可能降低（等待同步）	高可用
适用场景	银行转账、订单支付	社交动态、商品库存展示
代表系统	MySQL、PostgreSQL	DynamoDB、Cassandra

⚠BASE 并非"放任数据随便乱"，而是采用**最终一致性**，需要设计**冲突解决机制**（如读修复、反熵协议、向量时钟）。

选择依据：需要强一致性（金融/订单）→ 选 ACID；需要高可用高吞吐（社交/推荐）→ 选 BASE。

2 数据分片策略（Sharding）

- 数据分片
- 哈希分片
- 虚拟节点

13 为什么需要数据分片?

数据分片（Sharding）：将大规模数据集分割成多个子集并分布存储在不同节点，每个节点只负责部分数据的读写，实现水平扩展、突破单机瓶颈。

容量瓶颈	性能瓶颈	可用性风险
存储上限：单机磁盘有限，无法应对 PB 级数据 📌微信图片存储 > 100 PB，单机根本装不下	读写 QPS 存在物理上限，单机无法撑住高并发 📌淘宝双11峰值写入 > 1亿次/秒，远超单机极限	单点故障风险高，一台宕机即全库不可用 📌单库 MySQL 故障 → 整个业务停摆

■ 两大分片策略

#️⃣哈希分片（Hash Sharding）

按 Key 的哈希值对节点数取模，决定数据存入哪个节点
☑优点：分布均匀，负载均衡好
⚠缺点：节点数变化时大量数据需迁移（扩容代价高）
📌代表：Redis Cluster（CRC16 % 16384）

📏范围分片（Range Sharding）

按 Key 的值范围划分区间，同一范围的 Key 存入同一节点
☑优点：范围查询高效（顺序扫描）
⚠缺点：容易产生数据倾斜（热点节点）
📌代表：HBase、BigTable、Google Spanner

14 哈希分片（Hash Sharding）

西南交通大学 Southwest Jiaotong University

■ 核心原理

对数据的 Key 计算哈希值，然后对节点数 N 取模，决定数据存储在哪个节点：

node_id = hash(key) % N

例：hash("user:1001") % 3 = 2 → 存入 Node 2

```
// 哈希分片核心公式
node_id = hash(key) % N
// 3节点示例
hash("user:1001") % 3 = 2  // → Node 2
hash("order:5523") % 3 = 0  // → Node 0
// Redis Cluster 实际使用
slot = CRC16(key) % 16384
```

☑主要优点

- 数据均匀分布各节点，有效实现负载均衡
- 写入压力分散，避免单节点热点
- 代表：Redis Cluster 用 CRC16(key) % 16384 分配 16384 个槽位

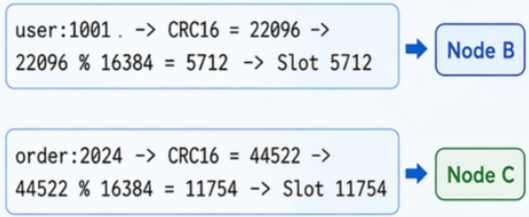
⚠致命缺点

节点数 N 变化时，几乎所有数据映射关系都改变，引发大规模数据迁移：

- 3台 → 4台：hash(key)%3 变为 hash(key)%4
  约 75% 的数据需要重新迁移！
- 无法支持高效范围查询（需扫描所有节点）

💡解决方案：使用一致性哈希（→ 下一页）

▲ 哈希分片示意图：3个节点均匀承载数据



15 一致性哈希（Consistent Hashing）

■ 设计目标

解决传统哈希节点增减时大量数据迁移的问题：

- 3台→4台，传统哈希迁移约 75% 数据
- 一致性哈希只需迁移约 1/N ≈ 25% 数据

■ 核心原理（Hash Ring）

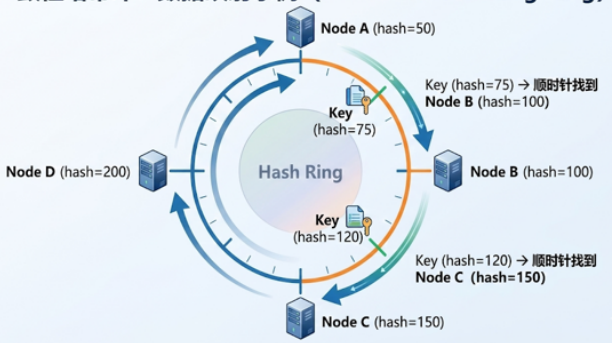
哈希空间组成虚拟圆环（0 ~ $2^{32}-1$），节点和 Key 均映射到环上。

- Key 顺时针找最近节点存入
- 类比：钟表盘——数据找顺时针方向最近的刻度（节点）

☑节点增减优势

删节点：只将该节点数据迁给顺时针下一节点
加节点：只将新节点覆盖区间的数据迁入
结论：扩/缩容只影响 1/N 的数据！
虚拟节点：每个物理节点映射多个虚拟节点，使分布更均匀
代表系统：Cassandra、DynamoDB、Riak

一致性哈希环：数据映射示例（Consistent Hashing Ring）



▲ *哈希环示例*

Node A/B/C 均匀分布在环上，数据 key 顺时针找最近节点存入。

典型系统： Cassandra、DynamoDB、Riak

16 一致性哈希 (Consistent Hashing)

西南交通大学 Southwest Jiaotong University

对比	传统哈希	一致性哈希
增删节点	几乎所有数据迁移 (N-1)/N ≈ 91%（10→11节点）	只影响相邻节点 约 1/N ≈ 9%（10→11节点）
迁移量公式	~(N-1)/N × 总数据量 10个节点扩到11个：迁移91%	~1/N × 总数据量 10个节点扩到11个：仅迁移9%
查询路由	直接计算：node = hash(key) % N 节点数变化则公式失效	顺时针找最近节点 节点增减不影响其他区段路由

一致性哈希并没有让查询速度变快！

类比： 快递公司新增一个配送站，只需把新站附近的包裹移交，其他线路完全不受影响。
传统哈希就像所有包裹都要重新分配站点——加一个站点，全员搬家，代价极高。

数字说话（10 → 11 个节点）： 传统哈希迁移 ≈ 90% 数据 **一致性哈希仅迁移 ≈ 9%** ✔

17 虚拟节点（Virtual Nodes）

⚠**一致性哈希的遗留问题**：节点在哈希环上的落点是随机的，少量节点时分布容易不均，某个节点承接了一大片区间，成为热点。

解法：虚拟节点（VNode）

每个物理节点在哈希环上注册多个"分身"，这些分身均匀散落在环的各处。数据照样顺时针找最近的节点，只不过现在找到的是某个分身，实际由对应的物理节点处理。

钟表盘比喻：3台服务器，各放3个贴纸，9个贴纸均匀贴满表盘——每台服务器大约各管三分之一。

```
节点 A  →  A-1(hash=10)   A-2(hash=80)   A-3(hash=190)
节点 B  →  B-1(hash=40)   B-2(hash=120)  B-3(hash=230)
节点 C  →  C-1(hash=60)   C-2(hash=150)  C-3(hash=250)
```

虚拟节点越多，分布越均匀。实践中通常每个物理节点对应 **100～200 个虚拟节点**，Cassandra、DynamoDB 均采用此方案。

18 范围分片（Range Sharding）

西南交通大学 Southwest Jiaotong University

原理：按 Key 的值范围将数据划分到不同节点，每个节点负责一段连续区间。查找数据时直接定位到对应区间的节点，无需遍历。

示例（按用户名首字母分片）：

```
Node A: key ∈ [A, G)  →  alice, ben, fan
Node B: key ∈ [G, N)  →  han, kim, max
Node C: key ∈ [N, Z]  →  nao, sam, zhang
```

热点问题与解法：

若 Z 开头有大量明星账号，Node C 成热点 → 可对热点区间再细分，或引入二级哈希打散。

典型系统：HBase（RowKey 范围自动分区）、Spanner、BigTable

☑优点

- 范围查询高效——只访问 1 个节点（如查所有 A-F 用户）
- 数据有序存储，天然支持排序和分页
- 分区边界可灵活调整（自动或手动再分裂）

⚠ 缺点

- 数据倾斜——热点 Key 集中时某节点压力过大
- 需要维护分区元数据（映射表），扩容要重新分裂区间

📌适用场景

- 时序数据（日志、监控，按时间段查）
- 用户数据按 ID 段分片（用户量均匀分布时）
- 需要频繁做范围扫描的 OLAP 场景

19 分片策略对比总结

西南交通大学 Southwest Jiaotong University

哈希分片（Hash Sharding）

- 公式：node = hash(key) % N
- ✅数据分布均匀，避免热点
- ⚠不支持范围查询；扩容需大量迁移
- 改进：一致性哈希 + 虚拟节点（迁移量降至 1/N）
- 典型：Redis Cluster（16384槽）、DynamoDB、Cassandra

范围分片（Range Sharding）

- 公式：按 Key 值区间分配节点
- ✅范围查询高效；有序存储；支持排序分页
- ⚠数据倾斜时产生热点；需维护分区元数据
- 典型：HBase（RowKey 自动分区）、BigTable、Spanner

如何选择？

选哈希分片，当：

- 需要均匀分散写压力（如用户行为日志）
- 主要是点查（按 Key 精确查找），不做区间扫描
- 例：用户账户系统、缓存集群、消息队列

选范围分片，当：

- 业务依赖范围查询（如按时间段查日志）
- 数据有天然顺序、需要排序或分页
- 例：时序数据库、订单按日期分区、监控指标

💡**实践中两者常结合使用**：先按业务维度哈希分片，再在单片内按时间范围分区（二级分片）。

28 向量时钟示例

向量时钟（Vector Clock）记录每个节点的写入次数，用于检测并发写入和冲突。

① 初始状态

x 的向量时钟初始化为零

版本：{A:0, B:0}

→

② Node A 写入 x=1

A 节点写入，A 的计数 +1

版本：{A:1, B:0}

③ Node B 并发写入 x=2

B 节点同时写入，B 的计数 +1

版本：{A:0, B:1}

→

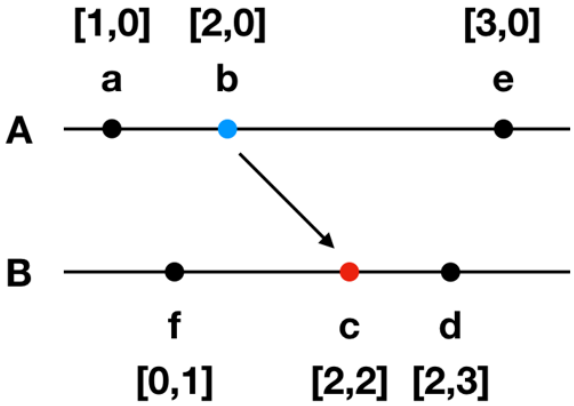
④ 检测到并发冲突

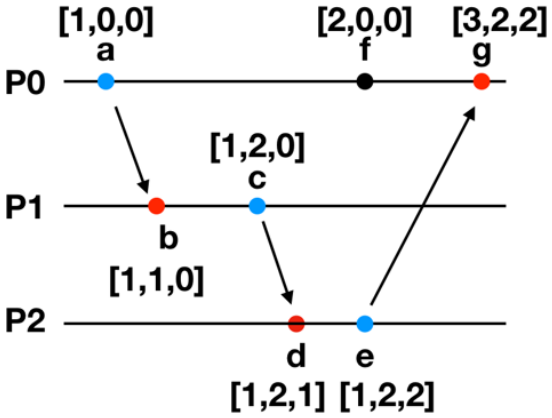
{A:1,B:0} 与 {A:0,B:1} 互不包含

⚠冲突！需应用层合并 → {A:1, B:1}

结论：向量时钟能精确检测（不是解决！）并发写入，但冲突合并需交由应用层处理

28 向量时钟示例





如果一个版本在所有维度上都不小于另一个，并且至少一维更大，就说明它“包含”另一个，是后继版本

48 列族存储对比总览

维度	HBase	Cassandra	Google Bigtable（云服务）
开发者	Apache	Apache (Facebook)	Google
数据模型	BigTable	BigTable + Dynamo	BigTable
一致性	强一致（CP）	可调一致性（AP）	强一致
架构	Master-Slave	P2P 去中心化	全托管
扩展性	好	极好（线性）	极好
运维	复杂	中等	无需运维
生态	Hadoop 生态	独立	GCP
适用场景	大数据分析 + 随机读写	高写入 + 全球分布	GCP 大数据

选型口诀：已有 Hadoop 生态、要强一致 → **HBase**；全球部署、要高可用 → **Cassandra**；用 GCP、要免运维 → **Cloud Bigtable**；需结构化查询则考虑 NewSQL。

NewSQL的定义与背景

■ 是什么？NewSQL 的定义

- 兼备 SQL 的关系型模型与 ACID 事务
  同时具备 NoSQL 的水平扩展能力
  “鱼与熊掌兼得”的新一代关系型数据库

■ 为什么出现？背景与动机

- 传统 SQL：扩展性差，难以支撑海量写入
  NoSQL：放弃事务，应用层值平高
  云原生腰力指数级富需求新层次

■ 代表产品

Google Spanner
全球分布式 SQL，无限扩展，TrueTime 保证外部一致性

CockroachDB
开源 NewSQL，兼容 PostgreSQL，跨 Region 多活

TiDB
国产 NewSQL，MySQL 兼容，HTAP 混合负载

60 分布式存储全景图

西南交通大学 Southwest Jiaotong University

类别	代表系统	数据模型	一致性	典型场景
KV 存储	Redis, DynamoDB	Key-Value	AP/CP	缓存、会话、配置
文档存储	MongoDB	JSON 文档	可调	内容管理、用户画像
列族存储	HBase, Cassandra	Column Family	CP/AP	时序、日志、大数据
图数据库	Neo4j, JanusGraph	图（节点 + 边）	CP	社交网络、知识图谱
NewSQL	TiDB, Spanner	关系表	CP	金融、电商核心业务
时序数据库	InfluxDB, TimescaleDB	时间序列	AP	监控、IoT

西南交通大學
Southwest Jiaotong University

云计算技术
第8讲 分布式文件系统

课程代码：SCAI004712

授课教师：丛荣

2026年4月

1 存储类型概述

- 三类存储概念
- 性能对比
- 选型建议

01 三种存储：块存储、文件存储、对象存储

西南交通大学 Southwest Jiaotong University

块存储（Block Storage）

- 将存储空间划分为固定大小的块（Block），块作为裸设备通过访问接口提供给上层。
- 类比：一块未格式化的本地硬盘
- 访问方式：类磁盘的设备接口（ Linux的 /dev/sda）
- 代表：数据库的底层页存储 AWS EBS、Ceph RBD

文件存储（File Storage）

- 提供文件和目录的层级树状结构，通过文件路径访问数据。
- 类比：一个网络共享文件夹
- 访问方式：符合POSIX的文件路径（如 /data/file.txt）
- 代表：NFS、HDFS、CephFS

01 三种存储类型

对象存储（Object Storage）

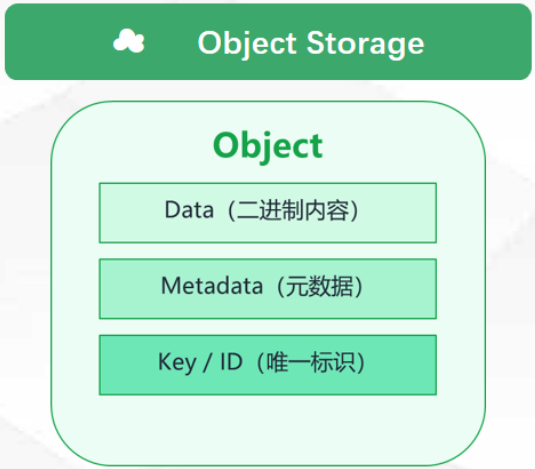
将数据作为对象（Object）存储，采用扁平命名空间。包含**数据、元数据和唯一键。**

类比：一个配有代客泊车凭证的无限大仓库

访问方式：HTTP REST API（如 GET /bucket/key）

注：key的斜杠主要是为了让人好读，而不是文件系统真正的树状目录

代表：AWS S3、MinIO、阿里云OSS



02 三种存储类型对比

维度	块存储	文件存储	对象存储
抽象层级	最低（裸盘）	中（树状目录）	最高（扁平桶）
访问接口	SCSI/iSCSI	POSIX/NFS/SMB	HTTP REST API
性能表现	极低延迟、高 IOPS	较高吞吐、中等延迟	侧重超高吞吐、延迟较高
扩展能力	较弱（受限于 SAN 架构）	中等（受限于目录树元数据）	极强（轻松扩展至 EB 级）
修改方式	支持随机修改	支持随机修改	只能整体覆盖替换
适用场景	数据库、虚拟机系统盘	多机共享日志、办公文档	海量图片 / 视频、冷数据归档

三种存储类型对比

跑系统与数据库选**块存储**，多机共享读写选**文件存储**，海量多媒体与静态资源选**对象存储**。

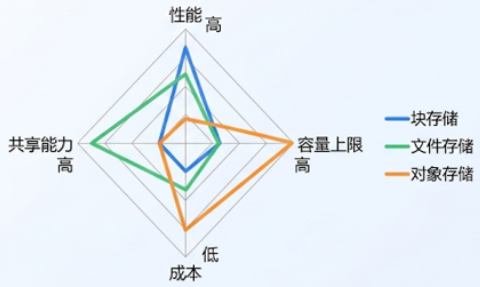
存储架构选型指南

块存储（Block Storage）
跑系统/跑DB

文件存储（File Storage）
多机共享读写

对象存储（Object Storage）
存海量多媒体/静态资源

业务场景维度倾向性对比



2 Google File System

- GFS 设计背景
- GFS 架构
- GFS 追加写入流程

01 2003年 Google面临的存储挑战

- ✓ 搜索引擎需要存储PB级网页索引数据
- ✓ 使用廉价x86商用服务器（组件故障是常态，非例外）
- ✓ 典型的负载特征：大文件为主（GB级），追加写入（Append）为主，随机修改极少
- ✓ 高吞吐率（Throughput）比低延迟更重要

PB
网页索引量级

GB
单文件平均大小

x86
廉价商用机

Append
顺序追加为主

02 GFS四大核心假设

1. **系统由大量廉价易损节点组成**（必须具备自我监控、容错、自动恢复机制）
2. **系统主要存储大文件**（数百万个100MB以上的对象，无需针对KB级小文件优化）
3. **读写模式固定**（大规模流式读取、少量随机读取；主要为顺序追加写入）
4. **并发追加语义**（多个客户端并发追加同一文件，需保证高效与一致性）

03 GFS架构：核心组件职责

Master（控制流）：唯一的主节点。管理整个文件系统元数据（目录树、文件到Chunk的映射、Chunk位置）。

ChunkServer（数据流）：存储实际数据块（Chunk）。每个Chunk默认大小64MB，通常保存3个副本。

Client（客户端）：向Master询问元数据，随后直接与ChunkServer进行数据交互。

Master
唯一主节点
管理元数据
目录树/Chunk映射

ChunkServer
存储 64MB Chunk
默认 3 副本
数据流出入

Client
先问 Master 元数据
再与 ChunkServer 直连
控制流/数据流分离

03 GFS架构：核心组件职责

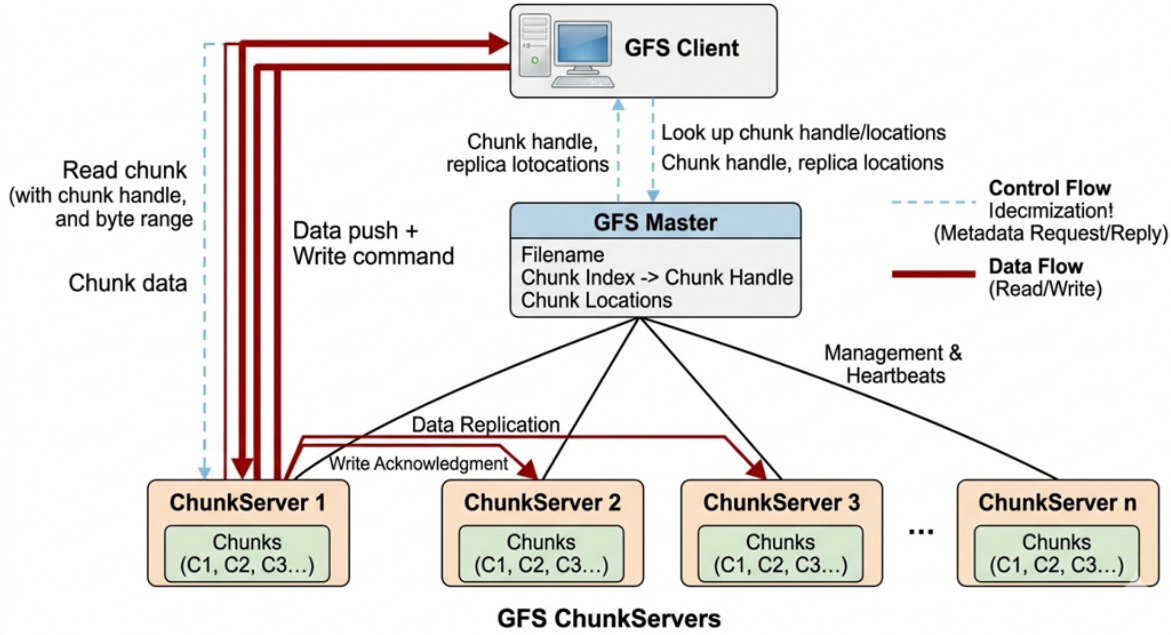
- **思考1：**chunk默认大小为什么是64MB？为什么不是操作系统常见的4KB /8KB？
- **思考2：**为什么是3副本不是4副本/2副本？

Master
唯一主节点
管理元数据
目录树/Chunk映射

ChunkServer
存储 64MB Chunk
默认 3 副本
数据流出入

Client
先问 Master 元数据
再与 ChunkServer 直连
控制流/数据流分离

GFS 架构总览（控制流 vs 数据流）



04 GFS 架构关键创新点

1.**单一Master设计**：大幅简化设计架构，避免了复杂的分布式一致性协议（如Paxos/Raft用于元数据），但通过内存映射和操作日志解决性能瓶颈。

2. **控制流与数据流分离**：Client仅找Master问路，数据传输绝不经过Master，避免Master成为网络瓶颈。

3. **大Chunk策略（64MB）**：远大于普通操作系统的4KB，极大减少了Master的元数据管理压力

05 GFS追加写入流程

西南交通大学 Southwest Jiaotong University

为什么关注追加（Record Append）而非随机读写？

- 搜索引擎、日志系统、批处理系统的很多场景，天然就是不断往文件尾部追加记录，而不是不停去中间改字节
- GFS专门优化了追加写入，允许数百个客户端并发向同一文件追加，而**不需加分布式锁。**

传统写入：随机修改
需分布式锁 · 高冲突
高延迟 · 低吞吐

GFS 追加：Record Append
数百客户端无锁并发
极高吞吐 · 适合日志场景

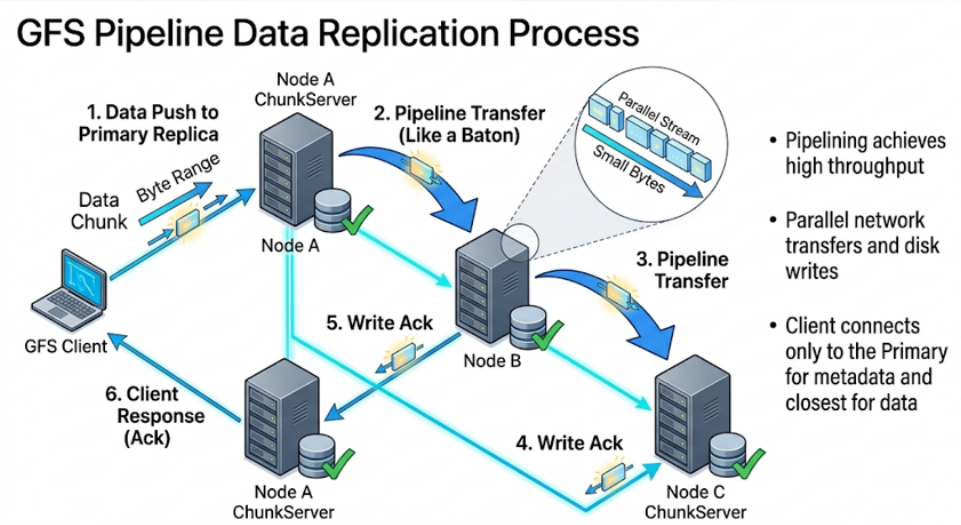
05 GFS追加写入流程

写入流程设计：流水线（Pipeline）机制

- **授权主副本**：Master在3个副本中指定一个为Primary（主），**负责确定写入顺序**。
- **数据链式推送**：Client不直接向3个副本同时发数据，而是发给最近的副本A，A推给B，B推给C（充分利用节点全双工带宽）。
- **执行写入顺序**：数据推送完成后，Client通知Primary执行，Primary定好顺序后通知所有Secondary副本按相同顺序落盘。

05 一致性妥协

一致性妥协：GFS 采用至少一次语义，GFS不保证副本字节级完全一致，可能存在重复废弃数据块并交由上层应用处理，秉持“够用就好”的哲学，为了极高的并发吞吐，放弃了完美的强一致性。



01 HDFS概述

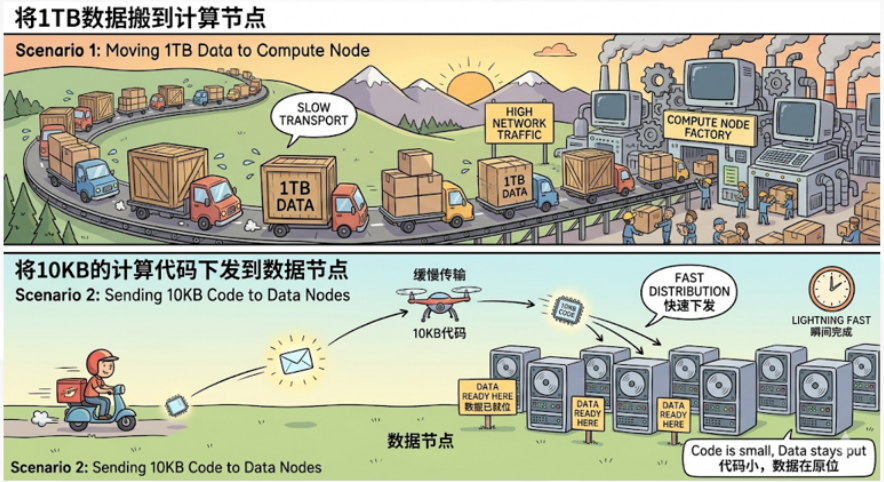
HDFS（Hadoop Distributed File System） 是Apache Hadoop大数据生态的底层基石。其设计深度借鉴了GFS论文，用Java实现。**开源！**

HDFS设计的核心取舍

设计目标	放弃的特性
超大文件处理（TB/PB 级别）	不适合海量小文件存储
流式数据访问（一次写入，多次读取）	不支持低延迟数据随机访问
高容错与低成本（运行在普通硬件）	强一致性随机修改（仅支持追加）
计算向数据移动（数据本地性）	计算与存储完全分离架构

02 计算向数据移动

传统架构是将数据读入内存进行计算（耗费网络带宽）；HDFS配合MapReduce/Spark，将计算程序调度到数据所在的DataNode上运行，实现网络零拷贝。



HDFS 架构：双心结构 (1/3)

1.NameNode（大脑）

- **职责**：维护文件系统树，管理文件和目录的元数据。
- **存储介质**：全内存运行（保证查询极快），持久化依赖FsImage（命名空间镜像）和EditLog（编辑日志）。
- **瓶颈**：1GB内存大约只能管理100万个Block，因此极度惧怕海量小文件。

HDFS 架构：双心结构 (2/3)

2. DataNode（躯干）

职责：实际存储数据的组件，以Block（默认128MB）为基本单位。

汇报机制：每3秒向NameNode发送一次心跳包（Heartbeat），每隔几个小时发送全量块报告（Block Report）。

HDFS 架构：双心结构 (3/3)

3. Secondary NameNode（秘书，非热备）

- **注意**：它不是高可用备份！
- **作用**仅为定期将EditLog与FsImage在本地进行合并（**Checkpoint**），防止日志无限膨胀，减轻NameNode启动时的压力。

⚠ **注意：Secondary NameNode ≠ 热备份**

它无法在 Active NN 宕机时直接接管

Checkpoint 流程

定期合并 EditLog + FsImage

防止日志无限膨胀

真正的热备

HDFS HA 中的 Standby NN

见 Slide 29

03 组件职责

组件	职责
NameNode	管理文件系统命名空间和元数据；存储文件→Block 映射
DataNode	存储实际数据 Block；向 NameNode 发送心跳和块报告
Secondary NameNode	定期合并 EditLog 和 FsImage（不是热备份！）
Block	数据块，默认 128MB（Hadoop 2.x+），每个 Block 3 副本

4 对象存储

- 扁平命名空间
- Amazon S3
- MinIO

01 对象存储概念

对象存储 采用扁平的命名空间（无目录层级），数据以对象（Object）为单位存储在桶（Bucket）中。

```
Bucket（桶）：my-company-assets
  ├── Object: /2024/images/logo.png  (这里的路径仅仅是一个长字符串Key，并没有真实的目录)
  │      ├── Data: [图片二进制流]
  │      └── Metadata: {Content-Type: image/png, Cache-Control: max-age=3600}
```

02 核心组成要素

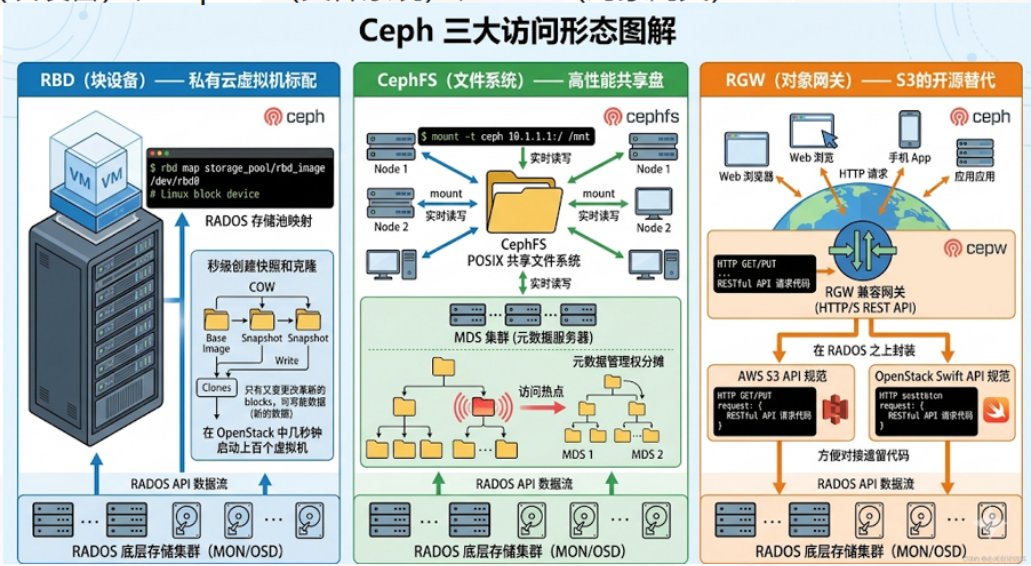
1. **Key（对象键）**：全局唯一标识符，用户眼中的“文件路径”。
2. **Value（数据体）**：被当作不透明二进制块存储的数据，最大可达5TB。
3. **Metadata（元数据）**：与数据紧密绑定！用户可以自定义标签（如 Department=Marketing），极大方便了后续的数据检索和生命周期管理。

01 Ceph：统一存储架构

Ceph 的核心愿景就是“All in One”。它底层构建了一个无限扩展的可靠分布式对象存储引擎（RADOS），在其上层“模拟”出块、文件、对象三种标准接口。一套硬件，满足所有需求。

Ceph 三大存储接口 (1/3)

RBD（块设备）、CephFS（文件系统）、RGW（对象网关）



Ceph 三大存储接口 (2/3)：RBD / CephFS

RBD（块设备）—— 私有云虚拟机标配

原理：将RADOS存储池映射成Linux下的块设备（如 /dev/rbd0）。

优势：支持秒级创建快照（Snapshot）和克隆（Clone）。在OpenStack中，用RBD启动上百个虚拟机只需几秒钟（因为利用了写时复制COW机制）。

常用：虚拟机系统盘、数据库云盘、OpenStack Cinder 后端。

CephFS（文件系统）—— 高性能共享盘

原理：提供高度兼容POSIX的文件系统，支持多台机器同时mount并实时读写。

优势：MDS集群采用动态子树分区策略。如果某个目录下文件访问量激增（热点），Ceph会自动把该目录的元数据管理权分摊给其他MDS节点。

Ceph 三大存储接口 (3/3)：RGW

RGW（对象网关）—— S3的开源替代

原理：在RADOS之上封装的一层HTTP网关转换协议。

优势：原生支持AWS S3和OpenStack Swift双重API规范，方便企业内不同时期遗留代码的对接。

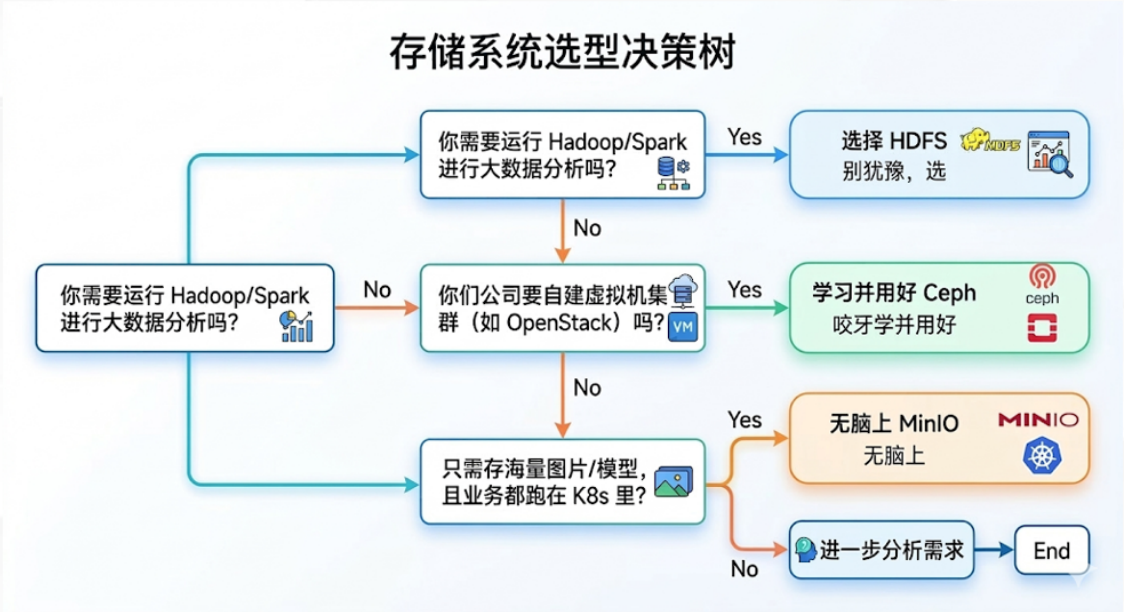
AWS S3 API
主流 SDK 直接对接
boto3 / aws-sdk
迁移零成本

OpenStack Swift API
历史代码兼容
OpenStack 生态
无需重写

01 分布式存储全景对比

对比维度	HDFS	Ceph	MinIO	GlusterFS
架构流派	中心化主从架构	去中心化统一架构	轻量级对象架构	无中心哈希架构
元数据管理	纯内存 NameNode	算法计算（CRUSH）	分布式去中心化	弹性哈希算法
核心优势	计算向数据移动	块 / 文件 / 对象通吃	极致轻量、超高吞吐	部署极其简单
痛点 / 劣势	极度害怕海量小文件	学习曲线陡、运维灾难	仅支持对象接口	小文件性能严重下滑
S3 生态兼容	不支持	较好（RGW）	完美原生兼容	不支持
适用黄金场景	**大数据批处理 (Spark 等)**	**IaaS 私有云底座 (OpenStack)**	**云原生 AI 数据湖**	**传统 NAS 平替**

CIO选型决策树


存储系统选型决策树
你需要运行 Hadoop/Spark 进行大数据分析吗?
Yes
选择 HDFS
别犹豫，选
No
你需要运行 Hadoop/Spark 进行大数据分析吗?
No
你们公司要自建虚拟机集群（如 OpenStack）吗?
Yes
学习并用好 Ceph
咬牙学并用好
ceph
No
只需存海量图片/模型，且业务都跑在 K8s 里?
Yes
无脑上 MinIO
无脑上
MINIO
No
进一步分析需求
End

西南交通大學
Southwest Jiaotong University

云计算技术

第9讲 并行编程模型

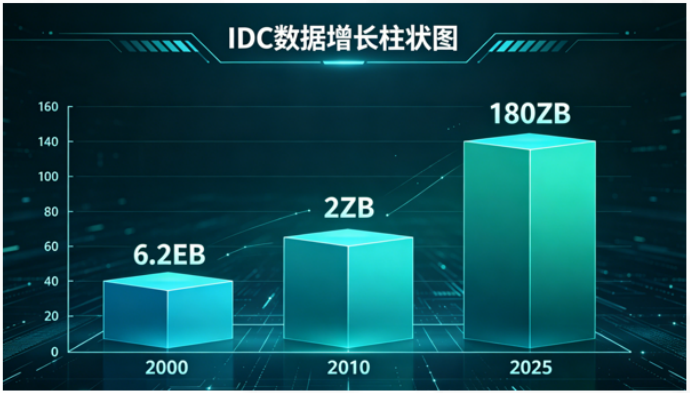
课程代码：SCAI004712

授课教师：丛荣

2026年5月

1 并行计算基础

01 为什么需要并行计算?

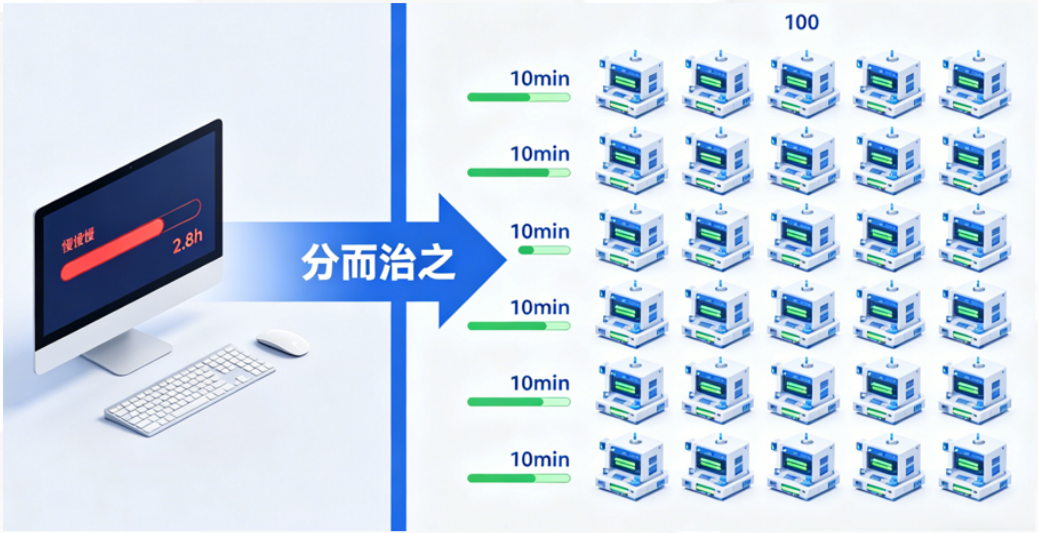


2000 → 2010 → 2025：6.2EB → 2ZB → 180ZB

单机硬盘 100MB/s × 1TB = 2.8 小时」

02 为什么需要并行计算?

1TB ÷ 100 台 = 每台 10GB，数小时 → 10 分钟



03 并行计算的三大流派

西南交通大学 Southwest Jiaotong University

数据并行

数据特征：不同数

操作特征：相同操作逻辑

代表技术

任务并行

数据特征：相同/不同数据

操作特征：不同操

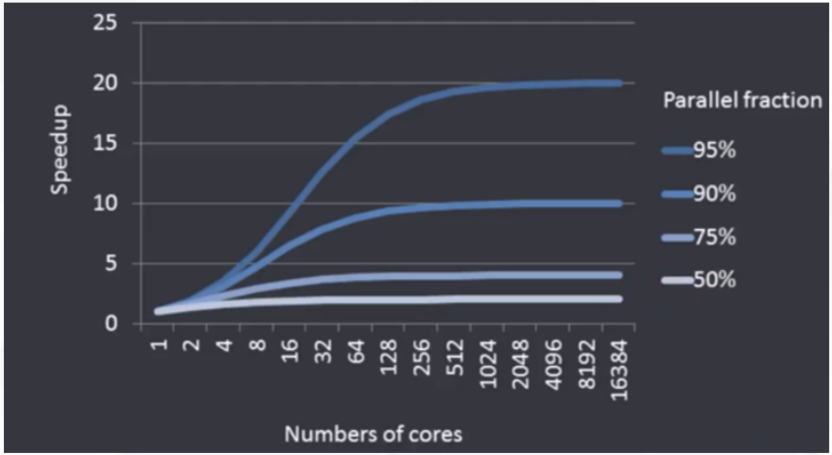
流水线

数据特征：源源不断的流式数据

操作特征：多阶段串联处理

04 Amdahl定律

加速比 S(N) = 1 / (f + (1-f)/N)
(其中 f=串行部分比例，N=处理器数量)



即便 N→∞，加速比上限 = 1/f

01 MapReduce的诞生与设计哲学

西南交通大学 Southwest Jiaotong University

历史坐标

2004年，Google在OSDI大会发表《MapReduce: Simplified Data Processing on Large Clusters》。它将复杂的分布式痛点（网络通信、容错、调度）全部封装到底层。

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

Programs written in this functional style are automatically parallelized and executed on a large cluster of commodity machines. The run-time system takes care of the details of partitioning the input data, scheduling the program's execution across a set of machines, handling machine failures, and managing the required inter-machine communication. This allows programmers without any experience with parallel and distributed systems to easily utilize the resources of a large distributed system.

Our implementation of MapReduce runs on a large cluster of commodity machines and is highly scalable: a typical MapReduce computation processes many terabytes of data on thousands of machines. Programmers find the system easy to use: hundreds of MapReduce programs have been implemented and upwards of one thousand MapReduce jobs are executed on Google's clusters every day.

1 Introduction

Over the past five years, the authors and many others at Google have implemented hundreds of special-purpose computations that process large amounts of raw data, such as crawled documents, web request logs, etc., to compute various kinds of derived data, such as inverted indices, various representations of the graph structure of web documents, summaries of the number of pages crawled per host, the set of most frequent queries in a given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new abstraction that allows us to express the simple computations we were trying to perform but hides the messy details of parallelization, fault-tolerance, data distribution and load balancing in a library. Our abstraction is inspired by the *map* and *reduce* primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a map operation to each logical "record" in our input in order to compute a set of intermediate key/value pairs, and then applying a *reduce* operation to all the values that shared the same key, in order to combine the derived data appropriately. Our use of a functional model with user-specified map and reduce operations allows us to parallelize large computations easily and to use re-execution as the primary mechanism for fault tolerance.

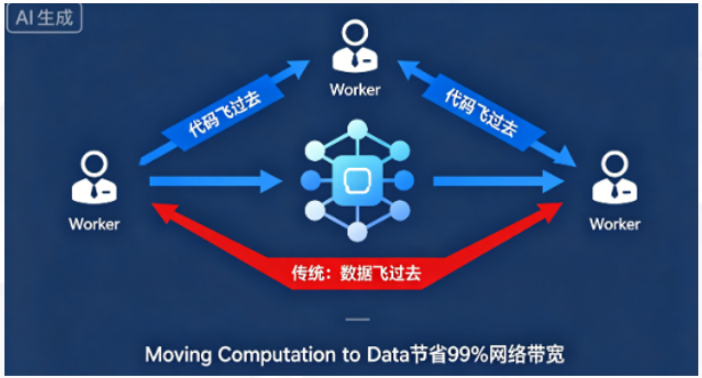
The major contributions of this work are a simple and powerful interface that enables automatic parallelization and distribution of large-scale computations, combined with an implementation of this interface that achieves high performance on large clusters of commodity PCs.

Section 2 describes the basic programming model and gives several examples. Section 3 describes an implementation of the MapReduce interface tailored towards our cluster-based computing environment. Section 4 describes several refinements of the programming model that we have found useful. Section 5 has performance measurements of our implementation for a variety of tasks. Section 6 explores the use of MapReduce within Google including our experiences in using it as the basis

To appear in OSDI 2004

1

计算向数据移动（Moving Computation to Data）：不移动庞大的数据，而是把轻量的程序代码发送到数据所在的节点执行。省下巨大的网络带宽。



03 Map与Reduce：数据变形

Map函数：数据打散与重塑

输入：键值对 <K1, V1>
输出：零个或多个中间键值对 list(<K2, V2>)
动作：过滤、提取、转换。

Reduce函数：数据聚合

输入：中间键和它的值集合 <K2, list(V2)>
输出：最终结果 list(<K3, V3>)
动作：求和、求最值、拼接。

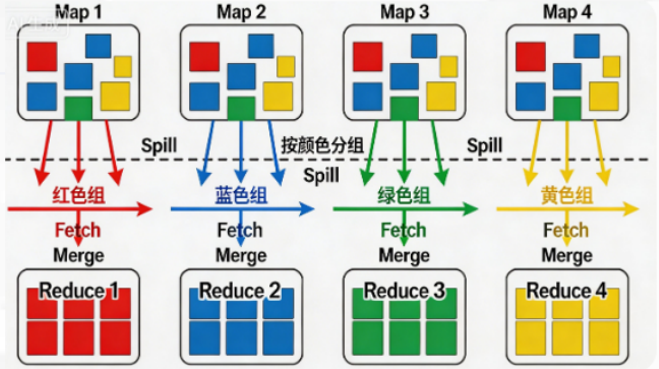
Map 过滤/提取/转换 |
Reduce 求和/求最值/拼接

键值对（Key-Value）是实现**分组（Group By）**的通用语言。同一种Key必然被送到同一个Reduce节点。

西南交通大学 Southwest Jiaotong University

什么是Shuffle?

介于Map和Reduce之间的过程。它的唯一目的是：**保证所有Key相同的中间数据，都能被送到同一个Reduce任务中去。**



Shuple = MR 性能顿顿：磁盘 I/O + 网络传输

Map端溢写（Spill）→ 网络拉取（Fetch）→ 归并排序（Merge）

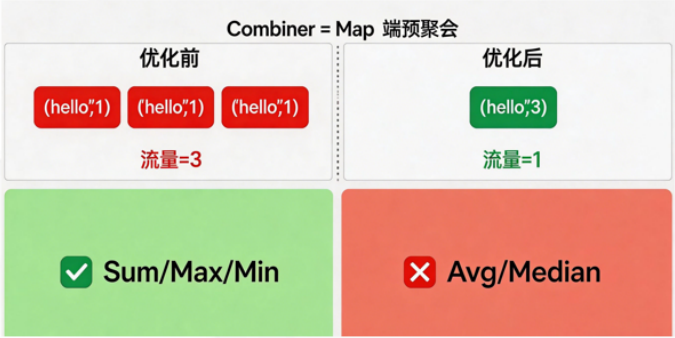
05 Combiner

为什么要引入Combiner?

既然网络传输是Shuffle的瓶颈，我们能否在数据离开Map节点前，**“就地”先合并一次**?

Combiner的作用

运行在Map端的“迷你Reduce”。
优化前Map输出：("hello",1), ("hello",1), ("hello",1) -> 传3条记录
Combiner优化后：("hello",3) -> 只传1条记录!



致命陷阱：滥用的代价

Combiner**绝不能改变最终结果**。它只适用于满足**交换律**和**结合律**的操作。
适用：Sum（求和）、Max（最大值）
不适用：Average（平均值），因为 Avg(Avg(1,3), 5) ≠ Avg(1,3,5)

01 MapReduce整体执行生命周期

核心角色与分工

1. **Client**：用户提交作业的起点。
2. **Master（JobTracker/AppMaster）**：大管家，负责拆分任务、调度和监控。
3. **Worker（TaskTracker/NodeManager）**：打工人，真正执行Map和Reduce代码的物理机。

数据的五步走

InputSplit：1TB文件按128MB切块（分片）。
Map阶段：每个分片启动一个Map Task执行。
Shuffle阶段：数据按规则在集群中大迁徙。
Reduce阶段：接收合并后的数据执行聚合。
Output阶段：最终结果写入HDFS文件系统。

02 容错机制（Fault Tolerance）

大规模集群中，**硬件故障不是意外，而是常态。**

为什么MR容错做得这么好？

无状态计算（Shared-Nothing）：Map和Reduce互不干扰，失败了重跑就行。

中间结果落盘：Map的结果写入本地磁盘，而不是直接流向Reduce。

03 推测执行（Speculative Execution）

西南交通大学 Southwest Jiaotong University

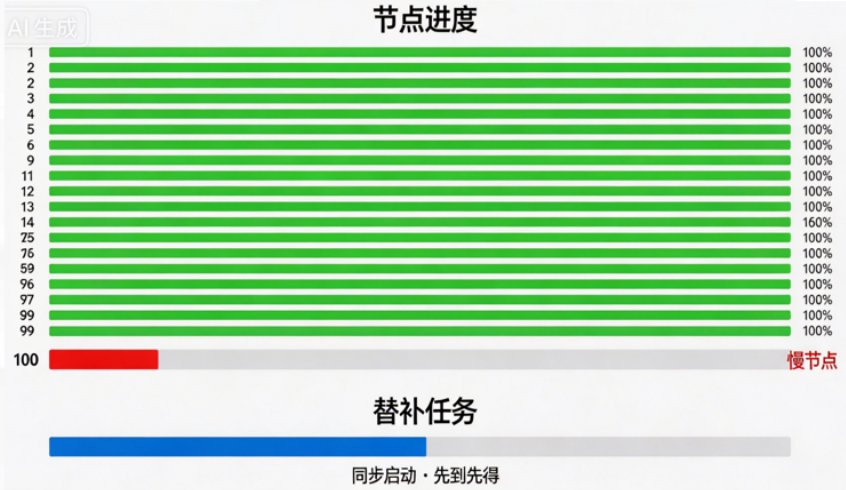
拖后腿的“长尾效应”

100个任务中，99个都只花了1分钟，偏偏最后1个任务因为所在机器硬盘老化，跑了10分钟。整个作业的时间就被这1个人拖累成了10分钟!

应对方案：找个替补

当Master发现某个Task进度**远低于**平均水平时：

1. 不去停掉这个慢任务。
2. 在其他空闲机器上，**同时启动一个一模一样的“替补任务”**。
3. 哪个先跑完，就用哪个的结果，并立刻杀掉另一个。



04 推测执行（Speculative Execution）

思考：任何任务都可以开启推测执行吗？（提示：如果任务涉及到写数据库呢？）

A、可以

B、不能

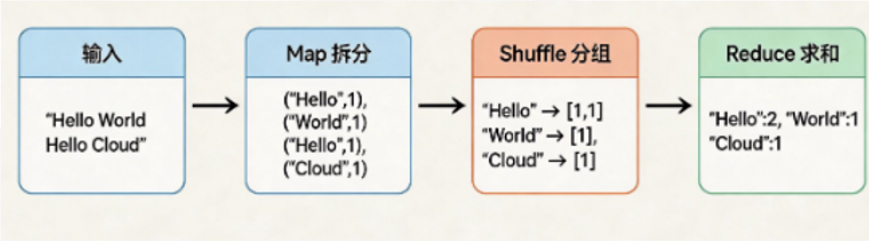
01 计数与汇总（以Word Count为例）

业务场景

网页PV统计、日志错误类型统计、词频分析。

核心逻辑解析

Map端：拆分并打标签。将一条包含多个单词的句子，拆解为多个独立的单词，并为每个单词附上数字`1`。
Reduce端：累加求和。接收同一个单词的所有`1`，进行加法运算。



02 计数与汇总（以Word Count为例）

西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# 核心 Map 逻辑
def map_function(document):
    for word in document.split():
        emit(word, 1)  # 输出格式：(Key, Value)

# 核心 Reduce 逻辑
def reduce_function(word, counts_list):
    total = sum(counts_list)  # [1, 1, 1...] 求和
    emit(word, total)
```

(注：框架会自动将Map输出的`1`聚合为`counts_list`)

05 排序与Top N

业务场景

热门商品排行榜、高频搜索词Top 10。在海量数据中找出最大/最小的N条。

核心思想：局部最优 -> 全局最优

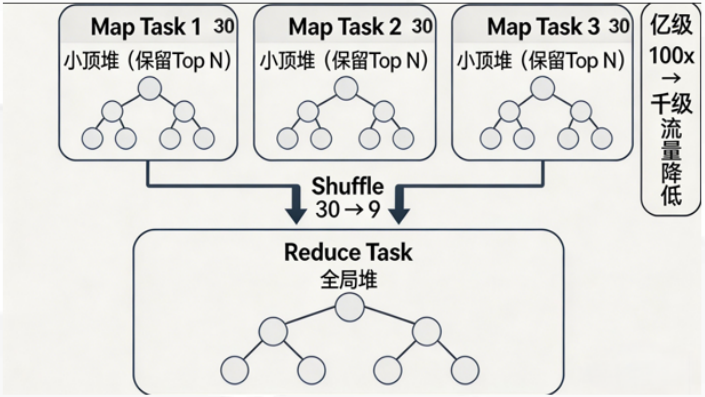
如果所有数据全丢给一个Reduce去排序，会撑爆内存！

解决之道：

1. **Map端发力**
2. **数据骤降**
3. **Reduce收尾**

核心逻辑说明

不拼全量数据，只拼局部精英。大幅度削减了网络洗牌（Shuffle）的压力。



西南交通大学 Southwest Jiaotong University

核心代码片段展示

```
# —— Map：每个节点只保留局部 TopN，大幅减少 Shuffle 传输量 ——
def topn_map(shard, N):
    heap = []
    for item, score in shard:
        heapq.heappush(heap, (score, item))
        if len(heap) > N:
            heapq.heappop(heap)  # 弹出最小值，堆中始终保留最大的 N 个
    return heap
# —— Reduce：汇总所有局部冠军，选出全局 TopN ——
def topn_reduce(candidates, N):
    return heapq.nlargest(N, candidates)  # 从所有局部 TopN 中选全局 TopN
```

排序与Top N

核心代码片段展示

```
# —— 执行 ——
N = 3
shards = [
    [("耳机", 1200), ("数据线", 300), ("手机壳", 500)],   # Map 分片 0
    [("手表", 2500), ("充电宝", 800), ("支架",  150)],   # Map 分片 1
    [("TWS", 4000),  ("键盘",   950), ("鼠标垫", 600)],   # Map 分片 2
]
local_tops = [topn_map(s, N) for s in shards]          # 各分片并行取局部 TopN
all_candidates = [item for top in local_tops for item in top]  # Shuffle 汇聚
result = topn_reduce(all_candidates, N)


for rank, (score, name) in enumerate(result, 1):
    print(f"第{rank}名: {name}  浏览量={score}")
```

```
# 预期输出
第1名: TWS  浏览量=4000
第2名: 手表  浏览量=2500
第3名: 耳机  浏览量=1200
```

西南交通大學
Southwest Jiaotong University

云计算技术

第10讲 Spark大数据处理

授课教师：丛荣

课程代码：SCAI004712

2026年5月

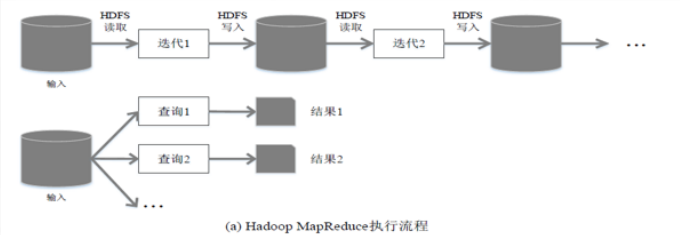
Spark的设计动机：MapReduce到Spark

Hadoop MapReduce 推动了离线大数据处理，但它的执行模型偏“批处理+磁盘中转”

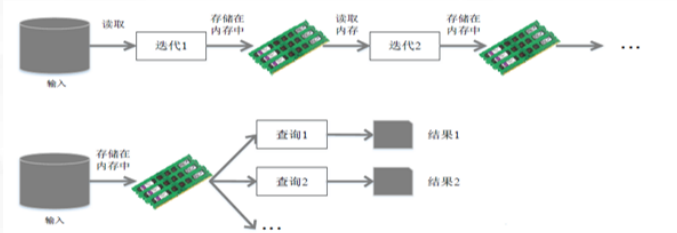
- 对日志清洗这类单次扫描任务，MapReduce 够用
- 对机器学习、交互式查询、图计算等**迭代优化**任务，中间结果反复落盘会非常慢

Spark 核心思想

- 尽量把中间结果保留在内存，减少磁盘 I/O
- DAG执行引擎
- 统一批处理与流处理



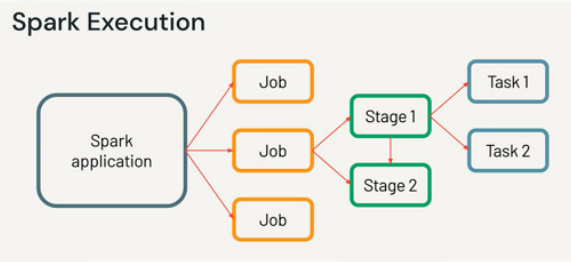
(a) Hadoop MapReduce执行流程



(b) Spark执行流程

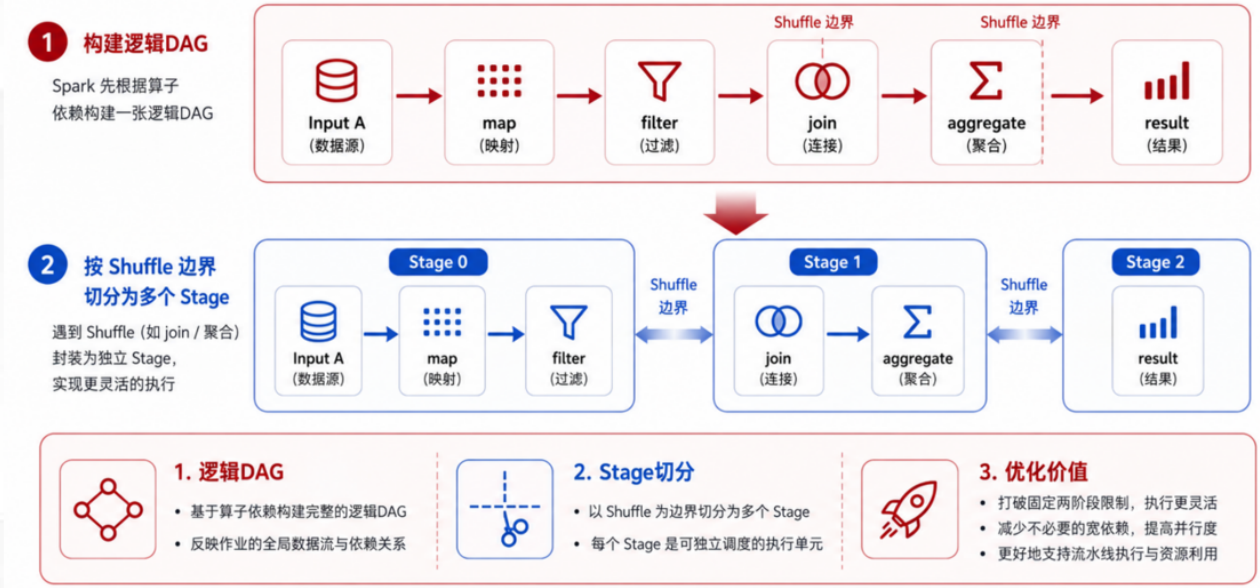
Spark：DAG 执行引擎

- Spark 把一连串 Transformation 看成一个有向无环图（DAG）
  - 节点是数据集/计算结果
  - 边表示依赖关系
- 执行流程：
  1. 先有逻辑 DAG
  2. 再拆成物理执行 Stage
     - 依据 Shuffle 边界，把 DAG 切成多个 Stage
     - Stage 内部尽量流水执行
- 好处：系统可以观察全局依赖，不只盯着单个 Map 或 Reduce



Spark：DAG 执行引擎

西南交通大学 Southwest Jiaotong University

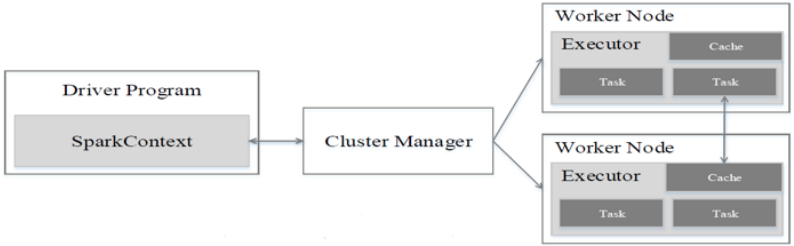


Spark 适用场景

场景	是否适合 Spark	原因
海量离线 ETL	适合	并行处理强，生态成熟
交互式 SQL 分析	适合	DataFrame + Catalyst 性能较好
迭代机器学习	适合	缓存与迭代计算优势明显
毫秒级事务处理	不适合	Spark 不是 OLTP 数据库
超低延迟实时控制	通常不适合	微批模型不一定满足毫秒级要求
超小规模数据脚本	未必需要	本地 Python / Pandas 可能更轻量

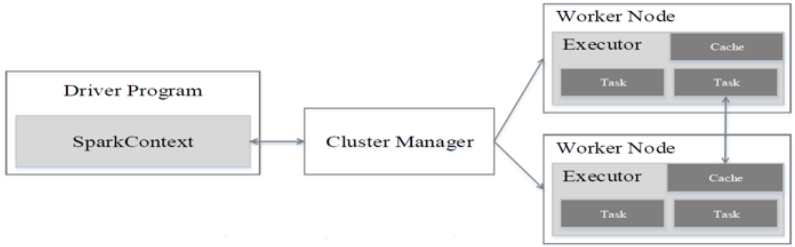
Spark 运行架构

- Spark 运行架构包括：
  - 集群资源管理器 (Cluster Manager)
  - 运行作业任务的工作节点 (Worker Node)
  - 每个应用的任务控制节点 (Driver)
  - 每个工作节点上负责具体任务的执行进程 (Executor)
- 与 Hadoop MapReduce 相比，Spark 所采用的 Executor 有两个优点：
  - 利用多线程来执行具体的任务，减少任务的启动开销。
  - Executor 中有一个 BlockManager 存储模块，会将内存和磁盘共同作为存储设备，有效减少 IO 开销。



Driver Program

- Driver 是应用的控制中心
  - 运行 main 函数
  - 创建 SparkSession
  - 解析用户写的Transformation / Action
- 当 Action 出现时，Driver 会把逻辑转换成 Job / Stage / Task，并向集群申请资源
- Driver还会维护元信息：例如哪些 RDD 被缓存、哪些 Executor 还活着、任务失败如何重试



Executor的工作模式：长期驻留

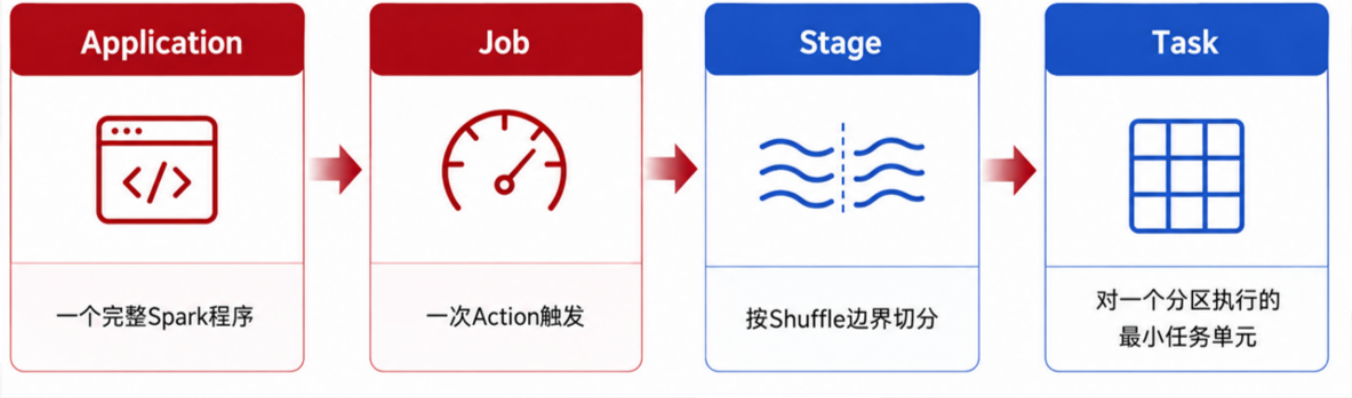
- Executor 是运行在工作节点上的进程
  - 负责接收 Driver 下发的 Task 并实际执行
  - 管理本地 Cache 与 Shuffle 数据。

- 长期驻留的优势：
  - 如果每个小任务都创建/销毁一次进程，分布式调度开销会很大
  - 长期驻留 Executor，而不是每个任务临时起进程，可以减少频繁启动进程的开销。

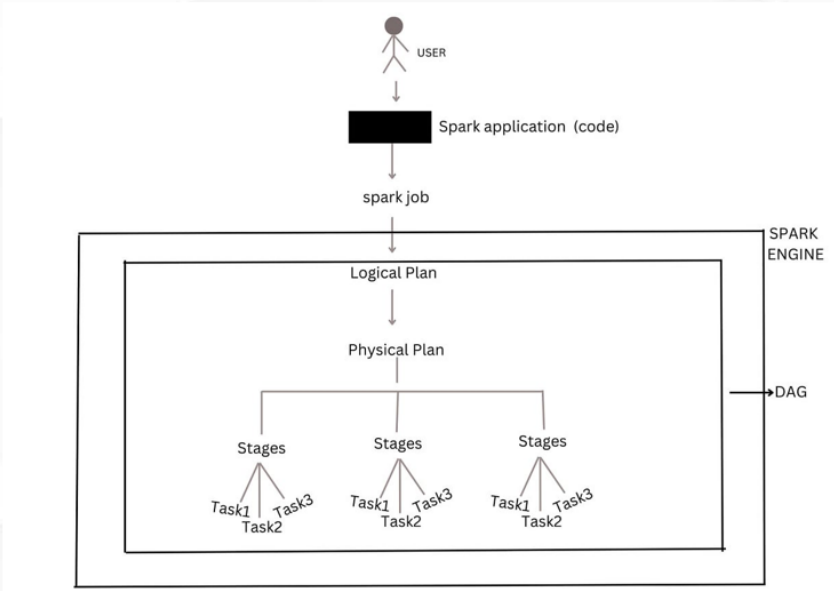
Executor的工作模式：作业执行层级

西南交通大学 Southwest Jiaotong University

分层抽象能把用户程序一步步映射到机器执行单元



Executor的工作模式：作业执行层级



RDD（Resilient Distributed Dataset）

西南交通大学 Southwest Jiaotong University

- RDD 是 Spark 最基础的数据抽象
- 用户不必担心底层数据的分布式特性，只需将应用逻辑表达为一系列转换处理
- 不同 RDD 之间的转换操作形成依赖关系，可以实现管道化，避免中间数据存储
- RDD的五大特性：分区、依赖、计算函数、分区器、首选位置

特性	含义	作用
Partitions	分区集合	决定并行度与任务划分
Dependencies	与父 RDD 的依赖	决定 Stage 与容错恢复
Compute Function	每个分区如何计算	定义转换逻辑
Partitioner	键值数据如何分区	优化聚合与连接
Preferred Locations	首选位置	提高数据本地性

Partition

Partition：

- RDD 会被切成多个分区，每个分区通常对应一个 Task
- 分区数越少，并行度可能不足；分区数过多，调度开销又会上升
- 合理分区有助于均衡负载、提高资源利用率

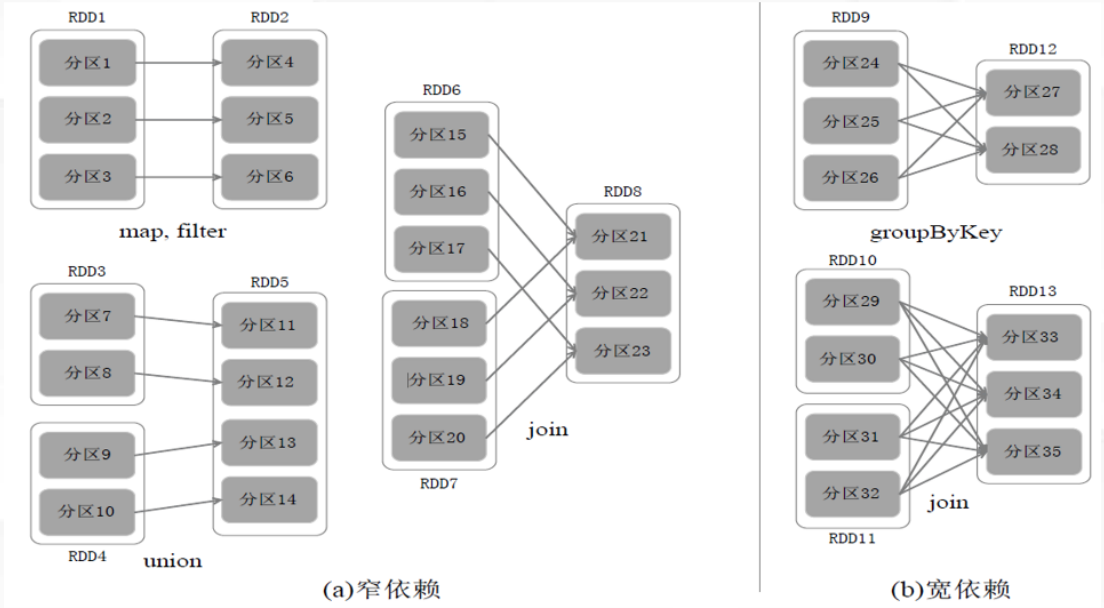
Dependency

西南交通大学 Southwest Jiaotong University

Dependency:

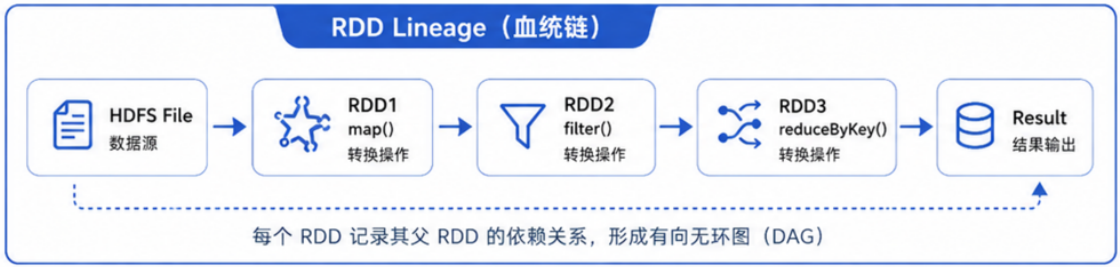
- RDD 之间通过 **Dependency** 连接
- 两种依赖：
  - **窄依赖**： 子分区只依赖少量父分区，适合流水执行，如 map、filter
  - **宽依赖**：父 RDD 的一个分区对应一个子 RDD 的多个分区。可能产生 Shuffle，如 reduceByKey、join、groupByKey
- Spark 根据依赖关系切分 Stage 并进行容错恢复

Dependency


RDD1
RDD2
分区1
分区2
分区3
分区4
分区5
分区6
map, filter
RDD3
RDD5
分区7
分区8
分区9
分区10
分区11
分区12
分区13
分区14
RDD4
union
RDD6
RDD8
分区15
分区16
分区17
分区18
分区19
分区20
分区21
分区22
分区23
RDD7
join
(a)窄依赖
RDD9
RDD12
分区24
分区25
分区26
分区27
分区28
groupByKey
RDD10
RDD13
分区29
分区30
分区31
分区32
分区33
分区34
分区35
RDD11
join
(b)宽依赖

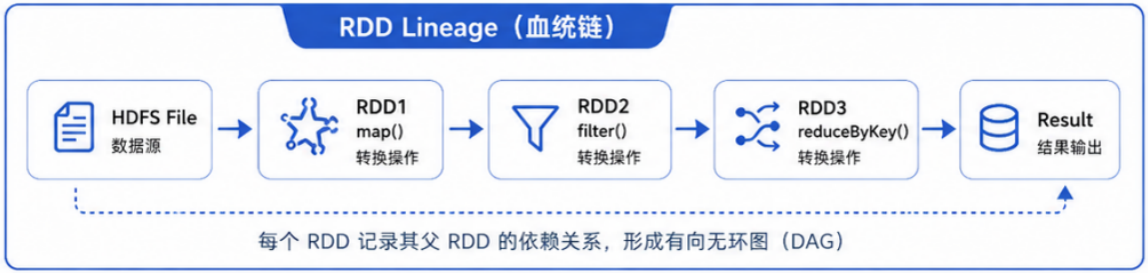
Compute Function

- 每个RDD分区如何计算
  - 每个 RDD 记录如何从父 RDD 的分区计算得到当前分区
  - **RDD 不可变**：每次 Transformation 都返回新 RDD，让血缘链更清晰，降低修改复杂性



血缘追踪（Lineage）与容错

- Spark 为 RDD 记录 **Lineage（血缘）**，而不是对每个中间结果都进行复制
- 一旦某个分区丢失，就沿着血缘链重新计算该分区
- **全量复制→按需重算**
- 如果某些数据代价太高，通过 Checkpoint 持久化提升恢复稳定性。



Transformation 与 Action

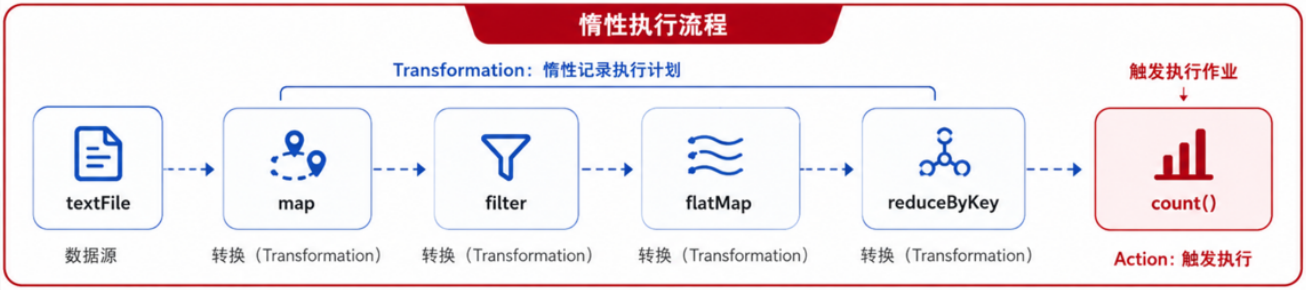
西南交通大学 Southwest Jiaotong University

计算功能和触发功能分离

类别	代表操作	特点
Transformation	map、filter、flatMap、reduceByKey	返回新 RDD，通常惰性执行
Action	collect、count、take、saveAsTextFile	触发真正计算

惰性执行（Lazy Evaluation）

- 惰性执行：Transformation 默认不会立即运行，它们只会构建一条逻辑计算链；Spark 等 Action 出现后统一执行
- 带来的好处：
  - 减少无意义中间结果
  - 给优化器留出全局优化空间
  - Spark 能做 DAG 优化与 Stage 切分的前提



PySpark RDD 示例：WordCount

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("RDDWordCount") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext
lines = sc.textFile("data/words.txt")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a + b)

print(counts.collect())
spark.stop()
```

- **textFile** 读取文本形成 RDD。
- **flatMap** 把一行拆成多个单词。
- **map** 构造 (word, 1) 键值对。
- **reduceByKey** 完成分布式聚合。

PySpark RDD 示例：值对统计

```python
data = [("A", 10), ("A", 20), ("B", 5), ("C", 8), ("B", 3)]
rdd = sc.parallelize(data, 2)

sum_rdd = rdd.reduceByKey(lambda x, y: x + y)
filtered = rdd.filter(lambda kv: kv[1] >= 8)

print("sum =", sum_rdd.collect())
print("filtered =", filtered.collect())
```

- **parallelize** 把本地集合并行化为 RDD。
- **reduceByKey** 对相同 Key 的值做聚合。
- **filter** 用条件筛选记录。
- Pair RDD 是实际业务中很常见的数据形式。

结构化数据处理的优势

为什么需要 DataFrame 与 Spark SQL?

- RDD 很灵活，但系统不知道字段名、类型和列结构，难以做高级自动优化
- **DataFrame** = 带 Schema 的分布式表
- **Spark SQL** 让用户用 SQL 描述结构化查询

name	age	city
Alice	23	Beijing
Bob	30	Shanghai
Cathy	28	Shenzhen
…	…	…

Schema（结构）

```
name: string
age: int
city: string
```

DataFrame = 数据 + 结构信息

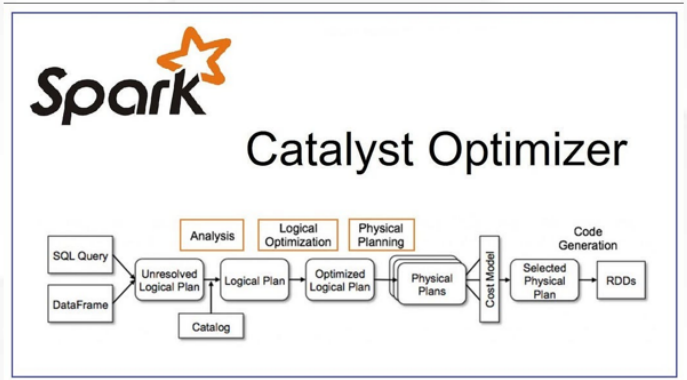
DataFrame常用API

西南交通大学 Southwest Jiaotong University

操作	作用	示例
select	列选择/表达式	df.select("name", "age")
filter/where	行过滤	df.filter(df.age > 20)
groupBy	分组聚合	df.groupBy("city").count()
orderBy	排序	df.orderBy(df.age.desc())
join	表连接	df1.join(df2, "id")

查询优化器Catalyst

- **Catalyst** 是 Spark SQL 的查询优化器
- Catalyst 的基本工作流程
  1. Logical Plan Creation
  2. Logical Plan Optimization
  3. Physical Plan Generation
  4. ~~Code Generation~~



查询优化器Catalyst

西南交通大学 Southwest Jiaotong University

- Step 2: Logical Plan Optimization
- 常见操作：
  - Push down filters
  - Combine operations
- 目的：提升优化计算效率

```
# Show the optimized logical plan
filtered_df.explain(True)
```

查询优化器Catalyst

西南交通大学 Southwest Jiaotong University

- Step 3: Physical Plan Generation
- 将逻辑计划转换为机器执行计划
- 根据数据、资源等状态，优化执行效率

```
# Physical plan explanation
filtered_df.explain()
```

```
summary.explain(mode="formatted")
```

```
# == Physical Plan ==
# AdaptiveSparkPlan
# +- Sort
#    +- HashAggregate
#       +- Exchange
#          +- HashAggregate
#             +- Filter
#                +- FileScan csv ...
```

- FileScan：读取数据。
- Filter：过滤操作。
- Exchange：发生 Shuffle。
- HashAggregate：聚合物理算子。

Tungsten内存管理与代码生成

- Step 4： Code Generation

- 好计划还不够，执行层也要足够高效
- Tungsten的职责
  - Spark内存管理
  - 堆外内存（off-heap memory） 管理
  - 代码生成

西南交通大學
Southwest Jiaotong University

云计算技术

第11讲 大规模图计算

课程代码：SCAI004712

授课教师：丛荣

2026年6月

01 图计算的典型应用场景

西南交通大学 Southwest Jiaotong University

1. 社交网络分析

好友推荐
社区发现
影响力分析

2. 推荐系统

协同过滤
商品推荐
内容推荐

3. 知识图谱

实体关系抽取
知识推理
问答系统

4. 金融风控

欺诈检测
洗钱追踪
风险传导分析

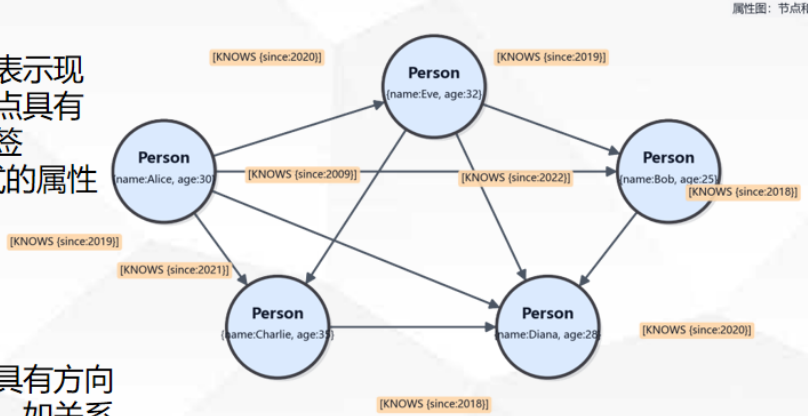
图数据的基本结构

顶点（Vertex）

顶点是图中的基本单元，用于表示现实世界中的实体对象。每个顶点具有唯一标识符（ID）、可选的标签（Label）以及一组键值对形式的属性（Properties）。

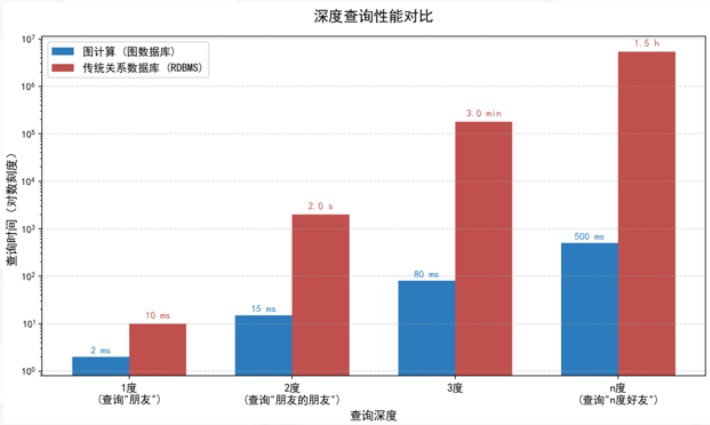
边（Edge）

边用于表示顶点之间的关系，具有方向性。边同样可以包含属性信息，如关系类型、创建时间、权重等



03 图计算 vs 传统关系数据库

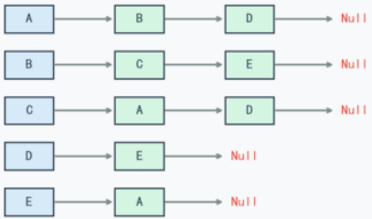
特性	图计算	关系数据库
数据模型	节点+关系	表+行+列
查询方式	遍历关系	SQL JOIN
好友查询	O(1)或O(n)	需要多表JOIN
深度查询	高效	性能急剧下降
适用场景	关系密集型	事务处理
扩展方式	边扩展	表扩展



04 图的存储模型

西南交通大学 Southwest Jiaotong University

邻接表（Adjacency List）



邻接矩阵（Adjacency Matrix）

	A	B	C	D	E
A	0	1	0	1	0
B	0	0	1	0	1
C	1	0	0	1	0
D	0	0	0	0	1
E	1	0	0	0	0

对比维度	邻接表（Adjacency List）	邻接矩阵（Adjacency Matrix）
空间占用 (Space)	O(V + E) 适合稀疏图 只存实际存在的边	O(V*V) 适合稠密图 存在大量零浪费空间
查节点所有邻居 (Find Neighbors)	O(Degree) 非常快 直接遍历对应的链表即可	O(V) 较慢 必须遍历该节点的整行/列
查两点是否相连 (Check Edge)	O(Degree) 较慢 需要在链表中顺序查找	O(1) 极快 直接通过数组下标 [i][j] 定位

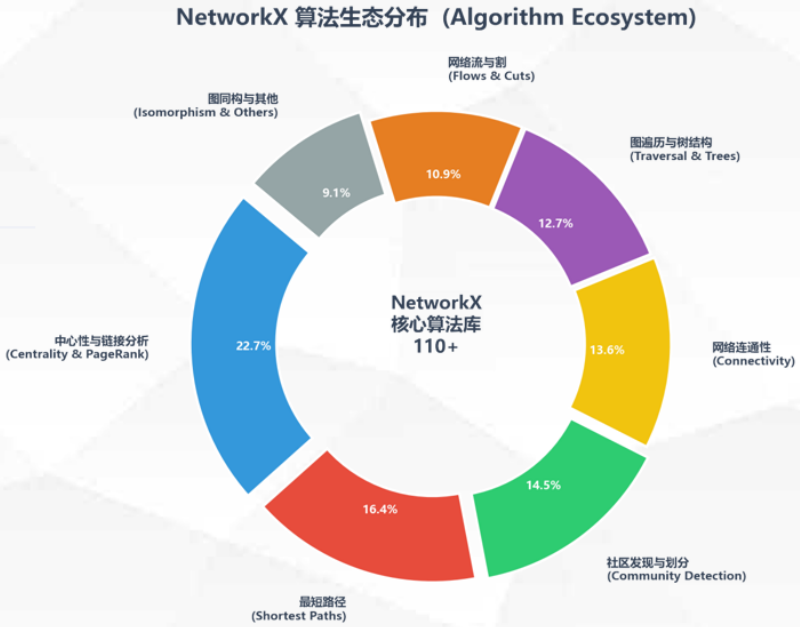
2 NetworkX图计算库

01 NetworkX概述

西南交通大学 Southwest Jiaotong University

定位与特点

1. 纯Python
2. 轻量
3. 适合教学
4. 原型验证
5. 算法库丰富



02 NetworkX的安装与基本使用

西南交通大学 Southwest Jiaotong University

安装方式

pip install "networkx[default]" matplotlib

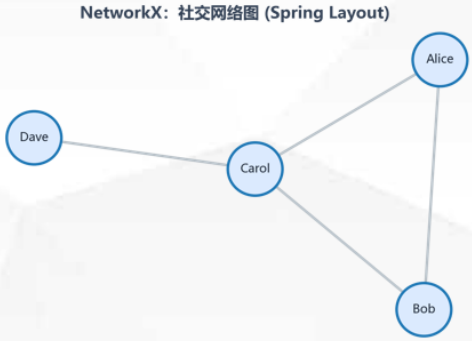
图类型	边是否有向	边是否有权重	是否允许多重边	典型应用
Graph	无向	否	否	朋友关系、无层级关系
DiGraph	有向	否	否	关注关系、网页链接
MultiGraph	无向	否	是	航班班次、交易记录
MultiDiGraph	有向	否	是	交通流量、通讯记录

03 图的创建与管理

无向图、有向图、加权图的创建

NetworkX支持多种图类型，根据数据特点选择合适的图类型：
无向图：用于表示没有方向性的关系，如同事关系、同学关系
有向图：用于表示有方向性的关系，如关注关系、转发关系、网页链接
加权图：在边属性中增加权重，表示关系的强弱或边的成本

```
# 创建无向图并添加节点和边
G = nx.Graph()
G.add_nodes_from(["Alice", "Bob", "Carol", "Dave"])
G.add_edges_from([("Alice","Bob"), ("Bob","Carol"), ("Carol","Dave"),
("Alice","Carol")])
# 查看基本信息
print(f"节点数: {G.number_of_nodes()}, 边数: {G.number_of_edges()}")
```



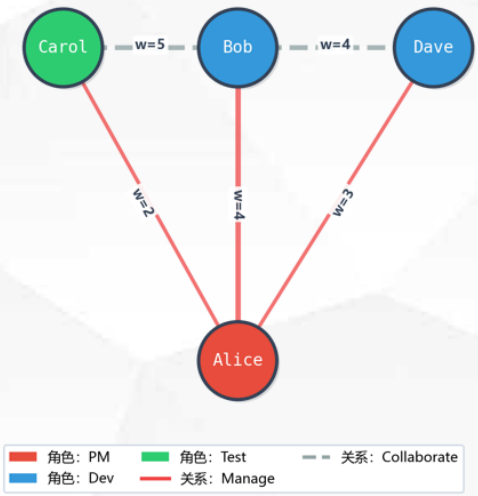
图的创建与管理


```
import networkx as nx

# 创建无向图
G = nx.Graph()
# 1. add_node: 添加单个节点 (带职务、层级属性)
G.add_node("Alice", role="PM", level=0)
# 2. add_nodes_from: 批量添加节点 (带属性字典)
G.add_nodes_from([
    ("Bob",   {"role": "Dev",  "level": 1}),
    ("Carol", {"role": "Test", "level": 1}),
    ("Dave",  {"role": "Dev",  "level": 1})
])
# 3. add_edge: 添加单条边 (带权重、关系类型属性)
G.add_edge("Alice", "Bob", weight=4, type="Manage")
# 4. add_edges_from: 批量添加边 (带属性字典)
G.add_edges_from([
    ("Alice", "Carol", {"weight": 2, "type": "Manage"}),
    ("Alice", "Dave",  {"weight": 3, "type": "Manage"}),
    ("Bob",   "Carol", {"weight": 5, "type": "Collaborate"}),
    ("Bob",   "Dave",  {"weight": 4, "type": "Collaborate"})
])

print(f"节点信息: {G.nodes(data=True)}")
print(f"连线信息: {G.edges(data=True)}")
```

包含属性的网络图（带属性增删改查）



05 图的属性管理

常用图属性方法一览

方法	功能	返回类型
G.number_of_nodes()	获取节点总数	int
G.number_of_edges()	获取边的总数	int
G.degree(n)	获取节点n的度（连接数）	int
G.neighbors(n)	获取节点n的所有邻居	迭代器
G.nodes[data=True]	获取所有节点及其属性	字典视图
G.edges[data=True]	获取所有边及其属性	字典视图
G.has_edge(u, v)	判断是否存在从u到v的边	bool

```
>>> # 查询单个节点的度数（连接数）
>>> G.degree("Alice")
2
```

01 案例1：社交网络影响力分析

业务场景

分析微博/微信等社交平台用户的影响力，识别意见领袖，为营销投放提供决策支持。

1. 识别平台中的核心用户（高影响力账号）
2. 分析信息的传播路径和速度
3. 预测热点话题的形成趋势
4. 发现潜在的KOL（关键意见领袖）

PageRank 影响力分析流程

数据采集 -> NetworkX 建图 -> PageRank -> 排序输出

02 案例1：社交网络影响力分析

示例代码

```
import networkx as nx

# 1. 构建有向社交网络
G = nx.DiGraph()
G.add_edges_from([
    ("Bob", "Alice"),
    ("Carol", "Alice"),
    ("Dave", "Alice"),
    ("Alice", "Bob"),
    ("Carol", "Bob"),
    ("Dave", "Carol"),
    ("Eve", "Dave"),
    ("Frank", "Alice"),
    ("Grace", "Bob"),
    ("Heidi", "Carol"),
])

# 2. 计算 PageRank
pr = nx.pagerank(G, alpha=0.85)

# 3. 取 Top 5 KOL
top5 = sorted(pr.items(), key=lambda x:
x[1], reverse=True)[:5]

# 4. 输出表格
print("+------+--------+------------+")
print("| Rank |  KOL   | PageRank   |")
print("+------+--------+------------+")

for rank, (name, score) in
enumerate(top5, start=1):
    print(f"| {rank:<4} | {name:<6} |
{score:<10.4f} |")

print("+------+--------+------------+")
```

示例输出

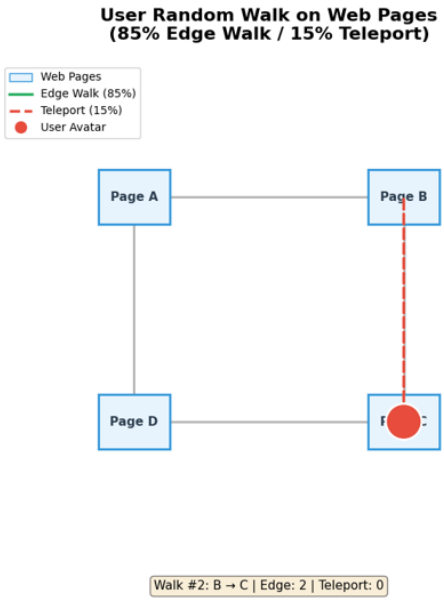
```
+------+--------+------------+
| Rank |  KOL   | PageRank   |
+------+--------+------------+
| 1    | Alice  | 0.3962     |
| 2    | Bob    | 0.3304     |
| 3    | Carol  | 0.1331     |
| 4    | Dave   | 0.0564     |
| 5    | Eve    | 0.0316     |
+------+--------+------------+
```

3 PageRank算法

02 PageRank迭代计算原理

西南交通大学 Southwest Jiaotong University

随机游走模型



03 PageRank迭代计算原理

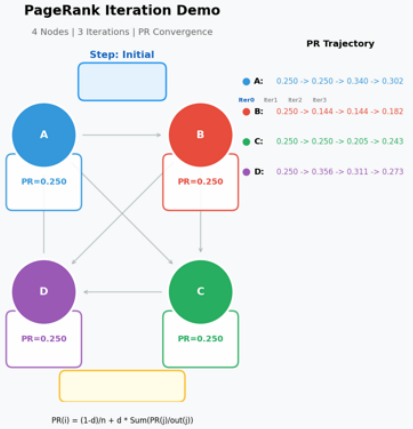
西南交通大学 Southwest Jiaotong University

迭代计算公式

$$PR(A) = \frac{1-d}{N} + d \times \left(\frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)} \right)$$

公式各参数含义：

PR(A)：页面A的PageRank值
d：阻尼因子（Damping Factor），通常取0.85
N：总页面数
L(B)、*L(C)*、*L(D)*：页面B、C、D各自的链出数量
PR(B)/L(B)：页面B分配给A的投票权重



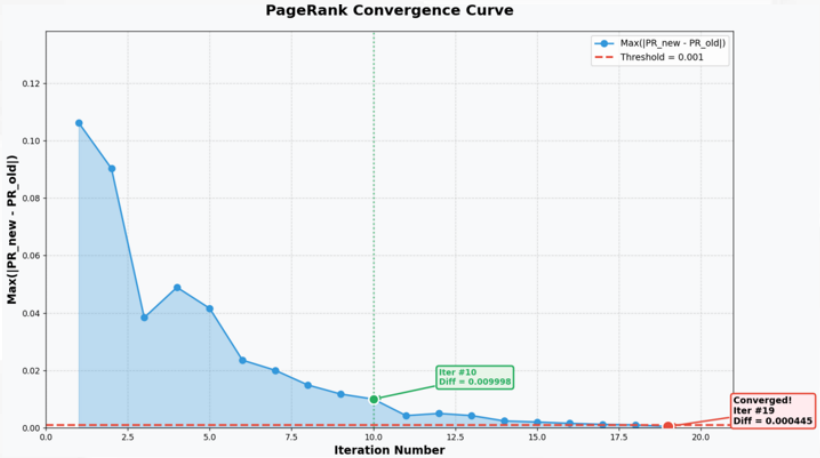
04 PageRank收敛条件详解

西南交通大学 Southwest Jiaotong University

收敛判断标准

差值阈值判断：迭代后取新旧PR差值绝对值最大值，小于阈值(10^{-6}/10^{-8})即收敛，阈值越小精度越高、迭代越多。

迭代次数上限：设置100–500次最大迭代数，避免极端不收敛导致程序卡死。



特殊节点处理

悬挂节点：
无出链，PR 值均分至所有页面；

孤立节点：
无任何连接，PR 迭代后衰减至 0

05 PageRank阻尼因子深入理解

阻尼因子的作用

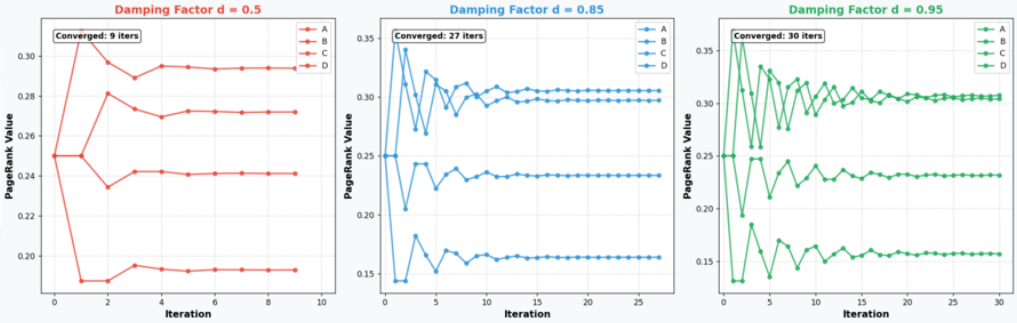
阻尼因子d（通常设为0.85）是PageRank算法中最关键的参数，它模拟了用户继续浏览的概率：

d = 0.85的含义：

1. 用户有85%的概率继续点击当前页面上的链接浏览
2. 用户有15%的概率"厌倦"跳转到一个完全随机的页面

为什么选择0.85?

1. 太低（如0.5）：PR值趋于均匀分布，无法区分页面重要性
2. 太高（如0.95）：收敛速度极慢，且对悬挂节点和噪声链接过于敏感
3. 0.85是一个经过实践检验的平衡点



学术引用网络 d 取 0.9-0.95（引用忠诚）
社交网络 d 取 0.8-0.85（跳转频繁）
网页搜索 d 设 0.85（行业标准）。

06 PageRank应用场景

西南交通大学 Southwest Jiaotong University

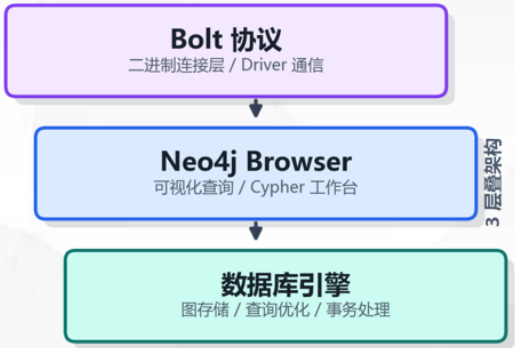
01 Neo4j概述

西南交通大学 Southwest Jiaotong University

什么是Neo4j?

核心定位:

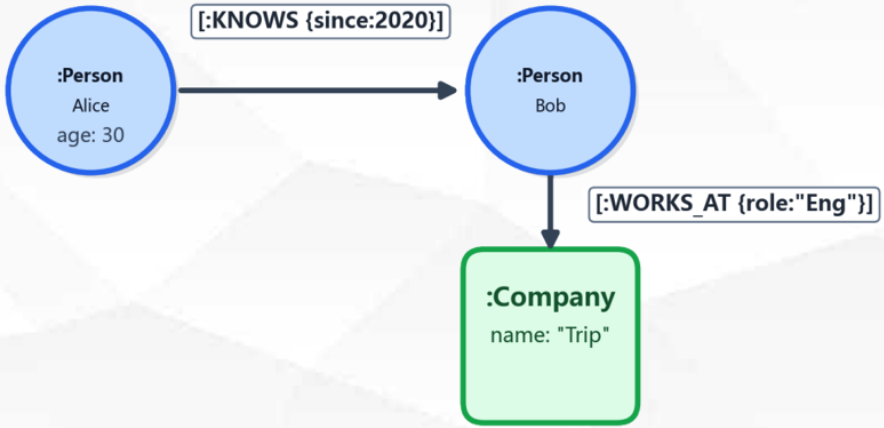
1. 专为存储和查询图数据设计的数据库
2. 使用**Cypher**查询语言（类SQL的图查询语言）
3. 支持大规模图数据的存储、查询和管理
4. 提供完整的**ACID**事务支持



03 属性图模型详解

西南交通大学 Southwest Jiaotong University

Neo4j 属性图：标签 + 属性 + 关系类型 + 方向



Cypher查询语言基础

MATCH - 模式匹配

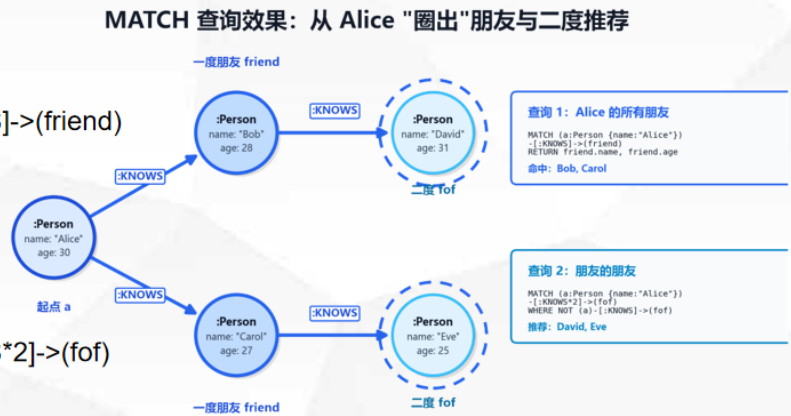
MATCH是Cypher中最核心的命令，用于在图中查找符合模式的节点和关系：

查询：找出Alice的所有朋友

```
MATCH (a:Person {name:"Alice"})-[:KNOWS]->(friend)
RETURN friend.name, friend.age
```

朋友的朋友（二度关系）

```
MATCH (a:Person {name:"Alice"})-[:KNOWS*2]->(fof)
WHERE NOT (a)-[:KNOWS]->(fof)
RETURN DISTINCT fof.name AS 推荐好友
```



05 Cypher查询语言基础

西南交通大学 Southwest Jiaotong University

RETURN - 返回结果

指定查询结果中包含哪些字段：

返回节点属性

```
MATCH (p:Person)
RETURN p.name, p.age
```

聚合统计：平均年龄

```
MATCH (p:Person)
RETURN COUNT(p) AS 总人数, AVG(p.age) AS 平均年龄
```

去重统计：有多少种城市

```
MATCH (p:Person)
RETURN COUNT(DISTINCT p.city) AS 城市数
```

CREATE - 创建数据

创建节点（无标签）

```
CREATE (n {name:"Alice", age:30})
```

创建带标签的节点

```
CREATE (n:Person {name:"Bob", age:25, city:"成都"})
```

创建关系（先MATCH找到节点，再CREATE关系）

```
MATCH (a:Person {name:"Alice"}), (b:Person {name:"Bob"})
CREATE (a)-[:KNOWS {since:2020}]->(b)
```

Cypher查询语言（续）

Cypher 写操作：UPDATE + DELETE + MERGE

UPDATE

```
MATCH (p:Person {name:"Alice"})
SET p.age = 31
RETURN p
```

效果：修改 Alice 的 age

DELETE

```
MATCH (p:Person {name:"Bob"})
DETACH DELETE p
```

效果：删除 Bob 及其关系

MERGE

```
MERGE (p:Person {name:"Alice"})
ON CREATE SET p.createdAt = datetime()
ON MATCH SET p.lastSeen = datetime()
RETURN p
```

效果：避免重复创建

07 图模式匹配与路径查询

西南交通大学 Southwest Jiaotong University

最短路径查询

Neo4j提供了shortestPath函数，可以高效地查找两点间的最短路径：

多跳路径查询

Neo4j 提供了变长路径语法 `[:KNOWS*]`、`[:KNOWS*3]` 和 `[:KNOWS*1..3]`，可以灵活查询多跳关系，用于发现固定跳数路径或一定范围内的 n 度人脉。

Cypher 路径查询：最短路径 + 多跳路径

蓝色高亮表示 shortestPath 实际命中的 Alice -> Bob -> Carol

查询 1：最短路径

```
MATCH path = shortestPath(
  (a:Person {name:"Alice"})-[:KNOWS*]-
  (b:Person {name:"Carol"})
)
RETURN path, length(path) AS distance
```

结果：Alice -> Bob -> Carol，distance = 2

查询 2：恰好经过 3 条 KNOWS 关系

```
MATCH (a:Person)-[:KNOWS*3]->(b:Person)
WHERE a <> b
RETURN a.name AS start_name,
       b.name AS end_name
```

含义：*3 表示恰好 3 跳

查询 3：1 到 3 跳以内可达用户

```
MATCH (a:Person {name:"Alice"})
-[:KNOWS*1..3]->(b:Person)
RETURN DISTINCT b.name AS reachable_user
```

含义：*1..3 更适合 n 度人脉场景

图模式匹配与路径查询

聚合与统计查询

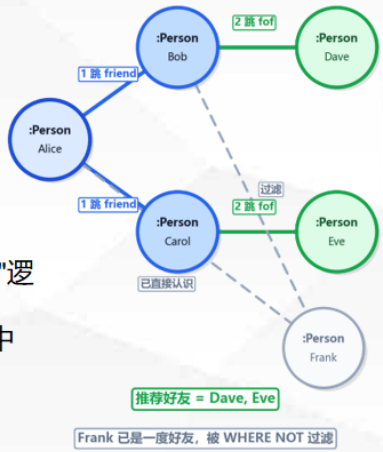
Cypher 支持使用 `COUNT()`、`ORDER BY` 等聚合与排序语法，对图中的关系数量进行统计，例如统计每个人主动关注了多少朋友，或被多少人关注。

朋友的朋友查询

通过多层关系匹配实现"推荐"逻辑：
这个查询可以用于社交网络中的"你可能认识的人"推荐

朋友的朋友推荐 + 聚合统计

从 Alice 的一度朋友继续扩展一跳，得到二度好友推荐；同时用 COUNT 做关注数统计



统计 1：主动关注的朋友数 outgoing

```
MATCH (p:Person)-[:KNOWS]->(friend)
RETURN p.name, COUNT(friend) AS friend_count
ORDER BY friend_count DESC
```

含义：每个人主动连出去多少条 KNOWS

统计 2：被多少人关注 incoming

```
MATCH (p:Person)<-[:KNOWS]-(follower)
RETURN p.name, COUNT(follower) AS follower_count
ORDER BY follower_count DESC
```

含义：每个人被多少人指向

推荐逻辑：朋友的朋友（二度好友）

```
MATCH (a:Person {name:"Alice"})
-[:KNOWS]->(friend)-[:KNOWS]->(fof)
WHERE NOT (a)-[:KNOWS]->(fof)
  AND fof <> a
RETURN DISTINCT fof.name AS recommended_friend
```

结果：Dave, Eve；排除已直接认识的人

11 NetworkX vs Neo4j 深度对比

西南交通大学 Southwest Jiaotong University

定位差异

维度	NetworkX	Neo4j
产品类型	Python库/分析工具	图数据库管理系统
计算模式	内存计算	持久化存储+计算
数据规模	小规模（<10万节点）	大规模（数十亿节点）
数据持久化	无（重启丢失）	有（ACID事务）
查询方式	API函数调用	Cypher查询语言
并发支持	单线程	多用户并发
适用场景	算法验证、原型开发	生产系统、实时查询

“决策口诀"——「教学/原型→NetworkX；生产/持久化→Neo4j；超大规模→GraphX」

西南交通大學
Southwest Jiaotong University

云计算技术

第12讲 云原生平台

授课教师：丛荣

课程代码：SCAI004712

2026年6月

为什么需要Helm?

- Kubernetes 原生对象繁多：Deployment、Service、ConfigMap、Ingress、RBAC
- 一个真实应用通常不是一个 YAML，而是一组相互关联的 YAML
- 手工维护多套环境配置时，配置漂移会快速累积

Helm Charts 帮助你定义、安装和升级即使是最复杂的 Kubernetes 应用程序。
Charts 易于创建、版本控制、共享和发布，所以开始使用 Helm，停止复制粘贴。

Wordpress应用程序中涉及到的kubernetes API对象

Helm的作用：Kubernetes的包管理器

西南交通大学 Southwest Jiaotong University

- **应用打包：**把一组 Kubernetes 资源组织成 Chart
- **参数化部署：**通过 values.yaml 适配 dev / test / prod
- **一键部署：**通过 helm install 将模板渲染后提交给 API Server
- **版本管理：**Chart 有版本，Release 也有历史版本

```
helm install myapp ./chart -f prod.yaml
```

Chart 结构：一个应用包里有什么？

- Chart.yaml：描述 Chart **元信息**，例如名称、版本、依赖
- values.yaml：提供**默认参数**，模板渲染的数据来源
- charts/：存放依赖 Chart，可选
- templates/：存放 Kubernetes **资源模板**

```
myapp/
├── Chart.yaml
├── values.yaml
├── charts/
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    └── ingress.yaml
```

Chart = 模板 + 参数 + 元信息

Chart 结构：Chart.yaml

Chart.yaml

```
apiVersion: v2
name: myapp
description: A Helm chart for Kubernetes
type: application
version: 0.1.0
appVersion: "1.0.0"
dependencies:
  - name: redis
    version: 18.0.0
    repository: https://charts.bitnami.com/bitnami
```

- version：Chart 包自身的版本
- appVersion：应用程序的版本
- dependencies：声明依赖组件
- Chart.yaml 让应用包具备可追踪性

Chart 结构：values.yaml

Values.yaml

```
replicaCount: 2  #副本数
image:  # 镜像
  repository: nginx
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
resources:  #资源限制上界
  limits:
    cpu: 500m
    memory: 512Mi
```

核心思想

- values.yaml 不直接提交给 Kubernetes
- 它为 templates/ 中的占位符提供值
- 同一个 Chart 可以通过不同 values 部署到 dev / test / prod
- 参数化是 Helm 复用能力的基础

Chart 结构：templates/

```
    spec:
      serviceAccount: deis-database
      containers:
        - name: deis-database
          image:
{{ .Values.imageRegistry }}/postgres:{{ .Values.dockerTag }}
          imagePullPolicy: {{ .Values.pullPolicy }}
          ports:
            - containerPort: 5432
          env:
            - name: DATABASE_STORAGE
              value: {{ default "minio" .Values.storage }}
```

- imageRegistry: The source registry for the Docker image.
- dockerTag: The tag for the docker image.
- pullPolicy: The Kubernetes pull policy.
- storage: The storage backend, whose default is set to "minio"

模板不是最终 YAML，渲染后才是 Kubernetes 资源。

多环境部署：同一个Chart，不同Values

西南交通大学 Southwest Jiaotong University

DEV

replicas=1
image=dev
service=NodePort

TEST

replicas=2
image=rc
resource=medium

PROD

replicas=5
image=stable
ingress=enabled

一套 Chart 通过不同 values 文件渲染出不同环境的部署结果

00 本讲内容大纲

西南交通大学 Southwest Jiaotong University

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

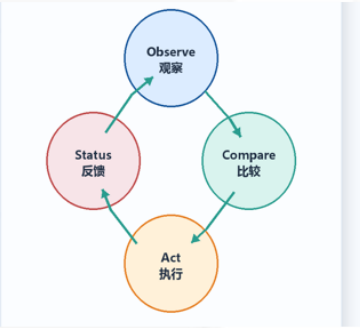
03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

为什么需要 Operator?

- Kubernetes 能维持通用资源的期望状态
  但不了解数据库、监控、大数据作业等领域知识
- 复杂应用需要初始化、备份、扩容、故障迁移和状态反馈
- Operator 把运维经验写成控制器，让系统持续自动收敛
- **Operator = CRD + Controller + 领域运维知识**



CRD：自定义资源

- CRD 全称 Custom Resource Definition，用于扩展 Kubernetes API
- CRD 定义资源类型
- Custom Resource 创建资源实例
- 用户像操作 Deployment 一样操作业务对象
- 控制器监听这些对象，并根据 spec 推动系统状态变化

CRD 定义方式

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: widgets.example.com
spec:
  group: example.com
  versions:
  - name: v1
    served: true
    storage: true
  scope: Namespaced
  names:
    plural: widgets
    singular: widget
    kind: Widget
```

- group + version 组成 API 路径: example.com/v1
- kind 资源类型名称
  plural 是 kubectl 访问名
- scope 决定资源是 Namespaced 还是 Cluster 级别

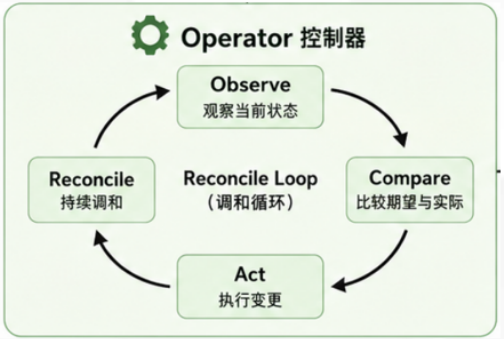
Custom Resource：用户提交的期望状态


```yaml
apiVersion: example.com/v1
kind: Widget
metadata:
  name: demo-widget
spec:
  size: 3
  image: nginx:1.25
status:
  phase: Running
  readyReplicas: 3
```

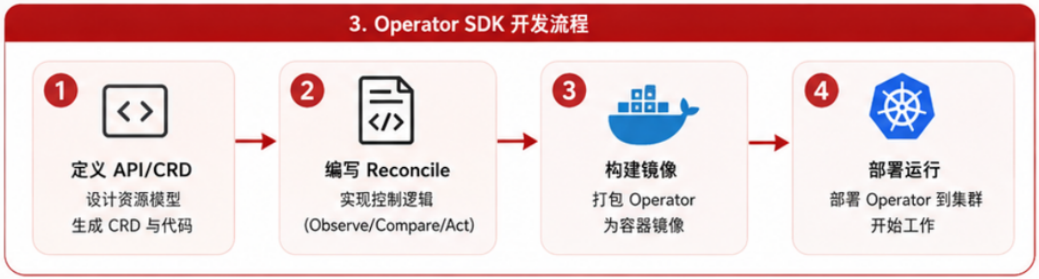
- spec：用户希望系统达到的状态
- status：控制器观测到并写回的实际状态
- spec/status 分离是声明式 API 的关键设计
- “定义资源类型” V.S. “创建资源实例”

Controller：Reconcile Loop

- 控制器不断比较期望状态与实际状态
- 发现差异后执行动作修正系统
- 动作完成后写回 status，并等待下一次事件或周期同步
- Controller 不应假设自己只运行一次，而要能重复执行
- 每次 Reconcile 都应尽量做到幂等：重复执行不会破坏系统



Operator SDK 开发流程




```
operator-sdk init --domain example.com --repo github.com/demo/op
operator-sdk create api --group apps --version v1 --kind WebApp
# edit api/v1/webapp_types.go
# edit controllers/webapp_controller.go
make manifests
make docker-build docker-push IMG=repo/webapp-operator:v1
make deploy IMG=repo/webapp-operator:v1
```

常见 Operator 实例

Operator	管理对象	典型能力
Prometheus Operator	Prometheus / ServiceMonitor	监控系统的自动化管理
MySQL Operator	MySQLCluster / Backup	有状态应用的自动化，数据库生命周期管理
Spark Operator	SparkApplication	Driver/Executor Pod 创建、作业状态管理
Cert-Manager	Certificate / Issuer	证书申请、续期和分发

Operator 的共同点：把某类系统的“运维动作”变成控制器逻辑。

为什么需要把 Spark 跑到 Kubernetes?

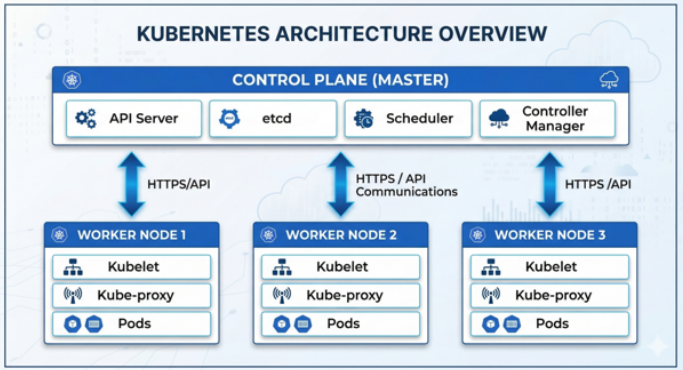
西南交通大学 Southwest Jiaotong University

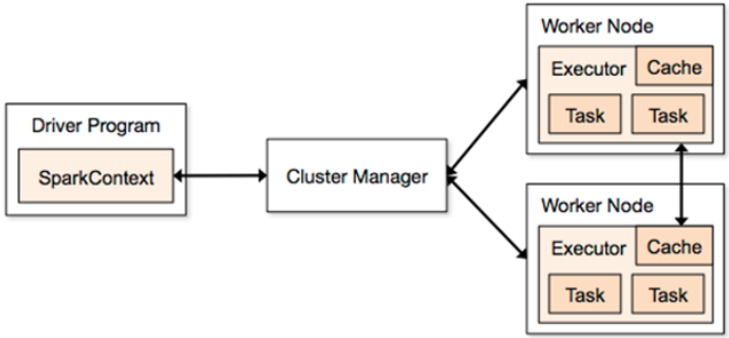
- 传统环境：Spark 运行在 Standalone 或 YARN 集群上
- 云原生环境：企业希望计算任务共享 Kubernetes 资源池
- K8S提供任务生命周期、资源隔离、镜像交付和弹性能力

Spark原生K8s支持的架构：Driver Pod + Executor Pod

西南交通大学 Southwest Jiaotong University

- Driver Pod 负责作业控制、DAG调度与任务分发
- Executor Pod 负责执行具体任务并汇报结果
- Kubernetes 负责 Pod 创建、资源分配与节点调度



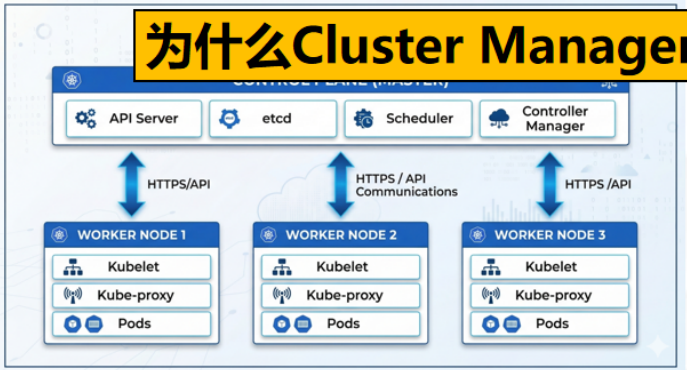


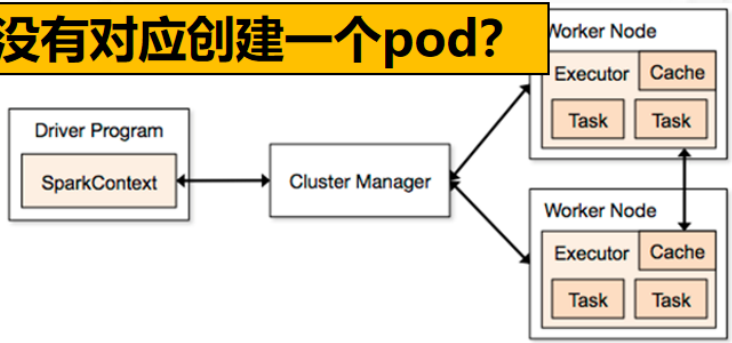
Spark on Kubernetes 让 Driver 和 Executor 以 Pod 形式运行

Spark原生K8s支持的架构：Driver Pod + Executor Pod

- Driver Pod 负责作业控制、DAG调度与任务分发
- Executor Pod 负责执行具体任务并汇报结果
- Kubernetes 负责 Pod 创建、资源分配与节点调度

为什么Cluster Manager没有对应创建一个pod?

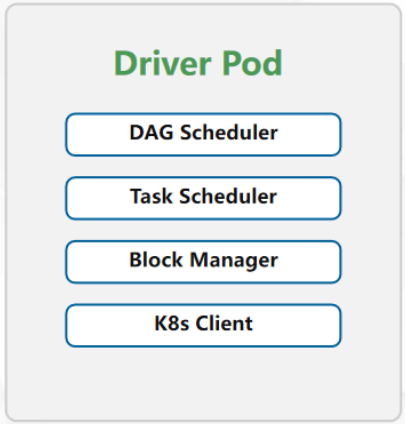




Spark on Kubernetes 让 Driver 和 Executor 以 Pod 形式运行

Driver Pod

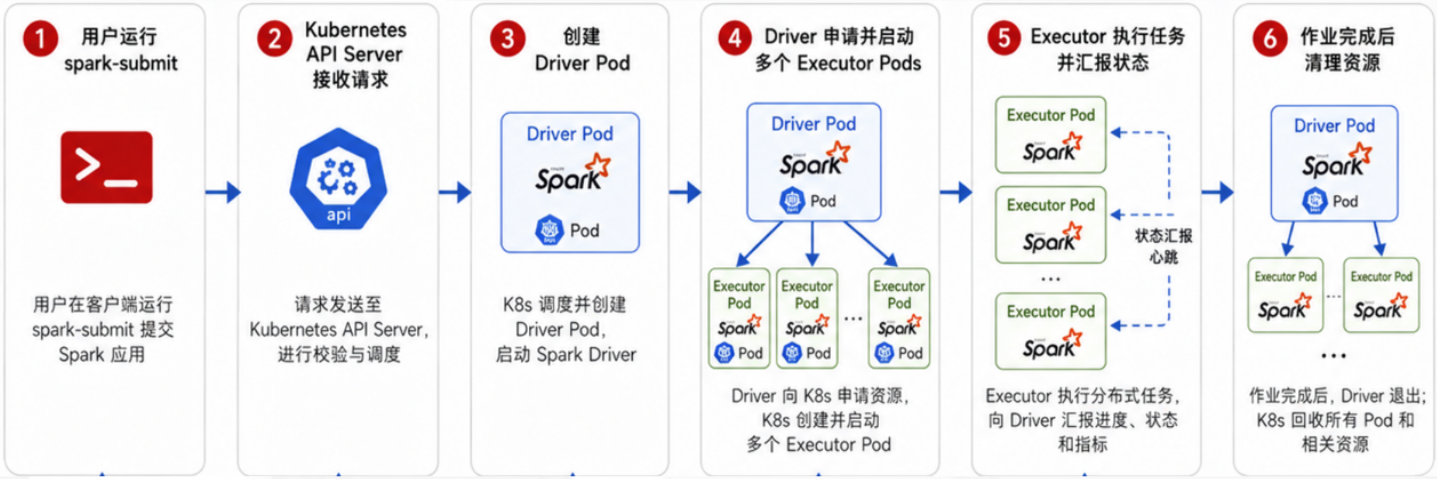
- 运行用户程序的 main 函数
- 创建 SparkContext / SparkSession
- 向 Kubernetes 请求 Executor Pod
- 把 Job 切分成 Stage 和 Task 并分发执行
- 维护任务状态、失败重试和作业生命周期



Executor Pod

- 每个 Executor Pod 运行一个 Spark Executor
- Executor 内部可以用多线程运行多个 Task
- Executor 使用内存缓存数据，也可写 Shuffle 临时文件
- Executor 的生命周期由 Driver 和 Kubernetes 共同管理
- **Executor Pod 越多，并行度越强，但资源开销也越大。**

Spark Operator执行流程

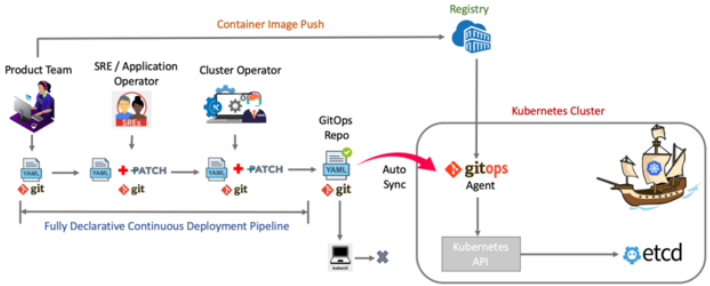


Spark on K8s vs Spark on YARN

	Spark on YARN	Spark on Kubernetes
生态基础	Hadoop 生态、HDFS、YARN 队列	容器平台、镜像、Namespace
资源隔离	YARN 队列与 Container	Namespace、ResourceQuota、Pod
部署方式	依赖 Hadoop 集群环境	依赖容器镜像与 K8s 集群
适合场景	已有 Hadoop 基础设施	云原生数据平台与统一调度

为什么需要 GitOps?

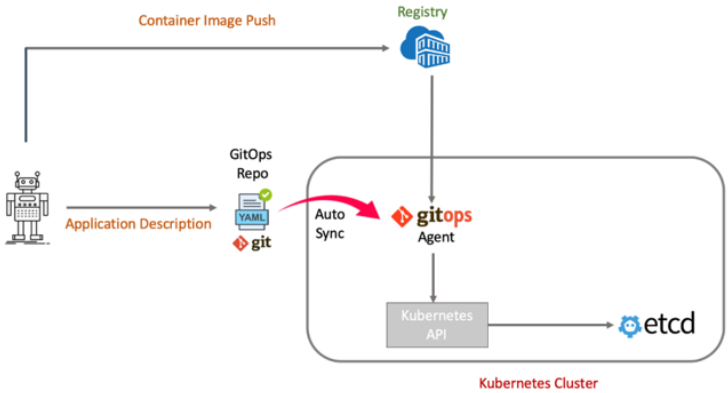
- 手工 kubectl apply 容易导致线上状态不可追踪
- 多人协作时，需要知道谁改了什么、何时改、如何回滚
- Git 天然具备版本管理、审查、分支和回滚能力
- GitOps 把这些能力用于基础设施和应用部署



GitOps的核心理念与标准工作流

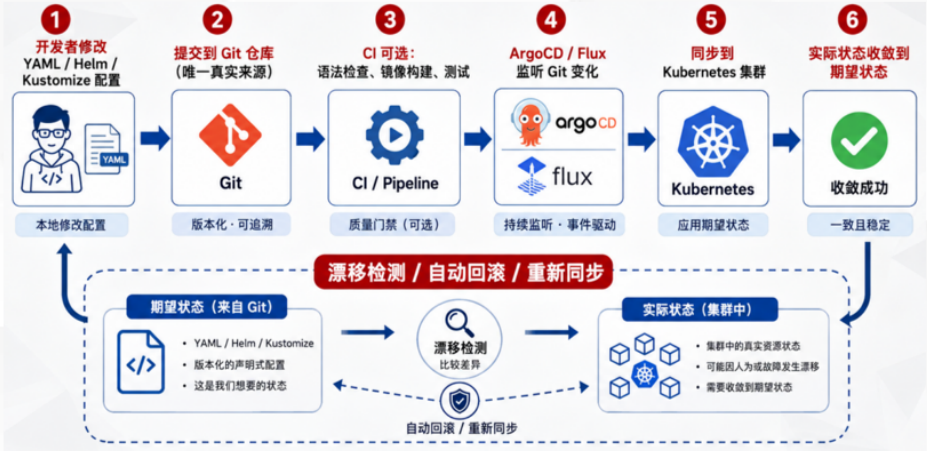
Git作为唯一的真实来源

- **声明式配置：**期望状态以 YAML / Helm Chart / Kustomize 等形式保存在 Git 中
- **任何变更先进入 Git**，再由控制器同步到集群
- 集群中被手工修改的状态会被检测为 **Drift**
- **开发者主动定义声明式配置，即定义期望状态**



ArgoCD

- ArgoCD是个自动化控制工具，可以定期监听比较 Git 期望状态和集群实际状态
- 状态监听与检测：状态一致为 Synced，存在差异为 OutOfSync
- 自动同步：将期望状态应用到集群实际状态
- 漂移修复：Sync 操作把差异同步到 Kubernetes 集群



Flux

- Flux也是一个GitOps自动化控制工具，为K8S集群的持续交付而设计
- 由多个控制器组成
  - source-controller：获取 Git / Helm 源
  - kustomize-controller：应用 Kustomize 配置
  - helm-controller：管理 Helm Release
  - notification-controller：事件通知
- Flux是Git原生，原生支持与Kustomize/Helm集成